



OXETECH Academy



SECTI
Secretaria de Estado de
Tecnologia e Inovação



 Softex

REGISTERED BY
AGÊNCIA REGULADORA
DE PROTEÇÃO

CONSUMIDOR FEDERAL
BRASIL
CONSUMIDOR PROTEGIDO

Cronograma



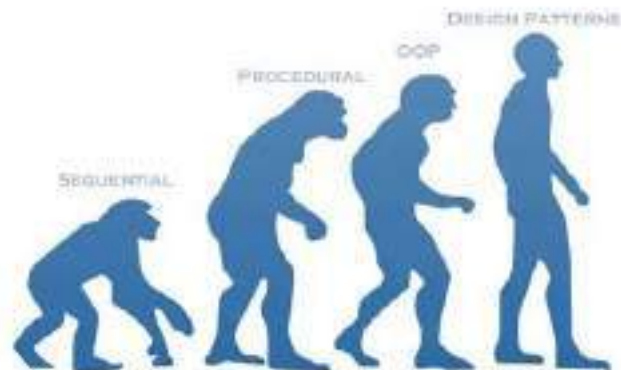
Módulos	Conteúdo
Módulo 1 — Fundamentos da Engenharia Moderna	Processos, versionamento e code review Gestão de dependências, Documentação viva
Módulo 2 — Clean Code e SOLID	Legibilidade, funções e acoplamento, SOLID aplicado, Refatoração incremental
Módulo 3 — Back -end com Node.js	Factory, Strategy, Observer, Repository, Adapter, Decorator, Anti -patterns comuns

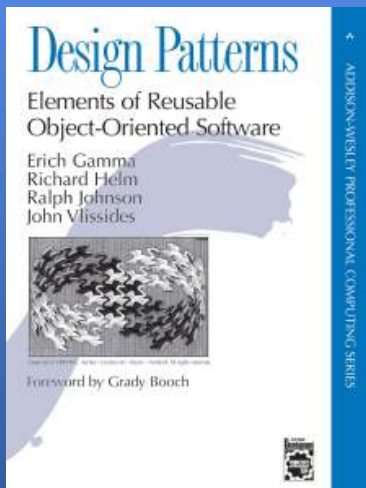
Módulos	Conteúdo
Módulo 4 — Arquiteturas e Escalabilidade	Monólito modular, hexagonal, Microserviços e mensageria, DDD básico
Módulo 5 — Qualidade, Testes e Segurança	Pirâmide de testes e cobertura, Sonar, linters, quality gates, Segurança básica em apps
Módulo 6 — CI/ CD e Operação	Pipelines (build/ test/ deploy), Blue green, canary e rollback, Observabilidade (logs/ metrics/ traces)
Módulo 7 — Projeto Final Integrador	Sistema modular com integração contínua, Qualidade e observabilidade integradas, Defesa técnica

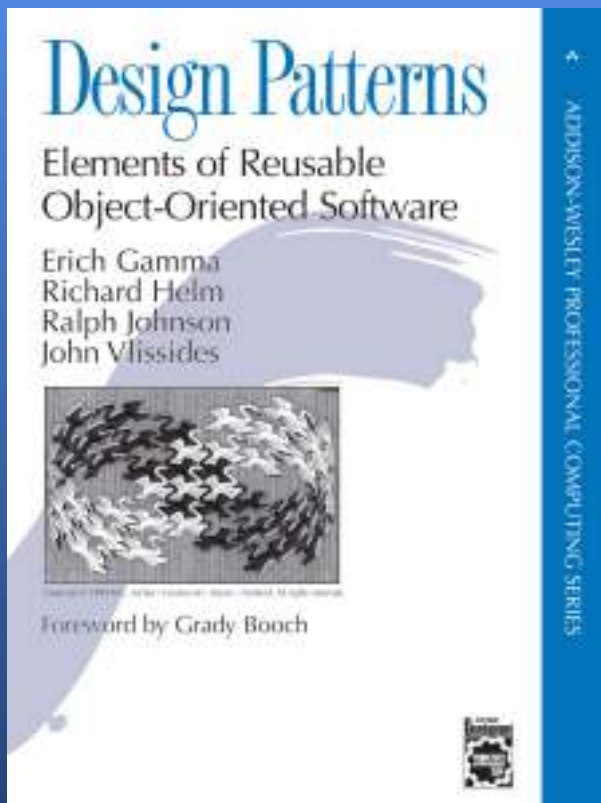
Revisão



Design Patterns: o guia definitivo dos Padrões de Projeto







Design patterns ou padrões de design são soluções já testadas para problemas recorrentes no desenvolvimento de software, que deixam seu código mais manutenível e elegante, pois essas soluções se baseiam em um baixo acoplamento.

Qual a diferença entre Design Patterns e Clean Code ou SOLID?



Design Patterns (padrões de projeto) são soluções reutilizáveis para problemas comuns de design, focando em *como estruturar* objetos. SOLID e Clean Code são princípios e práticas focados na *qualidade, legibilidade e manutenção* do código.

Diferenças Principais

Design Patterns (Padrões de Projeto)

São "modelos" ou "receitas" para resolver problemas recorrentes de design de software, como o padrão *Singleton*, *Factory* ou *Strategy*. Eles focam na estrutura do código.

SOLID:

Um conjunto de 5 princípios da orientação a objetos que visa tornar o código mais coeso, extensível e fácil de manter. Eles funcionam como diretrizes de arquitetura para evitar acoplamento.

Clean Code (Código Limpo):

Uma filosofia e conjunto de boas práticas focadas em legibilidade, simplicidade e facilidade de manutenção. Clean Code se preocupa com nomes de variáveis, funções pequenas e estrutura de arquivos.

Diferenças Principais

Design Patterns

Estrutura e organização de objetos
Reutilizar soluções para problemas conhecidos.

SOLID

Princípios de Design Orientado a Objetos
Aumentar manutenibilidade e flexibilidade.

Clean Code

Legibilidade e simplicidade do código
Facilitar a compreensão e manutenção por humanos.

Qual a diferença entre Design Patterns e Arquitetura de Software?



A principal diferença é o escopo: Arquitetura de Software define a estrutura geral, alto nível e componentes do sistema (o "esqueleto"), enquanto Design Patterns (padrões de projeto) são soluções reutilizáveis para problemas específicos e recorrentes no nível do código (a "montagem" de partes). A arquitetura foca em requisitos não funcionais (escalabilidade, segurança) e o design na organização interna..

Arquitetura de Software (Macro)	Design Patterns (Micro)
Foco: Estrutura geral, componentes, bibliotecas, banco de dados e comunicação entre sistemas.	Foco: Organização de classes e objetos para resolver problemas específicos.
Objetivo: Atender requisitos não funcionais, como escalabilidade, desempenho e manutenibilidade.	Objetivo: Melhorar a flexibilidade, legibilidade e reutilização do código, evitando reescrever soluções comuns.
Exemplos: Arquitetura em Camadas, Microserviços, Serverless, Event-Driven.	Exemplos: Singleton, Factory Method, Observer, Strategy.
Analogia: O projeto arquitetônico de uma casa (onde ficam as paredes, encanamento e rede elétrica)	Analogia: Como assentar os tijolos ou instalar as tomadas específicas para funcionar melhor

Principais Diferenças

Escopo: A arquitetura olha o todo, o Design Pattern olha para uma parte específica.

Abstração: Design patterns estão mais próximos do código-fonte; arquitetura está mais próxima da infraestrutura.

Impacto: Decisões arquiteturais são difíceis de mudar; padrões de design podem ser ajustados com menos esforço.

Para que serve Design Patterns?



O uso de Design Patterns deixará seu código mais fácil de ser mantido e testado, além da segurança de serem soluções já testadas repetidamente e que foram adotadas por terem dado certo. E mais, as discussões técnicas serão mais fáceis tendo em vista que todos estarão falando uma mesma língua, sem esquecer, é claro, de um código mais elegante.

Quais os benefícios de usar Design Patterns?



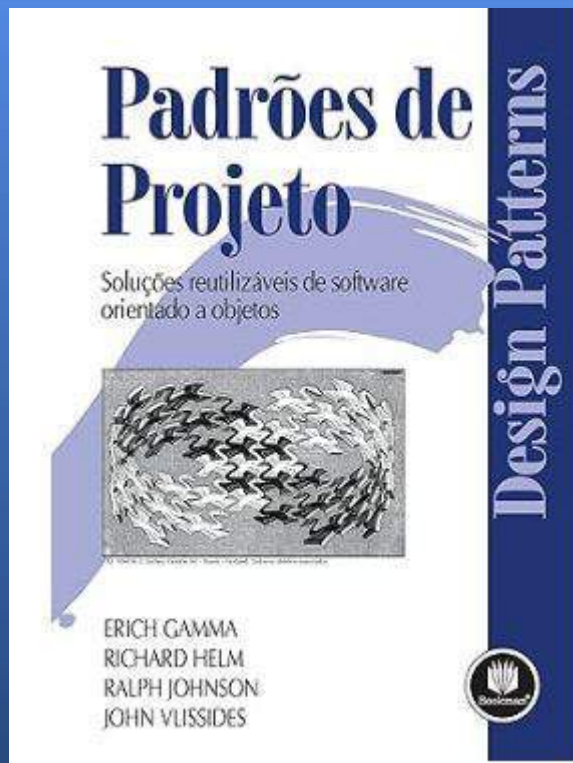
Design patterns são modelos que já foram utilizados e testados anteriormente, portanto podem representar um bom ganho de produtividade para os desenvolvedores.

Seu uso também contribui para a organização e manutenção de projetos, já que esses padrões se baseiam em baixo acoplamento entre as classes e padronização do código.

Além disso, com a padronização dos termos, as discussões técnicas são facilitadas. É mais fácil falar o nome de um design pattern em vez de ter que explicar todo o seu comportamento.

Design Patterns GoF





Os Padrões de Projeto da Gangue dos Quatro (GOF, na sigla em inglês) referem-se a um conjunto de 23 padrões fundamentais de projeto de software, introduzidos por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides em seu livro seminal, "Design Patterns: Elements of Reusable Object-Oriented Software" (Padrões de Projeto: Elementos de Software Orientado a Objetos Reutilizável).

Padrões de Criação



Os padrões de projeto criacionais concentram-se no processo de criação de objetos no desenvolvimento de software. Esses padrões garantem que desenvolvamos as coisas de uma maneira fácil e adaptável, permitindo atualizá-las posteriormente, se necessário. Eles ocultam os detalhes complexos de como as coisas são montadas.

Padrões Estruturais



Os padrões estruturais lidam com a composição ou estrutura dos objetos. Eles ajudam a garantir que, se uma parte de um sistema mudar, o sistema inteiro não precise mudar junto.

Padrões Comportamentais



Os padrões comportamentais focam na comunicação entre objetos, definindo como os objetos interagem e colaboram para realizar tarefas.

Vamos conhecer os padrões?



C	Abstract Factory	S	Facade	S	Proxy
S	Adapter	C	Factory Method	B	Observer
S	Bridge	S	Flyweight	C	Singleton
C	Builder	B	Interpreter	B	State
B	Chain of Responsibility	B	Iterator	B	Strategy
B	Command	B	Mediator	B	Template Method
S	Composite	B	Memento	B	Visitor
S	Decorator	C	Prototype		

Cada letra representa uma família de pattern por exemplo : Letra C = **Creational Design Patterns** / Letra B= **Behavioral Patterns** / Letra S= **Structural Design Patterns**

Padrões Criacionais - Abstract Factory



O Abstract Factory é um padrão de projeto criacional que permite que você produza famílias de objetos relacionados sem ter que especificar suas classes concretas.



Case 1 - Abstract Factory



Imagine que você está criando um simulador de loja de móveis. Seu código consiste de classes que representam:

1. Uma família de produtos relacionados, como: Cadeira + Sofá + MesaDeCentro.
2. Várias variantes dessa família. Por exemplo, produtos Cadeira + Sofá + MesaDeCentro estão disponíveis nessas variantes: Moderno, Vitoriano, ArtDeco.

Você precisa de um jeito de criar objetos de mobília individuais para que eles combinem com outros objetos da mesma família. Os clientes ficam muito bravos quando recebem mobília que não combina.



Problemática



E ainda, você não quer mudar o código existente quando adiciona novos produtos ou famílias de produtos ao programa. Os vendedores de mobílias atualizam seus catálogos com frequência e você não vai querer mudar o código base cada vez que isso acontece.

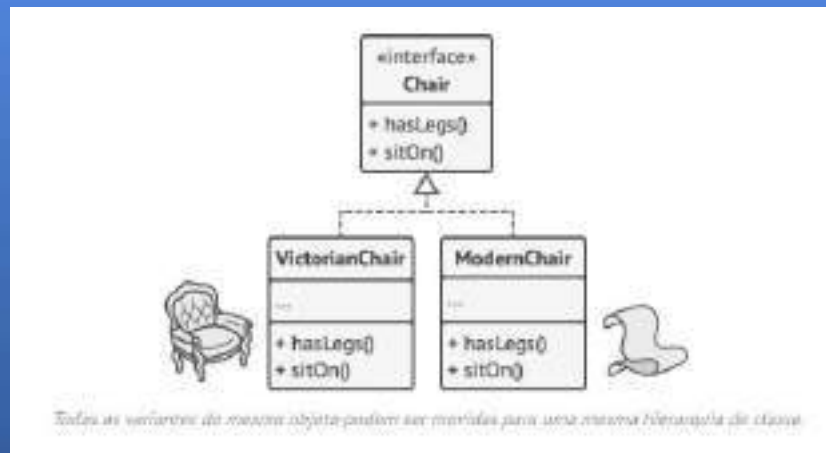


Um sofá no estilo Moderno não combina com cadeiras de estilo Vitoriano.

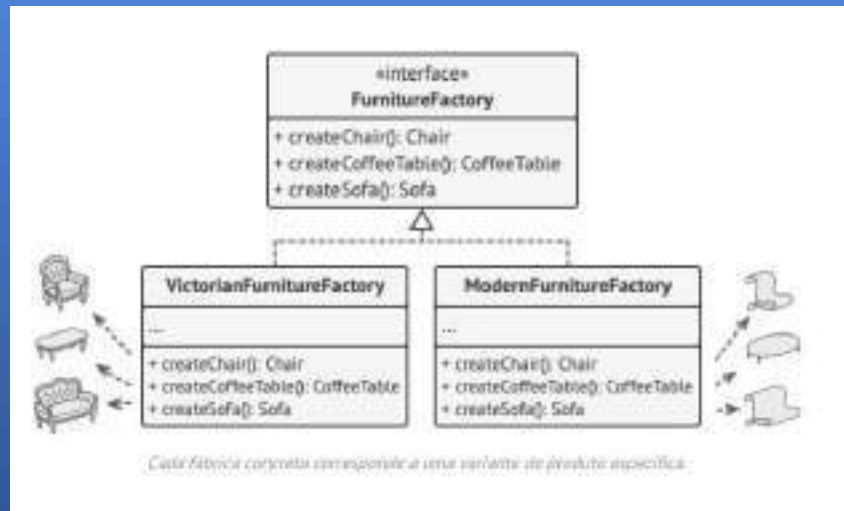
Case 1 - Solução

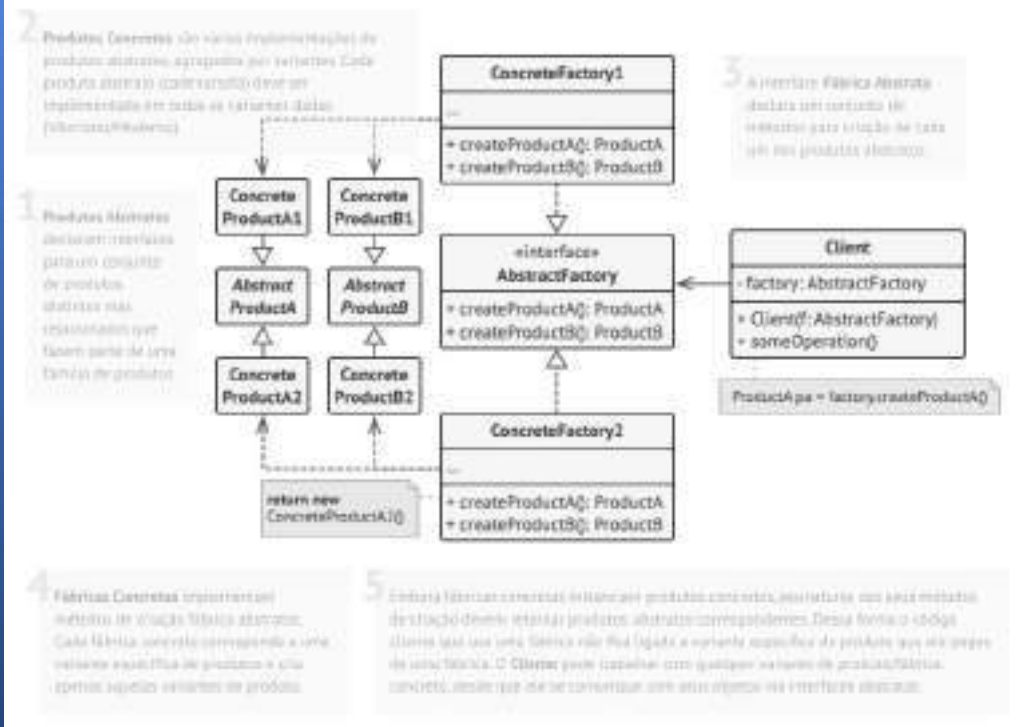


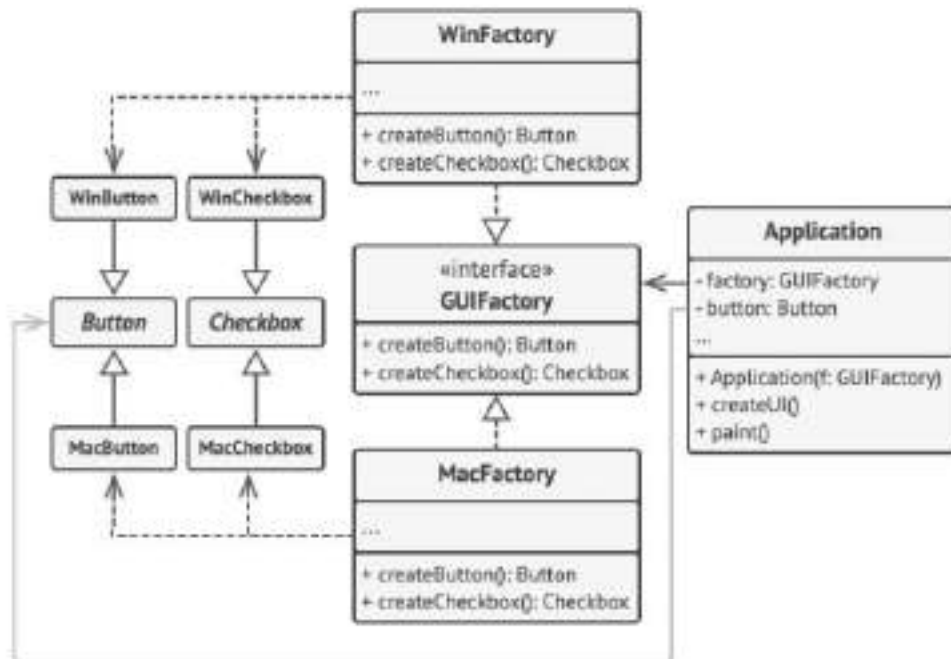
A primeira coisa que o padrão Abstract Factory sugere é declarar explicitamente interfaces para cada produto distinto da família de produtos (ex: cadeira, sofá ou mesa de centro). Então você pode fazer todas as variantes dos produtos seguirem essas interfaces. Por exemplo, todas as variantes de cadeira podem implementar a interface `Cadeira`; todas as variantes de mesa de centro podem implementar a interface `MesaDeCentro`, e assim por diante.



O próximo passo é declarar a *fábrica abstrata*—uma interface com uma lista de métodos de criação para todos os produtos que fazem parte da família de produtos (por exemplo, criarCadeira, criarSofá e criarMesaDeCentro). Esses métodos devem retornar tipos abstratos de produtos representados pelas interfaces que extraímos previamente: *Cadeira*, *Sofá*, *MesaDeCentro* e assim por diante.







Exemplo das classes UI multiplataforma.

Aplicabilidade - Abstract Factory



- Use o Abstract Factory quando seu código precisa trabalhar com diversas famílias de produtos relacionados, mas que você não quer depender de classes concretas daqueles produtos-eles podem ser desconhecidos de antemão ou você simplesmente quer permitir uma futura escalabilidade.
- O Abstract Factory fornece a você uma interface para a criação de objetos de cada classe das famílias de produtos. Desde que seu código crie objetos a partir dessa interface, você não precisará se preocupar em criar uma variante errada de um produto que não coincida com produtos já criados por sua aplicação.
- Considere implementar o Abstract Factory quando você tem uma classe com um conjunto de métodos fábrica que desfoquem sua responsabilidade principal.
- Em um programa bem desenvolvido *cada classe é responsável por apenas uma coisa*. Quando uma classe lida com múltiplos tipos de produto, pode valer a pena extrair seus métodos fábrica em uma classe fábrica solitária ou uma implementação plena do Abstract Factory

Como implementar - Abstract Factory

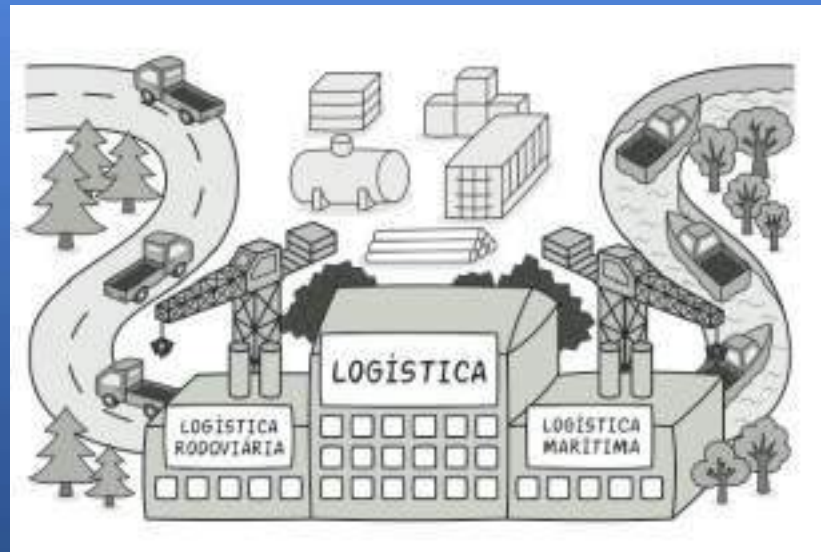


1. Mapeie uma matriz de tipos de produtos distintos versus as variantes desses produtos.
2. Declare interfaces de produto abstratas para todos os tipos de produto. Então, faça todas as classes concretas de produtos implementar essas interfaces.
3. Declare a interface da fábrica abstrata com um conjunto de métodos de criação para todos os produtos abstratos.
4. Implemente um conjunto de classes fábricas concretas, uma para cada variante de produto.
5. Crie um código de inicialização da fábrica em algum lugar da aplicação. Ele deve instanciar uma das classes fábrica concretas, dependendo da configuração da aplicação ou do ambiente atual. Passe esse objeto fábrica para todas as classes que constroem produtos.
6. Escaneie o código e encontre todas as chamadas diretas para construtores de produtos. Substitua-as por chamadas para o método de criação apropriado no objeto fábrica.

Padrões Criacionais - Factory Method



O Factory Method é um padrão criacional de projeto que fornece uma interface para criar objetos em uma superclasse, mas permite que as subclasses alterem o tipo de objetos que serão criados.

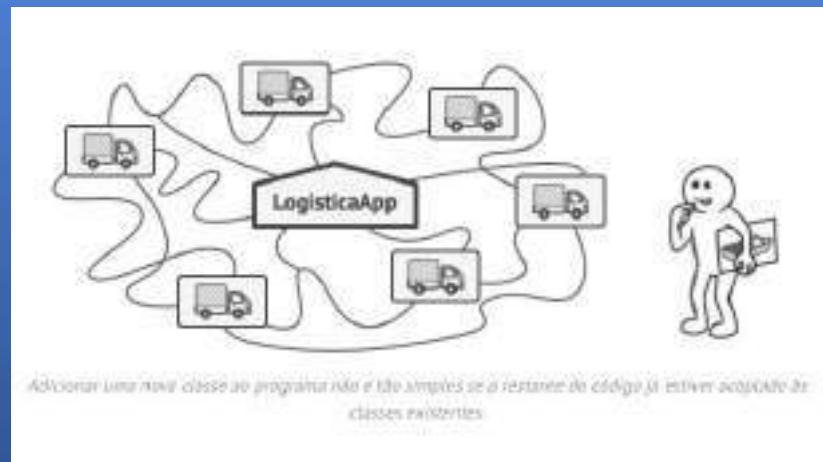


Problemática



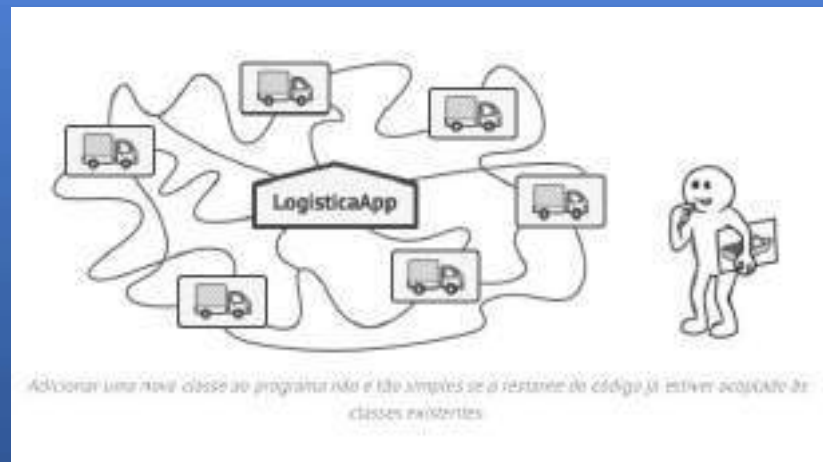
Imagine que você está criando uma aplicação de gerenciamento de logística. A primeira versão da sua aplicação pode lidar apenas com o transporte de caminhões, portanto a maior parte do seu código fica dentro da classe *Caminhão*.

Depois de um tempo, sua aplicação se torna bastante popular. Todos os dias você recebe dezenas de solicitações de empresas de transporte marítimo para incorporar a logística marítima na aplicação.



Boa notícia, certo? Mas e o código? Atualmente, a maior parte do seu código é acoplada à classe Caminhão. *Adicionar Navio à aplicação exigiria alterações em toda a base de código.* Além disso, se mais tarde você decidir adicionar outro tipo de transporte à aplicação, provavelmente precisará fazer todas essas alterações novamente.

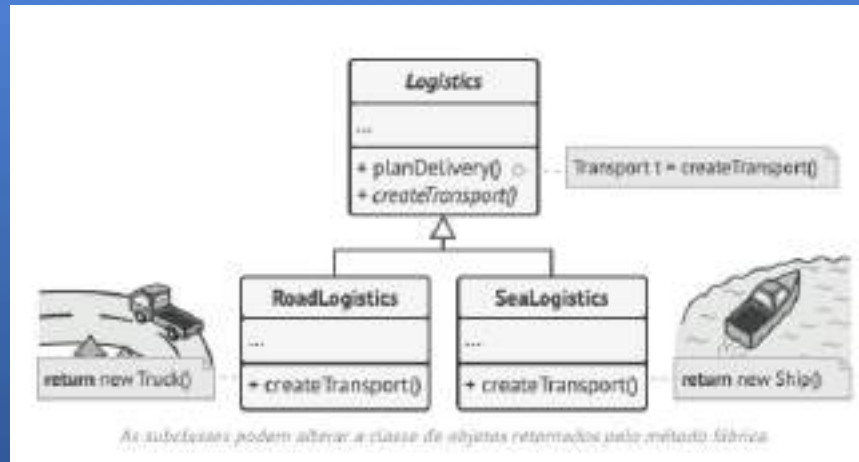
Como resultado, você terá um código bastante sujo, repleto de condicionais que alteram o comportamento da aplicação, dependendo da classe de objetos de transporte.



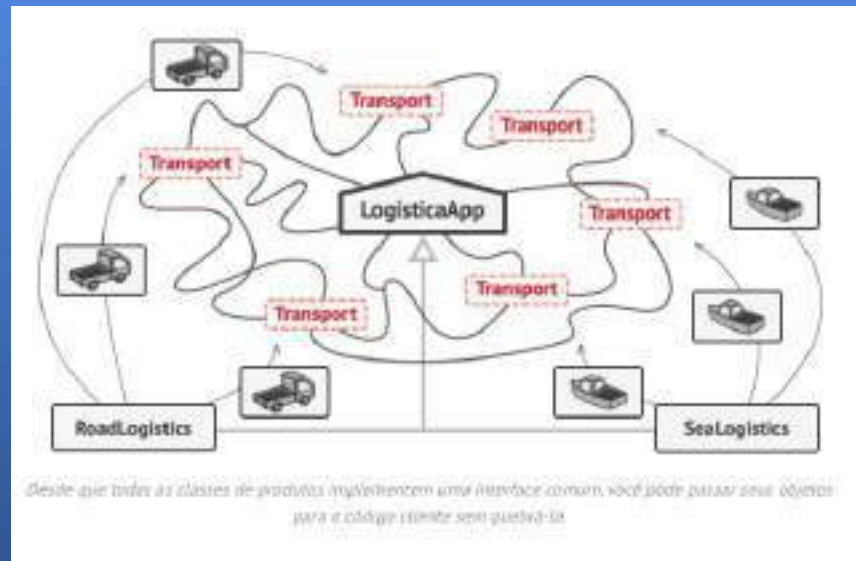
Case 1 - Solução



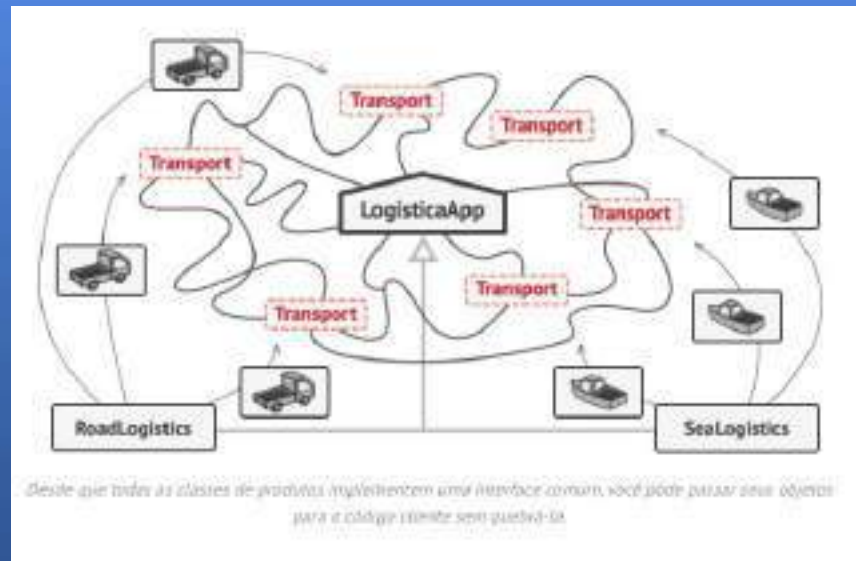
O padrão Factory Method sugere que você substitua chamadas diretas de construção de objetos (usando o operador new) por chamadas para um método *fábrica* especial. Não se preocupe: os objetos ainda são criados através do operador new, mas esse está sendo chamado de dentro do método fábrica. Objetos retornados por um método fábrica geralmente são chamados de *produtos*.



Por exemplo, ambas as classes Caminhão e Navio devem implementar a interface Transporte, que declara um método chamado entregar. Cada classe implementa esse método de maneira diferente: caminhões entregam carga por terra, navios entregam carga por mar. O método fábrica na classe LogísticaViária retorna objetos de caminhão, enquanto o método fábrica na classe LogísticaMarítima retorna navios.



O código que usa o método fábrica (geralmente chamado de código *cliente*) não vê diferença entre os produtos reais retornados por várias subclasses. O cliente trata todos os produtos como um Transporte abstrato. O cliente sabe que todos os objetos de transporte devem ter o método entregar, mas como exatamente ele funciona não é importante para o cliente.

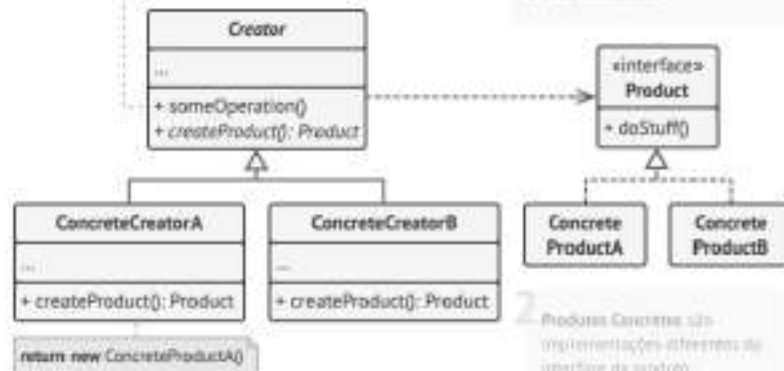


3. A classe `Creator` declara o método fábrica que retorna novos objetos produto. É importante que o tipo de retorno dessa método corresponda a interface do produto.

Well pode declarar o método fábrica como abstrato para forçar todas as subclasses a implementar suas próprias versões do método. Como alternativa, o método `Make` base pode retornar algum tipo de produto padrão.

Observe que, apesar do nome, a criação de produtos não é a principal responsabilidade da classe. Normalmente, a classe `Creator` já possui alguma lógica de negócios relacionada a um produto. O método fábrica aqui é apenas uma maneira de criar os objetos necessários do sistema. Aqui é responsável por uma grande porção da implementação de software para ter um comportamento de comportamento para proporcionar fluidez, e principal função de empresa com um todo ainda é resolver todos os problemas em produção.

```
Product p = createProduct();
p.doStuff();
```



4. Criadores Concretos sobrescrevem o método fábrica para criar instâncias de um tipo diferente de produto.

Observe que o método fábrica não precisa criar novos objetos a todo o tempo. Ele também pode retornar objetos existentes de um cache, um pool de objetos, ou outra fonte.

1. O Produto declara a interface que é comum a todos os objetos que podem ser produzidos pelo criador e suas subclasses.

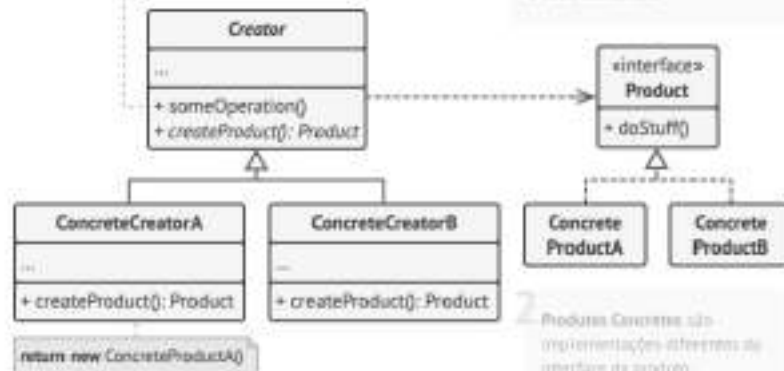
2. Produtos Concretos são implementações diferentes da interface do produto.

3. A classe `Creator` declara o método fábrica que retorna novos objetos produto. É importante que o tipo de retorno dessa método corresponda a interface do produto.

Well pode declarar o método fábrica como abstrato para forçar todas as subclasses a implementar suas próprias versões do método. Como alternativa, o método `Make` base pode retornar algum tipo de produto padrão.

Observe que, apesar do nome, a criação de produtos não é a principal responsabilidade da classe. Normalmente, a classe `Creator` já possui alguma lógica de negócios relacionada a um produto. O método fábrica aqui é apenas uma maneira fácil de criar um novo instance do produto. Aqui é responsável por uma grande porção da implementação de software para ter um comportamento de comportamento para proporcionar. No entanto, a principal função de empresa com o código ainda é escrever código que produza um produto.

```
Product p = createProduct();
p.doStuff();
```



4. Criadores Concretos sobrescrevem o método fábrica para criar instâncias de um tipo diferente de produto.

Observe que o método fábrica não precisa criar novos instances a todo tempo. Ele também pode retornar objetos existentes de um cache, um pool de objetos, ou outra fonte.

1. O Produto declara a interface que é comum a todos os objetos que podem ser produzidos pelo criador e suas subclasses.

2. Produtos Concretos são implementações diferentes da interface do produto.

Aplicabilidade - Factory Method



Use o Factory Method quando não souber de antemão os tipos e dependências exatas dos objetos com os quais seu código deve funcionar.

O Factory Method separa o código de construção do produto do código que realmente usa o produto. Portanto, é mais fácil estender o código de construção do produto independentemente do restante do código.

Use o Factory Method quando desejar fornecer aos usuários da sua biblioteca ou framework uma maneira de estender seus componentes internos.

Herança é provavelmente a maneira mais fácil de estender o comportamento padrão de uma biblioteca ou framework. Mas como o framework reconheceria que sua subclasse deve ser usada em vez de um componente padrão?

A solução é reduzir o código que constrói componentes no framework em um único método fábrica e permitir que qualquer pessoa sobrescreva esse método, além de estender o próprio componente.

Vamos ver como isso funcionaria. Imagine que você escreva uma aplicação usando um framework de UI de código aberto. Sua aplicação deve ter botões redondos, mas o framework fornece apenas botões quadrados. Você estende a classe padrão Botão com uma gloriosa subclasse BotãoRedondo. Mas agora você precisa informar à classe principal UIFramework para usar a nova subclasse no lugar do botão padrão.

Para conseguir isso, você cria uma subclasse UIComBotõesRedondos a partir de uma classe base do framework e sobrescreve seu método criarBotão. Enquanto este método retorna objetos Botão na classe base, você faz sua subclasse retornar objetos BotãoRedondo. Agora use a classe UIComBotõesRedondos no lugar de UIFramework. E é isso!

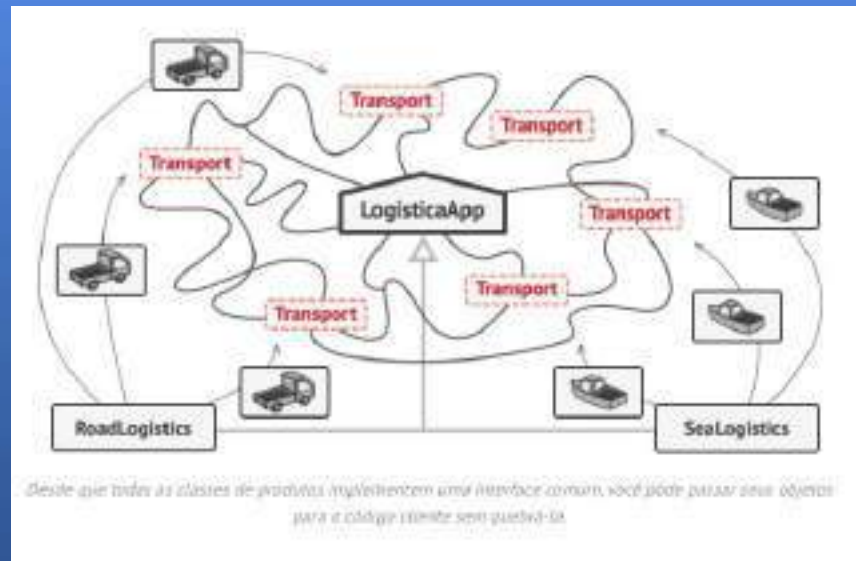
Como implementar - Factory Method



- Faça todos os produtos implementarem a mesma interface. Essa interface deve declarar métodos que fazem sentido em todos os produtos.
- Adicione um método fábrica vazio dentro da classe criadora. O tipo de retorno do método deve corresponder à interface comum do produto.
- No código da classe criadora, encontre todas as referências aos construtores de produtos. Um por um, substitua-os por chamadas ao método fábrica, enquanto extrai o código de criação do produto para o método fábrica. Pode ser necessário adicionar um parâmetro temporário ao método fábrica para controlar o tipo de produto retornado. Neste ponto, o código do método fábrica pode parecer bastante feio. Pode ter um grande operador `switch` que escolhe qual classe de produto instanciar. Mas não se preocupe, resolveremos isso em breve.

- Agora, crie um conjunto de subclasses criadoras para cada tipo de produto listado no método fábrica. Sobrescreva o método fábrica nas subclasses e extraia os pedaços apropriados do código de construção do método base.
- Se houver muitos tipos de produtos e não fizer sentido criar subclasses para todos eles, você poderá reutilizar o parâmetro de controle da classe base nas subclasses. Por exemplo, imagine que você tenha a seguinte hierarquia de classes: a classe base Correio com algumas subclasses: CorreioAéreo e CorreioTerrestre; as classes Transporte são Avião, Caminhão e Trem. Enquanto a classe CorreioAéreo usa apenas objetos Avião, o CorreioTerrestre pode funcionar com os objetos Caminhão e Trem. Você pode criar uma nova subclasse (por exemplo, CorreioFerroviário) para lidar com os dois casos, mas há outra opção. O código do cliente pode passar um argumento para o método fábrica da classe CorreioTerrestre para controlar qual produto ele deseja receber.
- Se, após todas as extrações, o método fábrica base ficar vazio, você poderá torná-lo abstrato. Se sobrar algo, você pode tornar isso em um comportamento padrão do método.

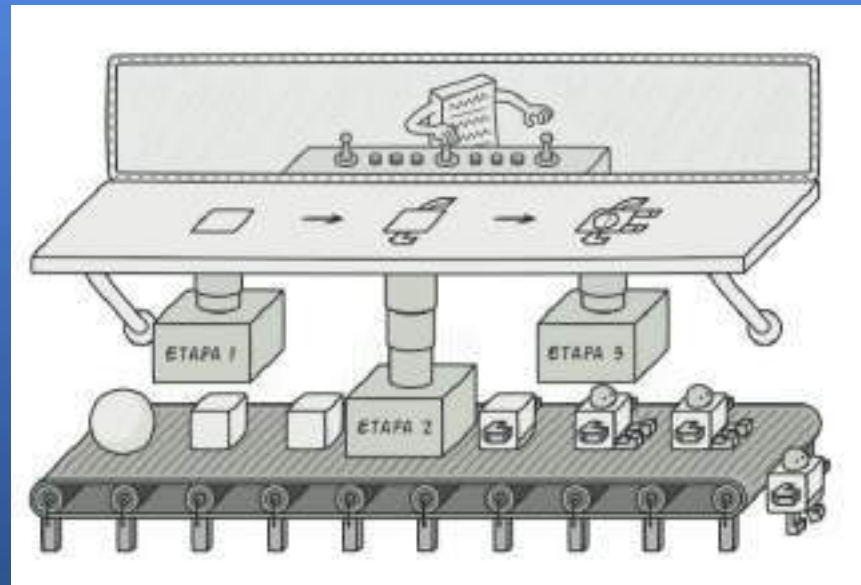
O código que usa o método fábrica (geralmente chamado de código *cliente*) não vê diferença entre os produtos reais retornados por várias subclasses. O cliente trata todos os produtos como um Transporte abstrato. O cliente sabe que todos os objetos de transporte devem ter o método entregar, mas como exatamente ele funciona não é importante para o cliente.



Padrões Criacionais - Builder



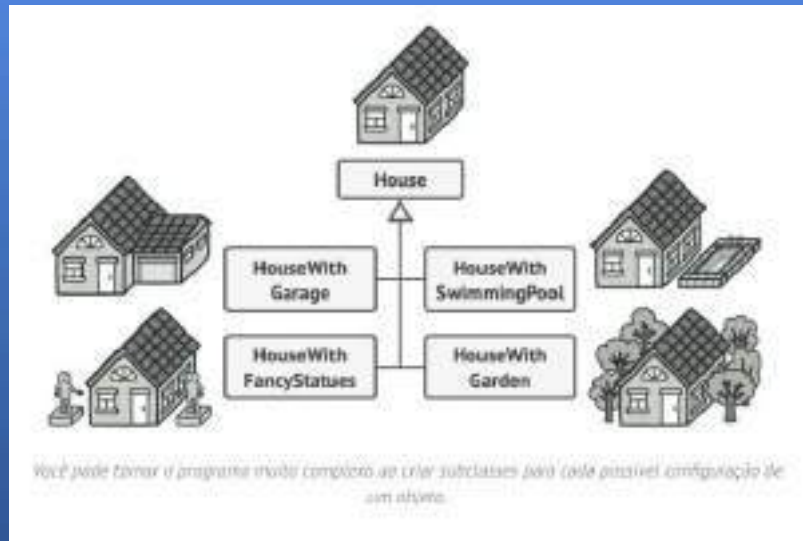
O Builder é um padrão de projeto criacional que permite a você construir objetos complexos passo a passo. O padrão permite que você produza diferentes tipos e representações de um objeto usando o mesmo código de construção.



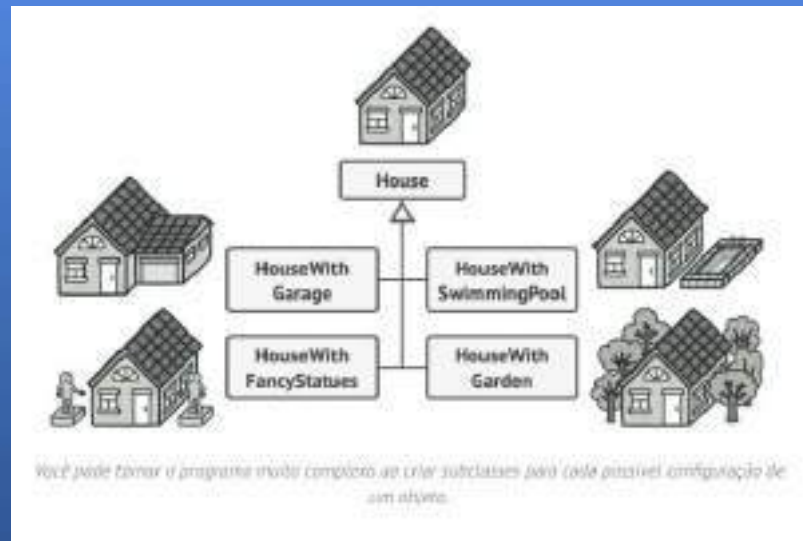
Problemática



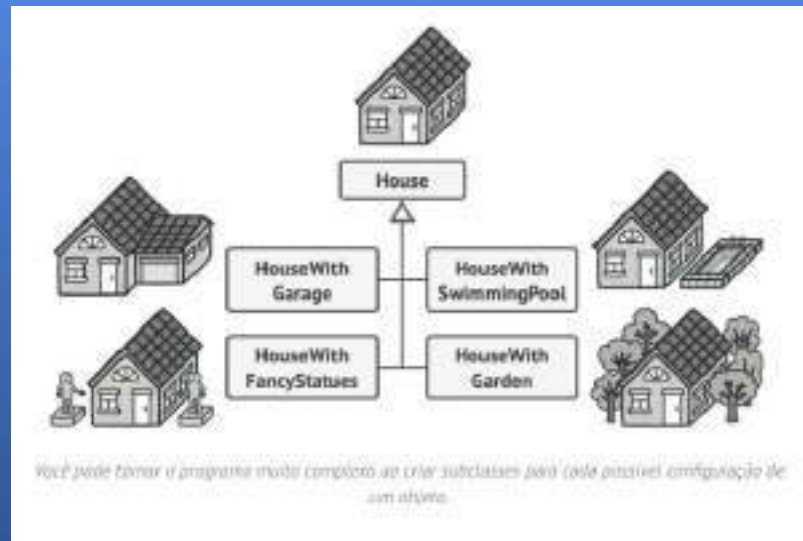
Imagine um objeto complexo que necessite de uma inicialização passo a passo trabalhosa de muitos campos e objetos agrupados. Tal código de inicialização fica geralmente enterrado dentro de um construtor monstruoso com vários parâmetros. Ou pior: espalhado por todo o código cliente.



Por exemplo, vamos pensar sobre como criar um objeto Casa. Para construir uma casa simples, você precisa construir quatro paredes e um piso, instalar uma porta, encaixar um par de janelas, e construir um teto. Mas e se você quiser uma casa maior e mais iluminada, com um jardim e outras miudezas (como um sistema de aquecimento, encanamento, e fiação elétrica)?

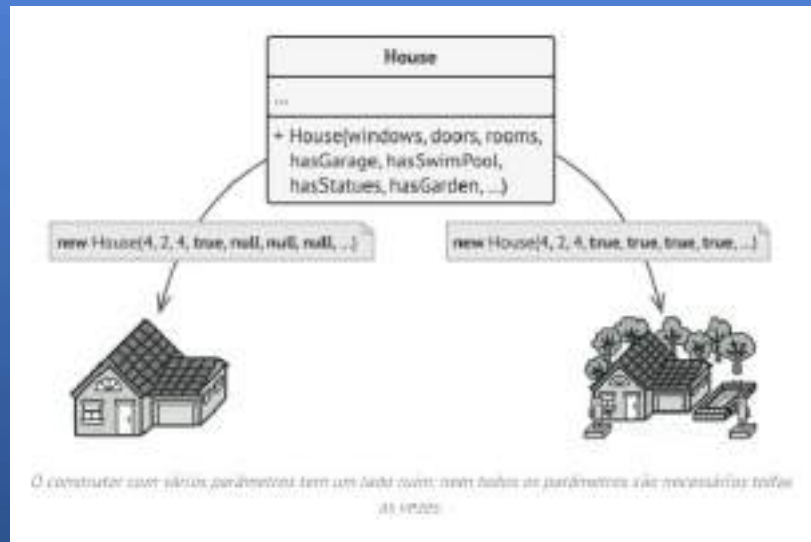


A solução mais simples é estender a classe base Casa e criar um conjunto de subclasses para cobrir todas as combinações de parâmetros. Mas eventualmente você acabará com um número considerável de subclasses. Qualquer novo parâmetro, tal como o estilo do pórtico, irá forçá-lo a aumentar essa hierarquia cada vez mais.



Há outra abordagem que não envolve a propagação de subclasses. Você pode criar um construtor gigante diretamente na classe Casa base com todos os possíveis parâmetros que controlam o objeto casa. Embora essa abordagem realmente elimine a necessidade de subclasses, ela cria outro problema.

Na maioria dos casos a maioria dos parâmetros não será usada, tornando as chamadas do construtor em algo feio de se ver. Por exemplo, apenas algumas casas têm piscinas, então os parâmetros relacionados a piscinas serão inúteis nove em cada dez vezes.

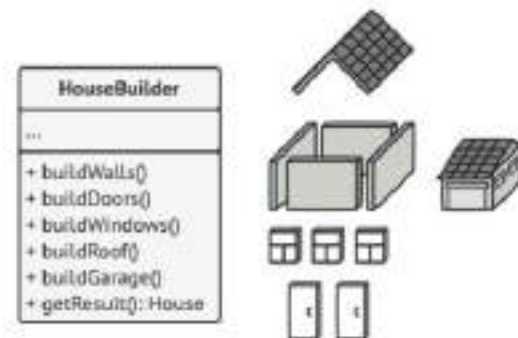


Case 1 - Solução

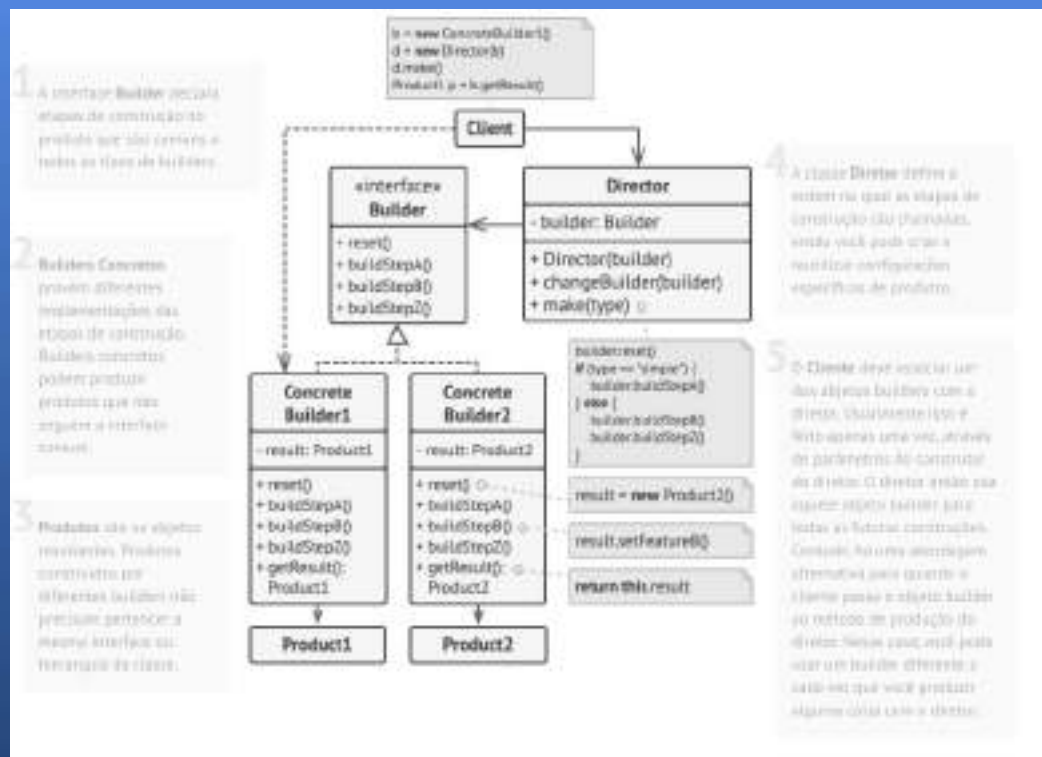


O padrão Builder sugere que você extraia o código de construção do objeto para fora de sua própria classe e mova ele para objetos separados chamados *builders*. “Builder” significa “construtor”, mas não usaremos essa palavra para evitar confusão com os construtores de classe.

O padrão organiza a construção de objetos em uma série de etapas (construirParedes, construirPorta, etc.). Para criar um objeto você executa uma série de etapas em um objeto builder. A parte importante é que você não precisa chamar todas as etapas. Você chama apenas aquelas etapas que são necessárias para a produção de uma configuração específica de um objeto.



O padrão Builder permite que você construa objetos complexos passo a passo. O Builder não permite que outros objetos acessem o produto enquanto ele está sendo construído.



Aplicabilidade - Builder



Use o padrão Builder para se livrar de um “construtor telescópico”.

Use o padrão Builder quando você quer que seu código seja capaz de criar diferentes representações do mesmo produto (por exemplo, casas de pedra e madeira).

O padrão Builder pode ser aplicado quando a construção de várias representações do produto envolvem etapas similares que diferem apenas nos detalhes.

A interface base do builder define todas as etapas de construção possíveis, e os builders concretos implementam essas etapas para construir representações particulares do produto. Enquanto isso, a classe diretor guia a ordem de construção.

Use o Builder para construir árvores Composite ou outros objetos complexos.

O padrão Builder permite que você construa produtos passo a passo. Você pode adiar a execução de algumas etapas sem quebrar o produto final. Você pode até chamar etapas recursivamente, o que é bem útil quando você precisa construir uma árvore de objetos.

Um builder não expõe o produto não finalizado enquanto o processo de construção estiver executando etapas. Isso previne o código cliente de obter um resultado incompleto.

Como implementar - Builder



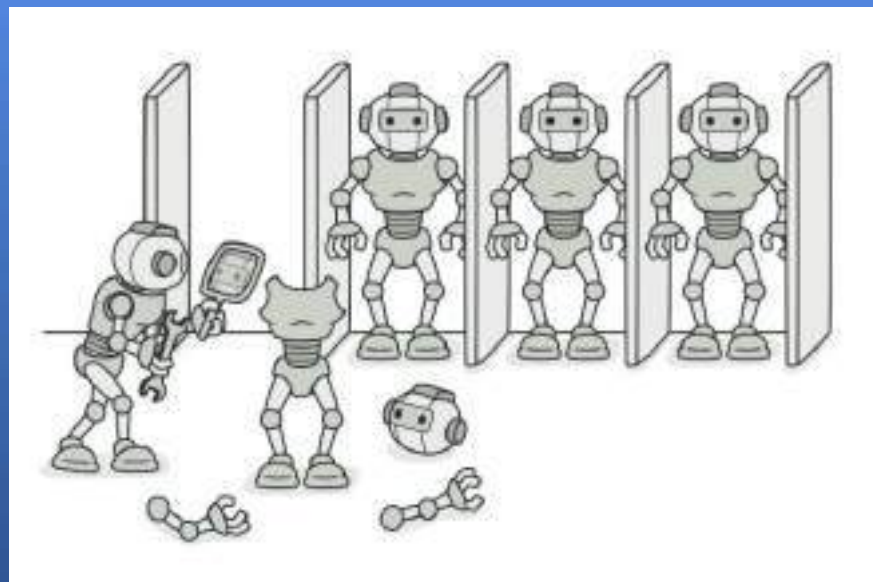
- Certifique-se que você pode definir claramente as etapas comuns de construção para construir todas as representações do produto disponíveis. Do contrário, você não será capaz de implementar o padrão.
- Declare essas etapas na interface builder base.
- Crie uma classe builder concreta para cada representação do produto e implemente suas etapas de construção.
- Não se esqueça de implementar um método para recuperar os resultados da construção. O motivo pelo qual esse método não pode ser declarado dentro da interface do builder é porque vários builders podem construir produtos que não tem uma interface comum. Portanto, você não sabe qual será o tipo de retorno para tal método. Contudo, se você está lidando com produtos de uma única hierarquia, o método de obtenção pode ser adicionado com segurança para a interface base.

- Pense em criar uma classe diretor. Ela pode encapsular várias maneiras de construir um produto usando o mesmo objeto builder.
- O código cliente cria tanto os objetos do builder como do diretor. Antes da construção começar, o cliente deve passar um objeto builder para o diretor. Geralmente o cliente faz isso apenas uma vez, através de parâmetros do construtor do diretor. O diretor usa o objeto builder em todas as construções futuras. Existe uma alternativa onde o builder é passado diretamente ao método de construção do diretor.
- O resultado da construção pode ser obtido diretamente do diretor apenas se todos os produtos seguirem a mesma interface. Do contrário o cliente deve obter o resultado do builder

Padrões Criacionais - Prototype



O Prototype é um padrão de projeto criacional que permite copiar objetos existentes sem fazer seu código ficar dependente de suas classes.



Problemática



Digamos que você tenha um objeto, e você quer criar uma cópia exata dele. Como você o faria? Primeiro, você tem que criar um novo objeto da mesma classe. Então você terá que ir por todos os campos do objeto original e copiar seus valores para o novo objeto.

Legal! Mas tem uma pegadinha. Nem todos os objetos podem ser copiados dessa forma porque alguns campos de objeto podem ser privados e não serão visíveis fora do próprio objeto.



*Copiar um objeto "do lado de fora" **nem sempre** é possível.*

Há ainda mais um problema com a abordagem direta. Uma vez que você precisa saber a classe do objeto para criar uma cópia, seu código se torna dependente daquela classe. Se a dependência adicional não te assusta, tem ainda outra pegadinha. Algumas vezes você só sabe a interface que o objeto segue, mas não sua classe concreta, quando, por exemplo, um parâmetro em um método aceita quaisquer objetos que seguem uma interface.



Copiar um objeto "do lado de fora" nem sempre é possível.

Case 1 - Solução



O padrão Prototype delega o processo de clonagem para o próprio objeto que está sendo clonado. O padrão declara um interface comum para todos os objetos que suportam clonagem. Essa interface permite que você clone um objeto sem acoplar seu código à classe daquele objeto. Geralmente, tal interface contém apenas um único método clonar.



Pré construir protótipos pode ser uma alternativa às subclasses.

A implementação do método `clonar` é muito parecida em todas as classes. O método cria um objeto da classe atual e carrega todos os valores de campo para do antigo objeto para o novo. Você pode até mesmo copiar campos privados porque a maioria das linguagens de programação permite objetos acessar campos privados de outros objetos que pertençam a mesma classe.

Um objeto que suporta clonagem é chamado de um *protótipo*. Quando seus objetos têm dúzias de campos e centenas de possíveis configurações, cloná-los pode servir como uma alternativa às subclasses.

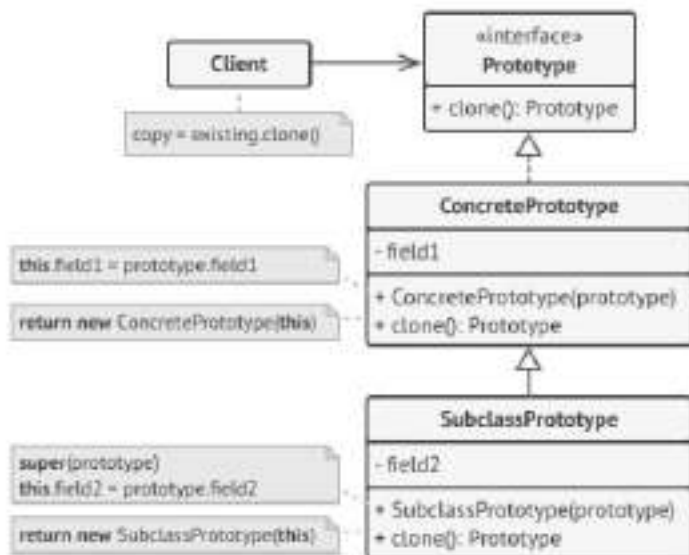


Pré construir protótipos pode ser uma alternativa às subclasses.

Implementação básica

3 O Cliente pode produzir uma cópia de qualquer objeto que segue a interface do protótipo.

1 A interface Protótipo declara os métodos de clonagem. Na maioria dos casos é apenas um método `clonar()`.



2 A classe **Protótipo Concreta** implementa o método de clonagem. Além de copiar os dados do objeto original para o clone, esse método também pode lidar com alguns casos específicos do processo de clonagem relacionados a clonar objetos ligados, desfazendo dependências recursivas, etc.

Aplicabilidade - Builder



- **Utilize o padrão Prototype quando seu código não deve depender de classes concretas de objetos que você precisa copiar.**
- Isso acontece muito quando seu código funciona com objetos passados para você de um código de terceiros através de alguma interface. As classes concretas desses objetos são desconhecidas, e você não pode depender delas mesmo que quisesse.
- O padrão Prototype fornece o código cliente com uma interface geral para trabalhar com todos os objetos que suportam clonagem. Essa interface faz o código do cliente ser independente das classes concretas dos objetos que ele clona.
- **Utilize o padrão quando você precisa reduzir o número de subclasses que somente diferem na forma que inicializam seus respectivos objetos. Alguém pode ter criado essas subclasses para ser capaz de criar objetos com uma configuração específica.**
- O padrão Prototype permite que você use um conjunto de objetos pré construídos, configurados de diversas formas, como protótipos.

Como implementar - Builder



1. Crie uma interface protótipo e declare o método clonar nela. Ou apenas adicione o método para todas as classes de uma hierarquia de classes existente, se você tiver uma.
2. Uma classe protótipo deve definir o construtor alternativo que aceita um objeto daquela classe como um argumento. O construtor deve copiar os valores de todos os campos definidos na classe do objeto passado para a nova instância recém criada. Se você está mudando uma subclasse, você deve chamar o construtor da classe pai para permitir que a superclasse lide com a clonagem de seus campos privados. Se a sua linguagem de programação não suporta sobrecarregamento de métodos, você pode definir um método especial para copiar os dados do objeto. O construtor é um local mais conveniente para se fazer isso porque ele entrega o objeto resultante logo depois que você chamar o operador new.

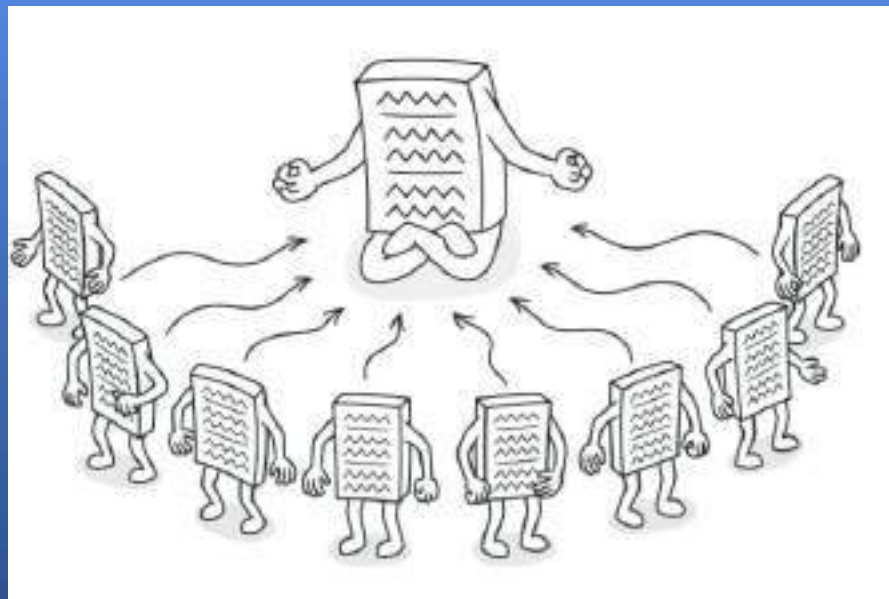
1. O método de clonagem geralmente consiste em apenas uma linha: executando um operador `new` com a versão protótipo do construtor. Observe que toda classe deve explicitamente sobrescrever o método de clonagem e usar sua própria classe junto com o operador `new`. Do contrário, o método de clonagem pode produzir um objeto da classe superior.
2. Opcionalmente, crie um registro protótipo centralizado para armazenar um catálogo de protótipos usados com frequência.

Você pode implementar o registro como uma nova classe `factory` ou colocá-lo na classe protótipo base com um método estático para recuperar o protótipo. Esse método deve procurar por um protótipo baseado em critérios de busca que o código cliente passou para o método. O critério pode ser tanto uma `string` ou um complexo conjunto de parâmetros de busca. Após o protótipo apropriado ser encontrado, o registro deve cloná-lo e retornar a cópia para o client

Padrões Criacionais - Singleton



O Singleton é um padrão de projeto criacional que permite a você garantir que uma classe tenha apenas uma instância, enquanto provê um ponto de acesso global para essa instância.



Problemática



Garantir que uma classe tenha apenas uma única instância. Por que alguém iria querer controlar quantas instâncias uma classe tem? A razão mais comum para isso é para controlar o acesso a algum recurso compartilhado—por exemplo, uma base de dados ou um arquivo. **Funciona assim:** imagine que você criou um objeto, mas depois de um tempo você decidiu criar um novo. Ao invés de receber um objeto fresco, você obterá um que já foi criado. Observe que esse comportamento é impossível implementar com um construtor regular uma vez que uma chamada do construtor **deve** sempre retornar um novo objeto por design.



Fornecer um ponto de acesso global para aquela instância. Se lembra daquelas variáveis globais que você (tá bom, eu) usou para guardar alguns objetos essenciais? Embora sejam muito úteis, elas também são muito inseguras uma vez que qualquer código pode potencialmente sobrescrever os conteúdos daquelas variáveis e quebrar a aplicação.

Assim como uma variável global, o padrão Singleton permite que você acesse algum objeto de qualquer lugar no programa. Contudo, ele também protege aquela instância de ser sobrescrita por outro código.

Há outro lado para esse problema: você não quer que o código que resolve o problema 1 fique espalhado por todo seu programa. É muito melhor tê-lo dentro de uma classe, especialmente se o resto do seu código já depende dela



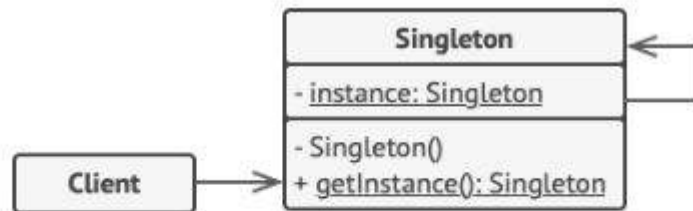
Case 1 - Solução



Todas as implementações do Singleton tem esses dois passos em comum:

- Fazer o construtor padrão privado, para prevenir que outros objetos usem o operador `new` com a classe singleton.
- Criar um método estático de criação que age como um construtor. Esse método chama o construtor privado por debaixo dos panos para criar um objeto e o salva em um campo estático. Todas as chamadas seguintes para esse método retornam o objeto em cache.

Se o seu código tem acesso à classe singleton, então ele será capaz de chamar o método estático da singleton. Então sempre que aquele método é chamado, o mesmo objeto é retornado.



1 A classe **Singleton** declara o método estático `getInstance` que retorna a mesma instância de sua própria classe.

O construtor da singleton deve ser escondido do código cliente. Chamando o método `getInstance` deve ser o único modo de obter o objeto singleton.

```

if (instance == null) {
    // Atenção: se você está criando uma
    // aplicação com apoio multithreading,
    // você deve colocar um thread lock aqui.
    instance = new Singleton()
}
return instance
    
```

Aplicabilidade - Builder



- **Utilize o padrão Singleton quando uma classe em seu programa deve ter apenas uma instância disponível para todos seus clientes; por exemplo, um objeto de base de dados único compartilhado por diferentes partes do programa.**
- O padrão Singleton desabilita todos os outros meios de criar objetos de uma classe exceto pelo método especial de criação. Esse método tanto cria um novo objeto ou retorna um objeto existente se ele já tenha sido criado.
- **Utilize o padrão Singleton quando você precisa de um controle mais estrito sobre as variáveis globais.**
- Ao contrário das variáveis globais, o padrão Singleton garante que há apenas uma instância de uma classe. Nada, a não ser a própria classe singleton, pode substituir a instância salva em cache.
- Observe que você sempre pode ajustar essa limitação e permitir a criação de qualquer número de instâncias singleton. O único pedaço de código que requer mudanças é o corpo do método `getInstance`.

Como implementar - Builder

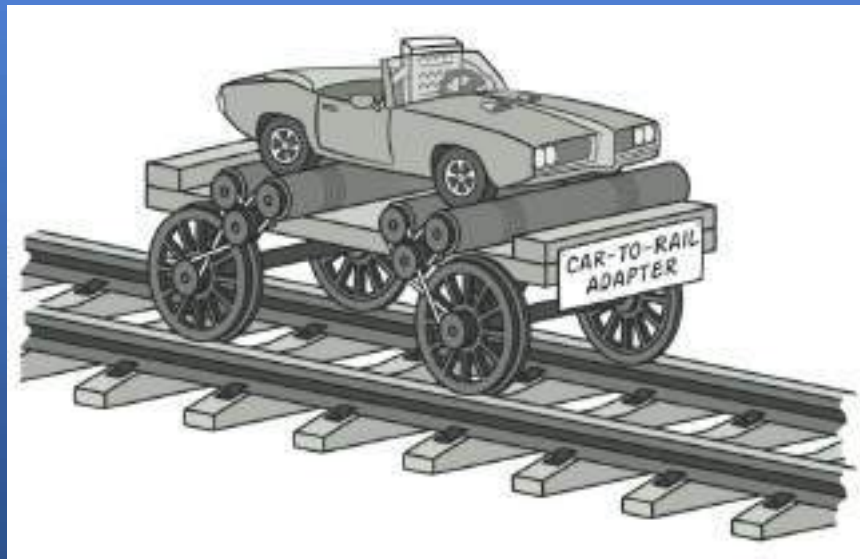


1. Adicione um campo privado estático na classe para o armazenamento da instância singleton.
2. Declare um método de criação público estático para obter a instância singleton.
3. Implemente a “inicialização preguiçosa” dentro do método estático. Ela deve criar um novo objeto na sua primeira chamada e colocá-lo no campo estático. O método deve sempre retornar aquela instância em todas as chamadas subsequentes.
4. Faça o construtor da classe ser privado. O método estático da classe vai ainda ser capaz de chamar o construtor, mas não os demais objetos.
5. Vá para o código cliente e substitua todas as chamadas diretas para o construtor do singleton com chamadas para seu método de criação estático.

Padrões Estruturais - Adapter



O **Adapter** é um padrão que permite que objetos com interfaces incompatíveis colaborem.

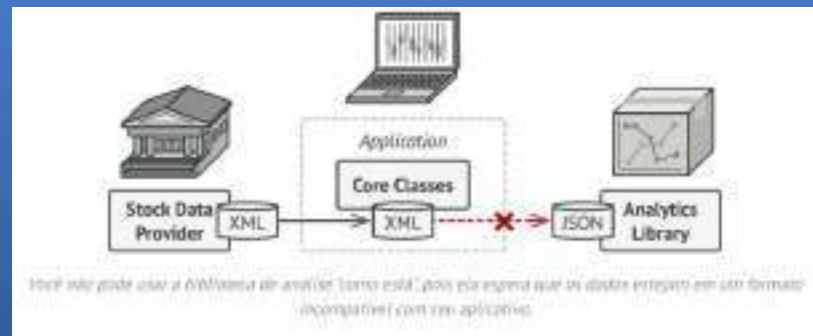


Problemática

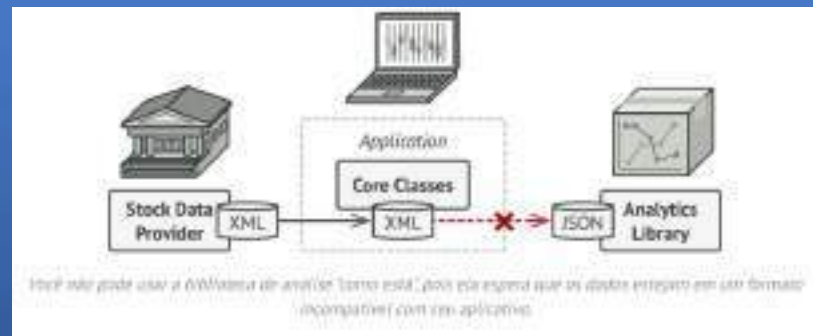


Imagine que você está criando um aplicativo de monitoramento do mercado de ações. O aplicativo baixa os dados das ações de várias fontes em formato XML e, em seguida, exibe gráficos e diagramas visualmente atraentes para o usuário.

Em determinado momento, você decide aprimorar o aplicativo integrando uma biblioteca inteligente de análise de terceiros. Mas há um porém: a biblioteca de análise só funciona com dados em formato JSON.



Você poderia modificar a biblioteca para funcionar com XML. No entanto, isso poderia quebrar algum código existente que depende da biblioteca. E pior, você pode nem ter acesso ao código-fonte da biblioteca, tornando essa abordagem inviável.

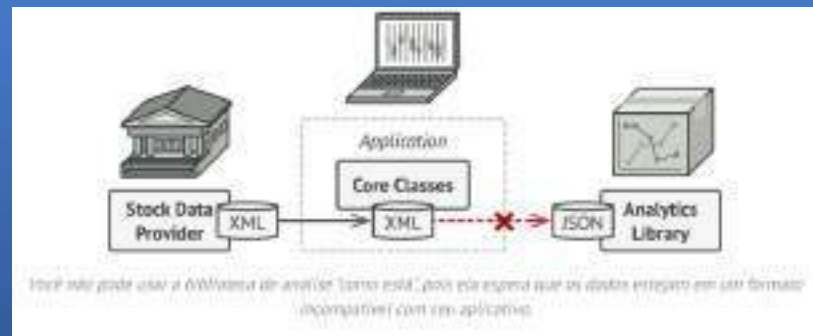


Case 1 - Solução



Você pode criar um *adaptador*. Este é um objeto especial que converte a interface de um objeto para que outro objeto possa entendê-la.

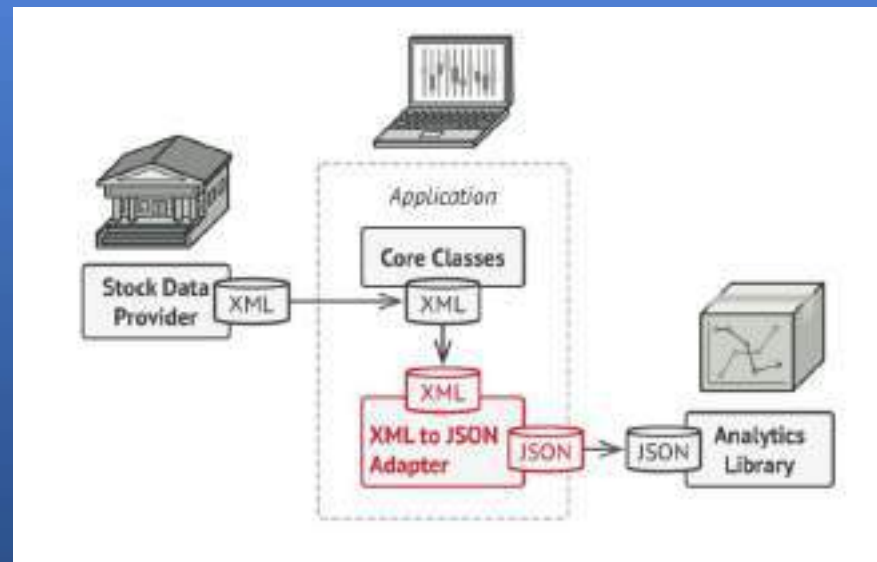
Um adaptador envolve um dos objetos para ocultar a complexidade da conversão que ocorre nos bastidores. O objeto envolvido sequer tem conhecimento do adaptador. Por exemplo, você pode envolver um objeto que opera em metros e quilômetros com um adaptador que converte todos os dados para unidades imperiais, como pés e milhas.



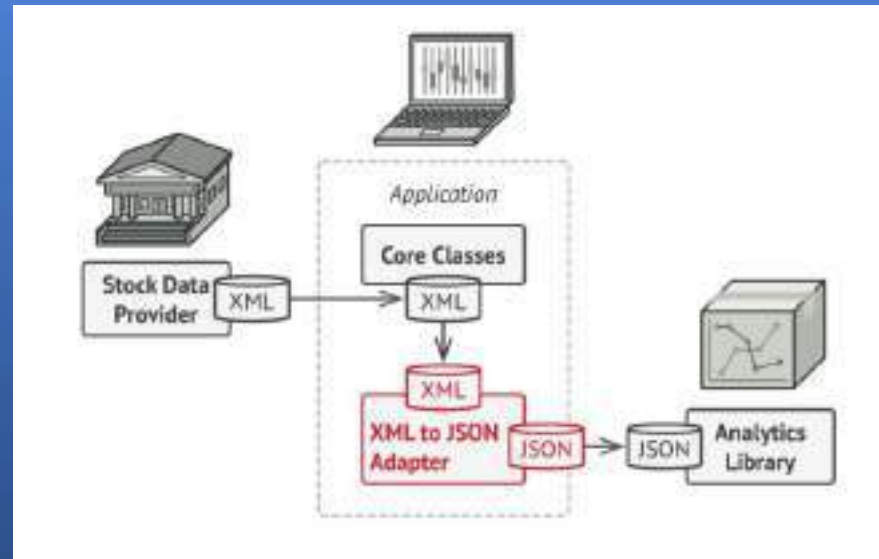
Os adaptadores não só convertem dados em vários formatos, como também ajudam objetos com interfaces diferentes a colaborarem. Veja como funciona:

1. O adaptador recebe uma interface compatível com um dos objetos existentes.
2. Utilizando essa interface, o objeto existente pode chamar os métodos do adaptador com segurança.
3. Ao receber uma chamada, o adaptador passa a solicitação para o segundo objeto, mas em um formato e ordem que o segundo objeto espera.

Às vezes, é até possível criar um adaptador bidirecional que possa converter as chamadas em ambas as direções.



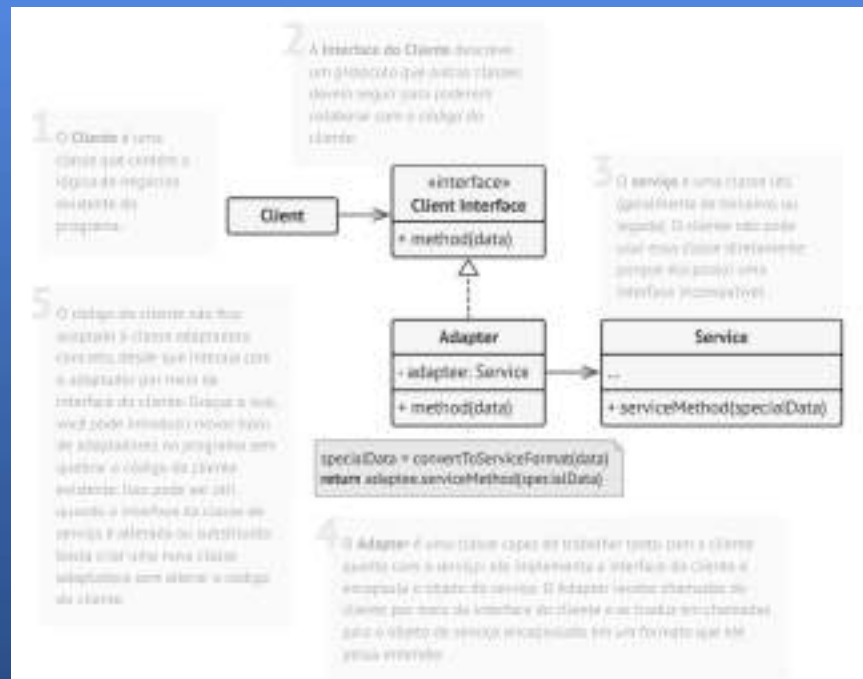
Vamos voltar ao nosso aplicativo de mercado de ações. Para resolver o dilema dos formatos incompatíveis, você pode criar adaptadores XML para JSON para cada classe da biblioteca de análise com a qual seu código interage diretamente. Em seguida, ajuste seu código para se comunicar com a biblioteca somente por meio desses adaptadores. Quando um adaptador recebe uma chamada, ele traduz os dados XML recebidos em uma estrutura JSON e passa a chamada para os métodos apropriados de um objeto de análise encapsulado.



Adaptador de objeto



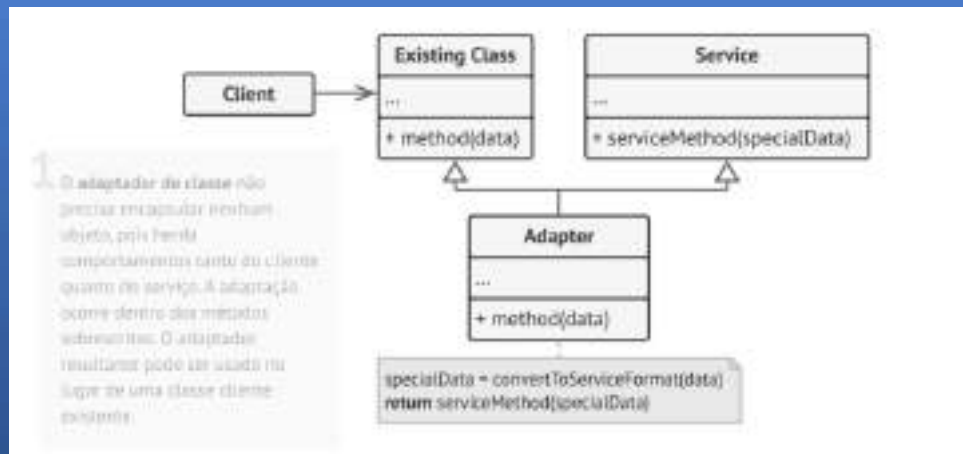
Esta implementação utiliza o princípio da composição de objetos: o adaptador implementa a interface de um objeto e encapsula o outro. Pode ser implementada em todas as linguagens de programação populares.



Adaptador de classe



Esta implementação utiliza herança: o adaptador herda interfaces de ambos os objetos simultaneamente. Observe que essa abordagem só pode ser implementada em linguagens de programação que suportam herança múltipla, como C++.



Aplicabilidade - Adapter



- **Use a classe Adapter quando você quiser usar alguma classe existente, mas a interface dela não for compatível com o restante do seu código.**
- O padrão Adapter permite criar uma classe de camada intermediária que serve como tradutora entre o seu código e uma classe legada, uma classe de terceiros ou qualquer outra classe com uma interface incomum.
- **Utilize esse padrão quando desejar reutilizar várias subclasses existentes que não possuem alguma funcionalidade comum que não pode ser adicionada à superclasse.**
- Você poderia estender cada subclasse e colocar a funcionalidade ausente em novas classes filhas. No entanto, você precisará duplicar o código em todas essas novas classes, o que **é uma péssima ideia**.
- Uma solução muito mais elegante seria colocar a funcionalidade ausente em uma classe adaptadora. Em seguida, você encapsularia os objetos com os recursos ausentes dentro do adaptador, obtendo os recursos necessários dinamicamente. Para que isso funcione, as classes de destino devem ter uma interface comum, e o campo do adaptador deve seguir essa interface.

Como implementar - Adapter

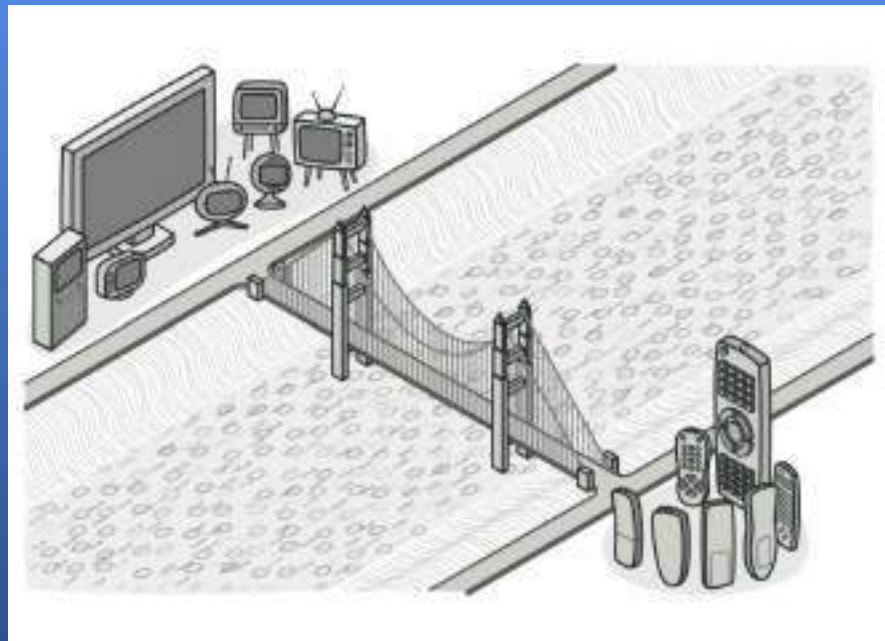


- Certifique-se de ter pelo menos duas classes com interfaces incompatíveis:
 - *Uma classe de serviço* útil , que você não pode alterar (geralmente de terceiros, legada ou com muitas dependências existentes).
 - Uma ou mais classes *de clientes* que se beneficiariam com o uso da classe de serviço.
- Declare a interface do cliente e descreva como os clientes se comunicam com o serviço.
- Crie a classe adaptadora e faça com que ela siga a interface do cliente. Deixe todos os métodos vazios por enquanto.
- Adicione um campo à classe do adaptador para armazenar uma referência ao objeto de serviço. A prática comum é inicializar esse campo por meio do construtor, mas às vezes é mais conveniente passá-lo para o adaptador ao chamar seus métodos.
- Implemente, um por um, todos os métodos da interface do cliente na classe adaptadora. O adaptador deve delegar a maior parte do trabalho pesado ao objeto de serviço, lidando apenas com a interface ou a conversão do formato de dados.
- Os clientes devem usar o adaptador por meio da interface do cliente. Isso permitirá que você altere ou estenda os adaptadores sem afetar o código do cliente.

Padrões Estruturais - Bridge



Bridge é um padrão que permite dividir uma classe grande ou um conjunto de classes intimamente relacionadas em duas hierarquias separadas — abstração e implementação — que podem ser desenvolvidas independentemente uma da outra

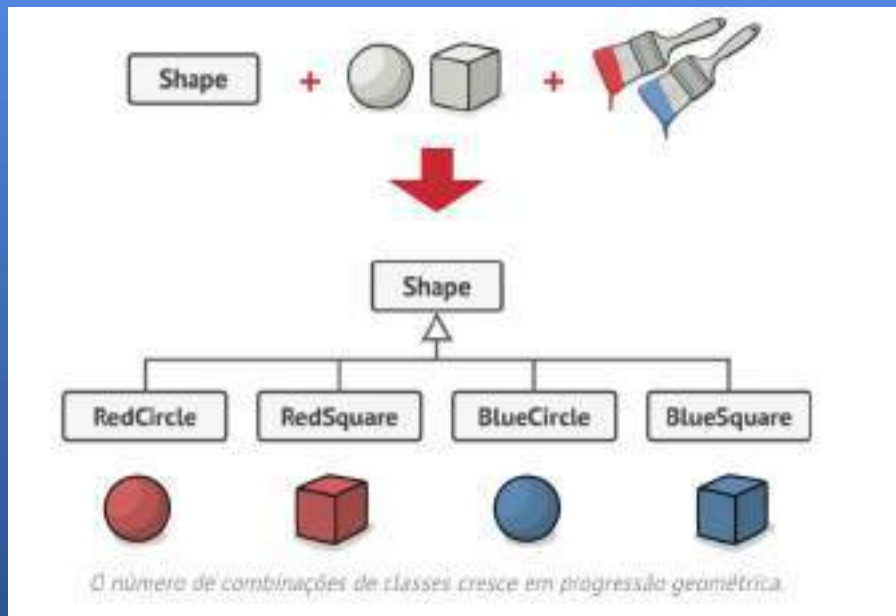


Problemática

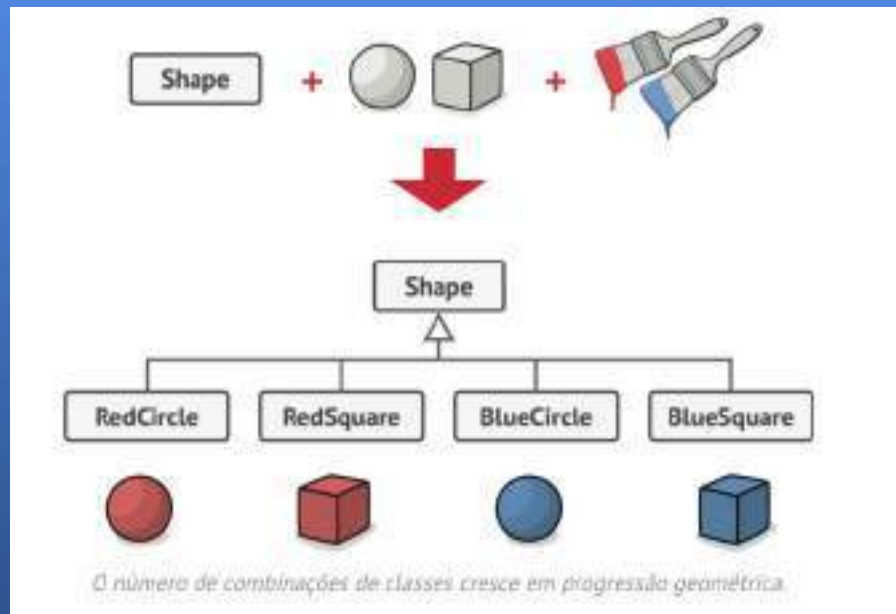


Abstração? Implementação? Parece assustador? Mantenha a calma e vamos considerar um exemplo simples.

Suponha que você tenha uma Shape classe geométrica com um par de subclasses: `Geometric` Circle` Shape` Square``. Você deseja estender essa hierarquia de classes para incorporar cores, então planeja criar subclasses `RedGeometric` e Blue`Shape``. No entanto, como você já tem duas subclasses, precisará criar quatro combinações de classes, como `Geometric`, `Shape` BlueCircle` e `Shape` RedSquare``.



Adicionar novos tipos de formas e cores à hierarquia a fará crescer exponencialmente. Por exemplo, para adicionar uma forma triangular, você precisaria introduzir duas subclasses, uma para cada cor. E depois disso, adicionar uma nova cor exigiria a criação de três subclasses, uma para cada tipo de forma. Quanto mais avançamos, pior fica.

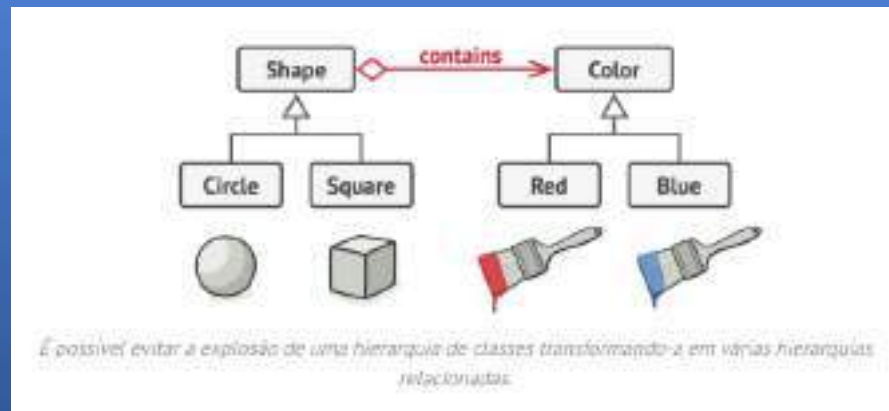


Case 1 - Solução

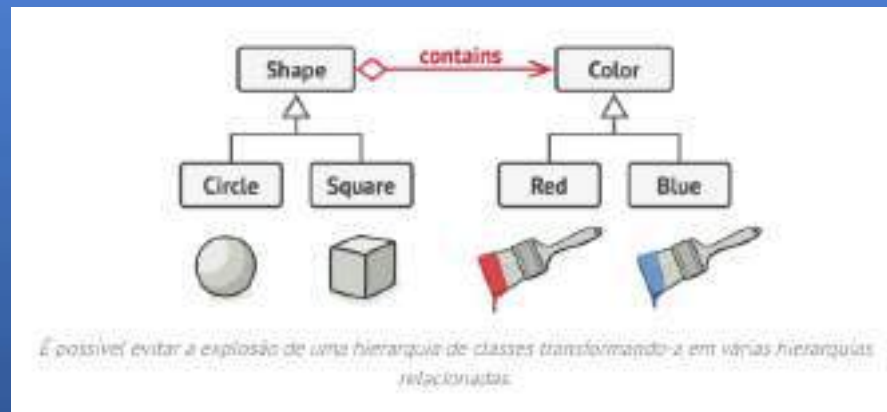


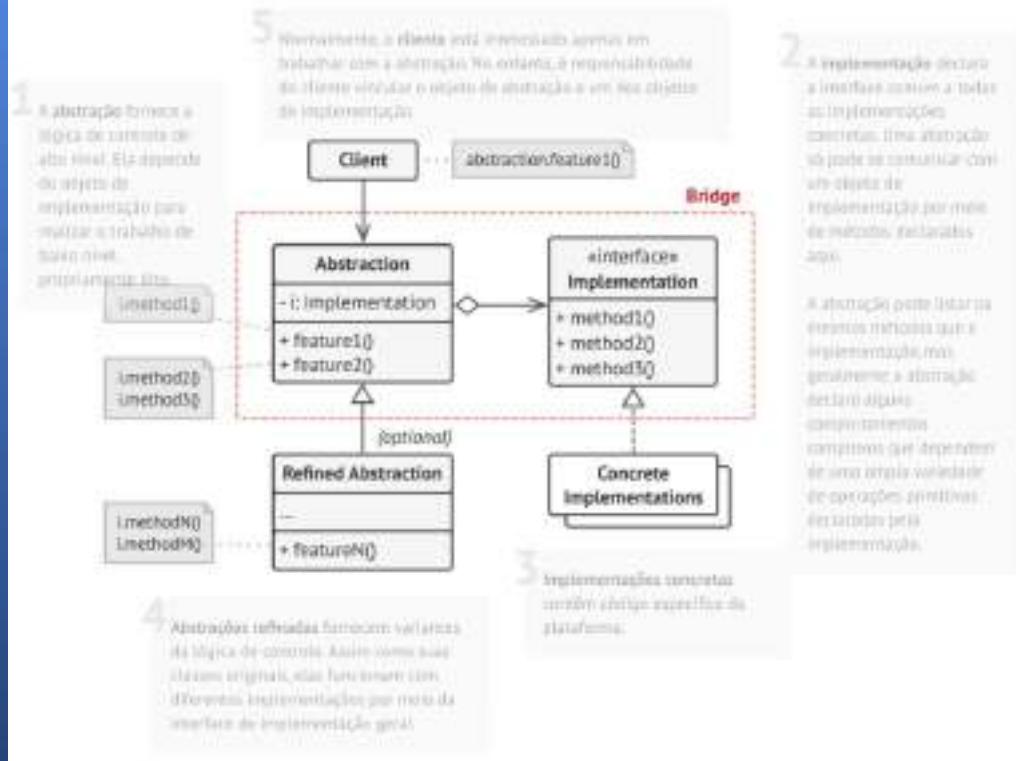
Esse problema ocorre porque estamos tentando estender as classes de forma em duas dimensões independentes: por forma e por cor. Esse é um problema muito comum com herança de classes.

O padrão Bridge tenta resolver esse problema substituindo a herança pela composição de objetos. Isso significa que você extrai uma das dimensões para uma hierarquia de classes separada, de forma que as classes originais referenciem um objeto da nova hierarquia, em vez de terem todo o seu estado e comportamentos dentro de uma única classe.



Seguindo essa abordagem, podemos extrair o código relacionado à cor para sua própria classe com duas subclasses: `RedColor` e `BlueColor`. A `Shape` classe `Color` então recebe um campo de referência apontando para um dos objetos de cor. Agora, a forma pode delegar qualquer trabalho relacionado à cor para o objeto de cor vinculado. Essa referência atuará como uma ponte entre as classes `Shape` `Color` e `ColorColor`. A partir de agora, adicionar novas cores não exigirá alterar a hierarquia da forma, e vice-versa.





Aplicabilidade - Bridge



- **Utilize o padrão Bridge quando desejar dividir e organizar uma classe monolítica que possui diversas variantes de alguma funcionalidade (por exemplo, se a classe puder funcionar com vários servidores de banco de dados).**
- Quanto maior uma classe, mais difícil é entender seu funcionamento e mais tempo leva para implementar uma alteração. As mudanças feitas em uma das variações de funcionalidade podem exigir alterações em toda a classe, o que frequentemente resulta em erros ou na não resolução de alguns efeitos colaterais críticos.
- O padrão Bridge permite dividir uma classe monolítica em várias hierarquias de classes. Depois disso, você pode alterar as classes em cada hierarquia independentemente das classes nas outras. Essa abordagem simplifica a manutenção do código e minimiza o risco de quebrar o código existente.
- **Utilize esse padrão quando precisar estender uma classe em várias dimensões ortogonais (independentes).**
- A proposta do Bridge é extrair uma hierarquia de classes separada para cada uma das dimensões. A classe original delega o trabalho relacionado aos objetos pertencentes a essas hierarquias, em vez de realizar tudo sozinho

- Embora seja opcional, o padrão Bridge permite substituir o objeto de implementação dentro da abstração. É tão simples quanto atribuir um novo valor a um campo.
- Aliás, este último ponto é o principal motivo pelo qual tantas pessoas confundem o padrão Bridge com o padrão **Strategy** . Lembre-se de que um padrão é mais do que apenas uma maneira específica de estruturar suas classes. Ele também pode comunicar uma intenção e um problema que está sendo abordad

Como implementar - Bridge



1. Identifique as dimensões ortogonais em suas classes. Esses conceitos independentes podem ser: abstração/plataforma, domínio/infraestrutura, front-end/back-end ou interface/implementação.
2. Veja quais operações o cliente precisa e defina-as na classe de abstração base.
3. Determine as operações disponíveis em todas as plataformas. Declare aquelas que a abstração necessita na interface de implementação geral.
4. Para todas as plataformas em seu domínio, crie classes de implementação concretas, mas certifique-se de que todas sigam a interface de implementação.
5. Dentro da classe de abstração, adicione um campo de referência para o tipo de implementação. A abstração delega a maior parte do trabalho ao objeto de implementação referenciado nesse campo.
6. Se você tiver várias variantes de lógica de alto nível, crie abstrações refinadas para cada variante, estendendo a classe de abstração base.
7. O código do cliente deve passar um objeto de implementação para o construtor da abstração para associá-los. Depois disso, o cliente pode esquecer a implementação e trabalhar apenas com o objeto de abstração.

Padrões Estruturais - Composite



O padrão de projeto **Composite** permite compor objetos em estruturas de árvore e, em seguida, trabalhar com essas estruturas como se fossem objetos individuais.



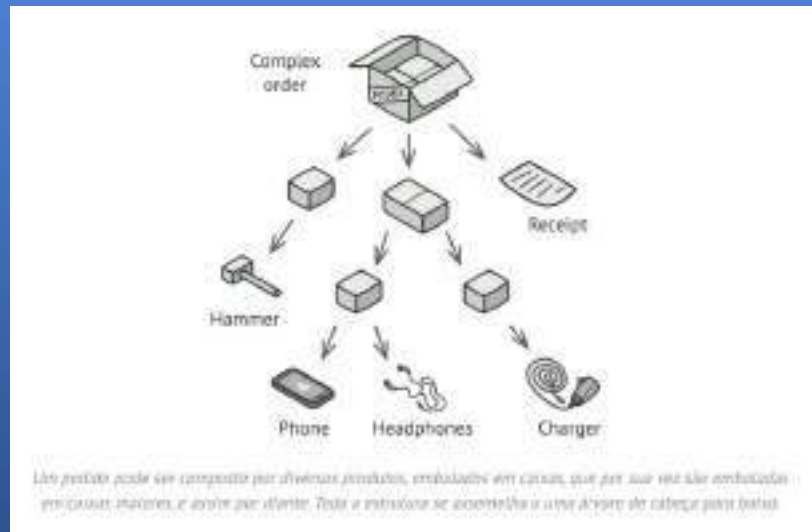
Problemática



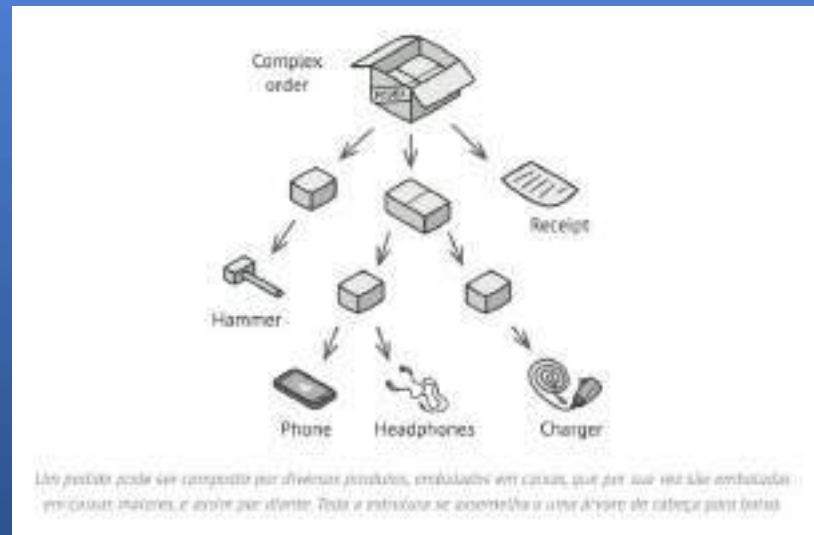
O uso do padrão Composite só faz sentido quando o modelo principal do seu aplicativo pode ser representado como uma árvore.

Por exemplo, imagine que você tenha dois tipos de objetos: `ProductsA` e `BoxesB`. Um `A Box` pode conter vários `A Products`, bem como uma série de `A` menores `Boxes`. Esses `A` menores `Boxes` também podem conter alguns `A Products` ou até mesmo `A` menores `Boxes`, e assim por diante.

Imagine que você decide criar um sistema de pedidos que utiliza essas classes. Os pedidos podem conter produtos simples sem embalagem, bem como caixas recheadas de produtos... e outras caixas. Como você determinaria o preço total de um pedido desse tipo?



Você poderia tentar a abordagem direta: desembrulhar todas as caixas, percorrer todos os produtos e calcular o total. Isso seria viável no mundo real; mas em um programa, não é tão simples quanto executar um loop. Você precisa conhecer as classes dos itens `Products` que `Boxes` está percorrendo, o nível de aninhamento das caixas e outros detalhes complexos antecipadamente. Tudo isso torna a abordagem direta muito complicada ou até mesmo impossível.



Case 1 - Solução



O padrão Composite sugere que você trabalhe com `Products` e `Boxes` através de uma interface comum que declara um método para calcular o preço total.

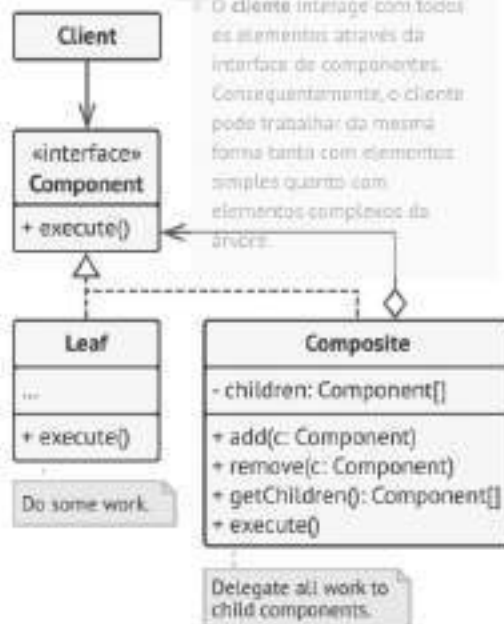
Como esse método funcionaria? Para um produto, ele simplesmente retornaria o preço do produto. Para uma caixa, ele analisaria cada item que a caixa contém, perguntaria o preço de cada um e, em seguida, retornaria o total da caixa. Se um desses itens fosse uma caixa menor, o cálculo também começaria a analisar o conteúdo dessa caixa, e assim por diante, até que os preços de todos os componentes internos fossem calculados. Uma caixa poderia até adicionar um custo extra ao preço final, como o custo da embalagem.

A maior vantagem dessa abordagem é que você não precisa se preocupar com as classes concretas dos objetos que compõem a árvore. Você não precisa saber se um objeto é um produto simples ou uma caixa sofisticada. Você pode tratá-los todos da mesma forma por meio da interface comum. Quando você chama um método, os próprios objetos repassam a solicitação pela árvore.

1 A interface **Component** descreve operações que são comuns tanto a elementos simples quanto a elementos complexos da árvore.

2 A folha é um elemento básico de uma árvore que não possui subelementos.

Normalmente, os componentes folha acabam realizando a maior parte do trabalho real, já que não têm a quem delegar essa tarefa.



4 O cliente interage com todos os elementos através da interface de componentes. Consequentemente, o cliente pode trabalhar da mesma forma tanto com elementos simples quanto com elementos complexos da árvore.

3 O **Container** (também conhecida como **Elemento composto**) é um elemento que possui subelementos: folhas ou outros containers. Um container não conhece as classes concretas de seus filhos. Ele interage com todos os subelementos somente através da interface do componente.

Ao receber uma solicitação, um container delega o trabalho aos seus subelementos, processa os resultados intermediários e, em seguida, retorna o resultado final ao cliente.

Aplicabilidade - Composite



- **Utilize o padrão Composite quando precisar implementar uma estrutura de objetos em forma de árvore.**
- O padrão Composite oferece dois tipos básicos de elementos que compartilham uma interface comum: folhas simples e contêineres complexos. Um contêiner pode ser composto tanto de folhas quanto de outros contêineres. Isso permite construir uma estrutura de objetos recursiva aninhada que se assemelha a uma árvore.
- **Utilize esse padrão quando desejar que o código do cliente trate elementos simples e complexos de maneira uniforme.**
- Todos os elementos definidos pelo padrão Composite compartilham uma interface comum. Usando essa interface, o cliente não precisa se preocupar com a classe concreta dos objetos com os quais trabalha.

Como implementar - Composite

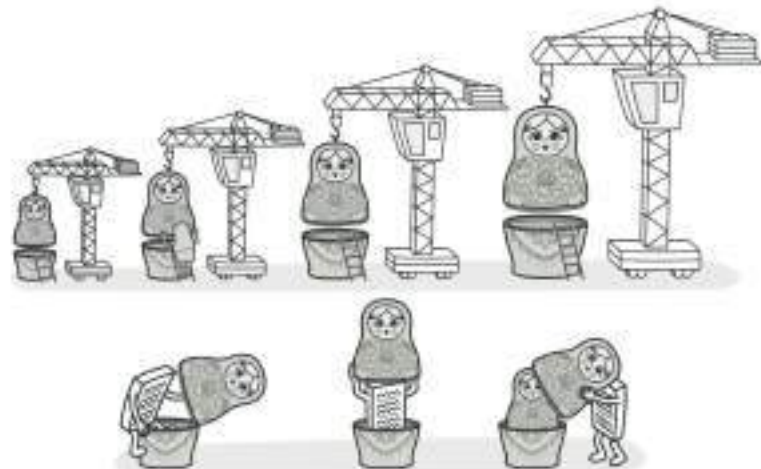


- Certifique-se de que o modelo principal do seu aplicativo possa ser representado como uma estrutura em árvore. Tente dividi-lo em elementos e contêineres simples. Lembre-se de que os contêineres devem ser capazes de conter tanto elementos simples quanto outros contêineres.
- Declare a interface do componente com uma lista de métodos que façam sentido tanto para componentes simples quanto para componentes complexos.
- Crie uma classe folha para representar elementos simples. Um programa pode ter várias classes folha diferentes.
- Crie uma classe de contêiner para representar elementos complexos. Nessa classe, forneça um campo de array para armazenar referências a subelementos. O array deve ser capaz de armazenar tanto elementos folha quanto contêineres, portanto, certifique-se de que ele seja declarado com o tipo de interface `Component` do `Component` componente. Ao implementar os métodos da interface do componente, lembre-se de que um contêiner deve delegar a maior parte do trabalho aos subelementos.
- Por fim, defina os métodos para adicionar e remover elementos filhos no contêiner. Lembre-se de que essas operações podem ser declaradas na interface do componente. Isso violaria o *Princípio da Segregação de Interfaces*, pois os métodos estariam vazios na classe folha. No entanto, o cliente poderá tratar todos os elementos igualmente, mesmo ao compor a árvore.

Padrões Estruturais - Decorator



Decorator é um padrão de projeto que permite associar novos comportamentos a objetos, colocando esses objetos dentro de objetos encapsuladores especiais que contêm os comportamentos.

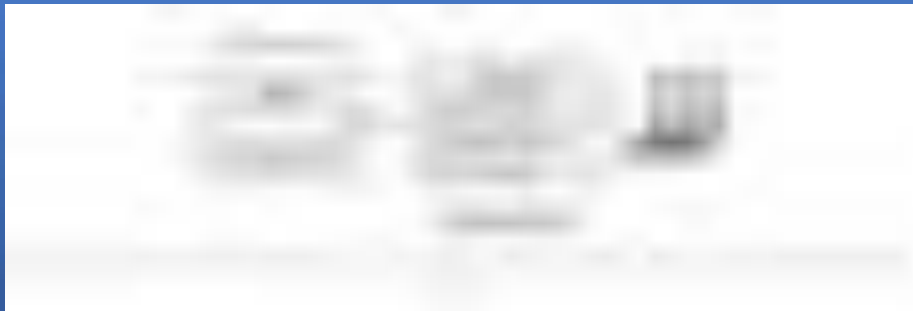


Problemática

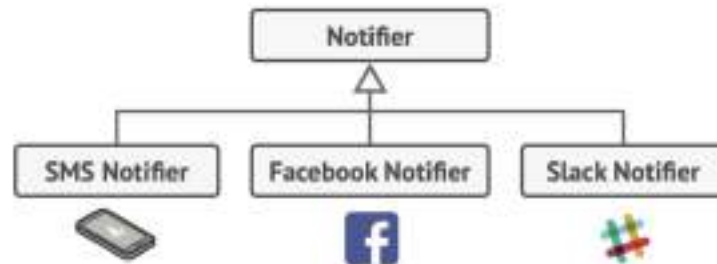


Imagine que você está trabalhando em uma biblioteca de notificações que permite que outros programas notifiquem seus usuários sobre eventos importantes.

A versão inicial da biblioteca era baseada em uma `Notifier` classe que possuía apenas alguns campos, um construtor e um único `send` método. O método podia receber um argumento de mensagem de um cliente e enviar a mensagem para uma lista de e-mails que eram passados para o notificador através de seu construtor. Um aplicativo de terceiros, que atuava como cliente, deveria criar e configurar o objeto notificador uma única vez e, em seguida, utilizá-lo sempre que algo importante acontecesse.



Em algum momento, você percebe que os usuários da biblioteca esperam mais do que apenas notificações por e-mail. Muitos gostariam de receber um SMS sobre assuntos críticos. Outros gostariam de ser notificados pelo Facebook e, claro, os usuários corporativos adorariam receber notificações pelo Slack.

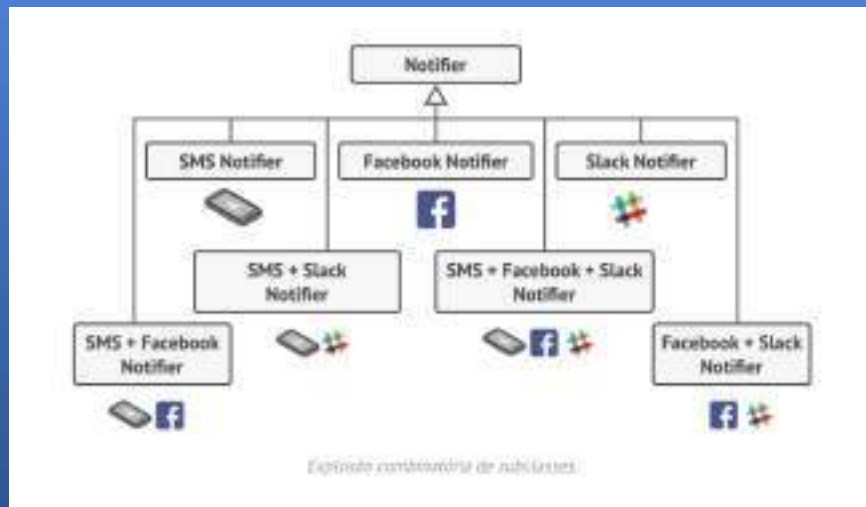


Cada tipo de notificação é implementado como uma subclasse de um notificador.

Qual a dificuldade? Você estendeu a `Notifier` classe e colocou os métodos de notificação adicionais em novas subclasses. Agora, o cliente deveria instanciar a classe de notificação desejada e usá-la para todas as notificações subsequentes.

Mas aí alguém, com razão, perguntou: "Por que não usar vários tipos de notificação ao mesmo tempo? Se sua casa estivesse pegando fogo, você provavelmente gostaria de ser informado por todos os canais."

Você tentou resolver esse problema criando subclasses especiais que combinavam vários métodos de notificação em uma única classe. No entanto, logo ficou evidente que essa abordagem inflaria o código imensamente, não apenas o código da biblioteca, mas também o código do cliente.



Case 1 - Solução



Estender uma classe é a primeira coisa que vem à mente quando você precisa alterar o comportamento de um objeto. No entanto, a herança tem várias ressalvas importantes que você precisa conhecer.

- A herança é estática. Você não pode alterar o comportamento de um objeto existente em tempo de execução. Você só pode substituir o objeto inteiro por outro criado a partir de uma subclasse diferente.
- Subclasses podem ter apenas uma classe pai. Na maioria das linguagens, a herança não permite que uma classe herde comportamentos de múltiplas classes simultaneamente.

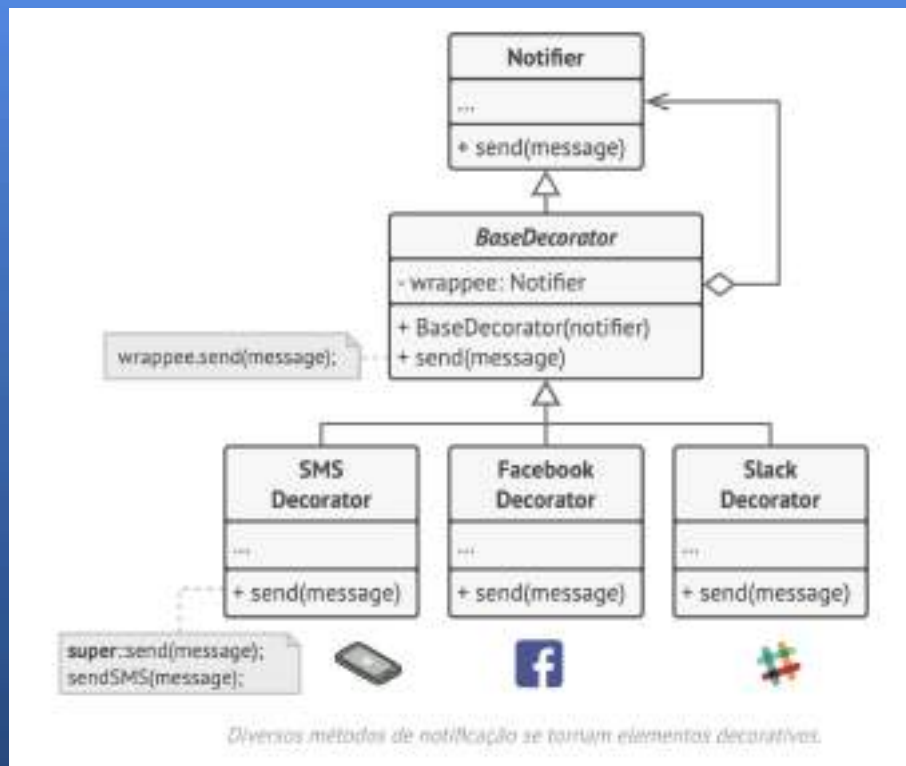
Uma das maneiras de superar essas limitações é usando **agregação ou composição**. Em vez de *herança*. Ambas as alternativas funcionam de maneira quase idêntica: um objeto *tem uma* referência a outro e delega a ele algumas tarefas, enquanto que, com a herança, o próprio objeto é capaz de realizar essa tarefa, herdando o comportamento de sua superclasse.

Com essa nova abordagem, você pode facilmente substituir o objeto "auxiliar" vinculado por outro, alterando o comportamento do contêiner em tempo de execução. Um objeto pode usar o comportamento de várias classes, tendo referências a múltiplos objetos e delegando a eles todos os tipos de trabalho. Agregação/composição é o princípio fundamental por trás de muitos padrões de projeto, incluindo o Decorator. Dito isso, vamos retornar à discussão sobre padrões.

"Wrapper" é um apelido alternativo para o padrão Decorator que expressa claramente a ideia principal do padrão. Um *wrapper* é um objeto que pode ser vinculado a um objeto *alvo* . O wrapper contém o mesmo conjunto de métodos que o alvo e delega a ele todas as requisições que recebe. No entanto, o wrapper pode alterar o resultado executando alguma ação antes ou depois de passar a requisição para o alvo.

Quando um simples wrapper se torna um decorador de fato? Como mencionei, o wrapper implementa a mesma interface que o objeto encapsulado. É por isso que, da perspectiva do cliente, esses objetos são idênticos. Faça com que o campo de referência do wrapper aceite qualquer objeto que siga essa interface. Isso permitirá que você cubra um objeto com múltiplos wrappers, adicionando o comportamento combinado de todos eles.

Em nosso exemplo de notificações, vamos manter o comportamento simples de notificação por e-mail dentro da `Notifier` classe base, mas transformar todos os outros métodos de notificação em decoradores.



Aplicabilidade - Decorator



- **Utilize o padrão Decorator quando precisar atribuir comportamentos extras a objetos em tempo de execução sem quebrar o código que utiliza esses objetos.**
- O Decorator permite estruturar sua lógica de negócios em camadas, criar um decorator para cada camada e compor objetos com várias combinações dessa lógica em tempo de execução. O código do cliente pode tratar todos esses objetos da mesma maneira, já que todos seguem uma interface comum.
- **Utilize esse padrão quando for complicado ou impossível estender o comportamento de um objeto usando herança.**
- Muitas linguagens de programação possuem a `final` palavra-chave ``new``, que pode ser usada para impedir a extensão de uma classe. Para uma classe final, a única maneira de reutilizar o comportamento existente seria envolver a classe com seu próprio wrapper, usando o padrão Decorator.

Como implementar - Decorator

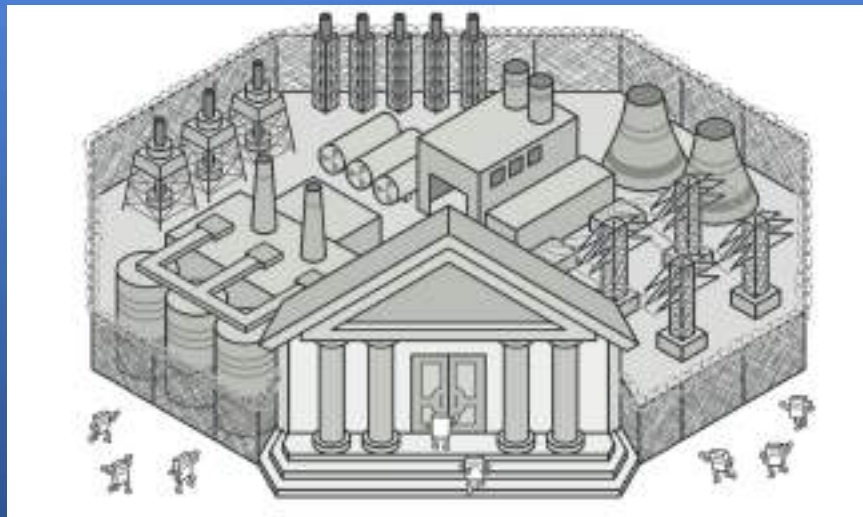


- Certifique-se de que seu domínio de negócios possa ser representado como um componente primário com várias camadas opcionais sobrepostas.
- Identifique os métodos comuns ao componente principal e às camadas opcionais. Crie uma interface de componente e declare esses métodos nela.
- Crie uma classe de componente concreta e defina o comportamento básico nela.
- Crie uma classe decoradora base. Ela deve ter um campo para armazenar uma referência a um objeto encapsulado. O campo deve ser declarado com o tipo de interface do componente para permitir a vinculação a componentes concretos, bem como a decoradores. O decorador base deve delegar todo o trabalho ao objeto encapsulado.
- Certifique-se de que todas as classes implementem a interface do componente.
- Crie decoradores concretos estendendo-os a partir do decorador base. Um decorador concreto deve executar seu comportamento antes ou depois da chamada ao método pai (que sempre delega ao objeto encapsulado).
- O código do cliente deve ser responsável por criar os decoradores e compô-los da maneira que o cliente precisa

Padrões Estruturais - Facade



Fachada é um padrão de projeto que fornece uma interface simplificada para uma biblioteca, um framework ou qualquer outro conjunto complexo de classes.



Problemática



Imagine que você precisa fazer seu código funcionar com um amplo conjunto de objetos pertencentes a uma biblioteca ou framework complexo. Normalmente, você precisaria inicializar todos esses objetos, controlar as dependências, executar os métodos na ordem correta e assim por diante.

Como resultado, a lógica de negócios de suas classes ficaria fortemente acoplada aos detalhes de implementação de classes de terceiros, dificultando a compreensão e a manutenção.

Case 1 - Solução

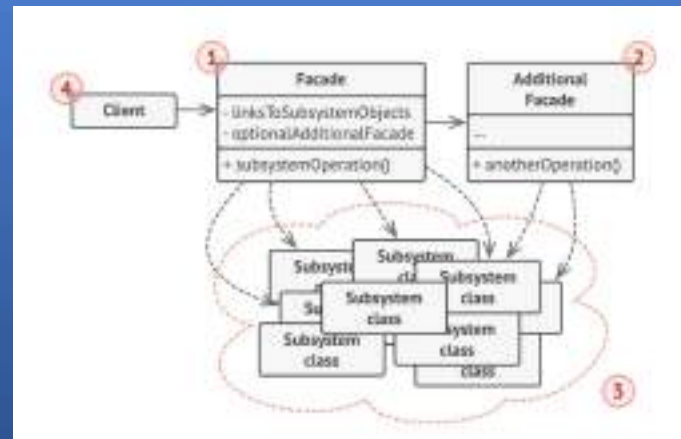


Uma fachada é uma classe que fornece uma interface simples para um subsistema complexo que contém muitas partes móveis. Uma fachada pode oferecer funcionalidade limitada em comparação com o trabalho direto com o subsistema. No entanto, ela inclui apenas os recursos que os clientes realmente consideram importantes.

Ter uma fachada é útil quando você precisa integrar seu aplicativo a uma biblioteca sofisticada que possui dezenas de recursos, mas você só precisa de uma pequena parte de sua funcionalidade.

Por exemplo, um aplicativo que publica vídeos curtos e engraçados de gatos em redes sociais poderia potencialmente usar uma biblioteca profissional de conversão de vídeo. No entanto, tudo o que ele realmente precisa é de uma classe com um único método `encode(filename, format)`. Depois de criar essa classe e conectá-la à biblioteca de conversão de vídeo, você terá sua primeira fachada.

- A **Fachada** proporciona acesso conveniente a uma parte específica da funcionalidade do subsistema. Ela sabe para onde direcionar a solicitação do cliente e como operar todas as partes móveis.
- Uma classe **de fachada adicional** pode ser criada para evitar que uma fachada principal seja poluída com elementos não relacionados, o que poderia transformá-la em mais uma estrutura complexa. Fachadas adicionais podem ser usadas tanto por clientes quanto por outras fachadas.
- O **Subsistema Complexo** consiste em dezenas de objetos diferentes. Para que todos eles executem tarefas significativas, é necessário analisar detalhadamente a implementação do subsistema, como inicializar os objetos na ordem correta e fornecer dados no formato adequado. As classes de subsistema não têm conhecimento da existência da fachada. Elas operam dentro do sistema e interagem diretamente entre si.
- O **cliente** utiliza a fachada em vez de chamar os objetos do subsistema diretamente.



Aplicabilidade - Decorator



- **Utilize o padrão Fachada quando precisar de uma interface limitada, porém simples, para um subsistema complexo.**
- Frequentemente, os subsistemas tornam-se mais complexos com o tempo. Mesmo a aplicação de padrões de projeto normalmente leva à criação de mais classes. Um subsistema pode se tornar mais flexível e fácil de reutilizar em diversos contextos, mas a quantidade de configuração e código repetitivo que exige do cliente aumenta cada vez mais. O Facade busca solucionar esse problema fornecendo um atalho para os recursos mais utilizados do subsistema, que atendem à maioria dos requisitos do cliente.
- **Use a Fachada quando quiser estruturar um subsistema em camadas.**
- Crie fachadas para definir pontos de entrada para cada nível de um subsistema. Você pode reduzir o acoplamento entre múltiplos subsistemas exigindo que eles se comuniquem apenas por meio de fachadas.
- Por exemplo, vamos retornar à nossa estrutura de conversão de vídeo. Ela pode ser dividida em duas camadas: uma relacionada a vídeo e outra a áudio. Para cada camada, você pode criar uma fachada e, em seguida, fazer com que as classes de cada camada se comuniquem entre si por meio dessas fachadas. Essa abordagem é muito semelhante ao padrão **Mediator**.
-

Como implementar - Decorator

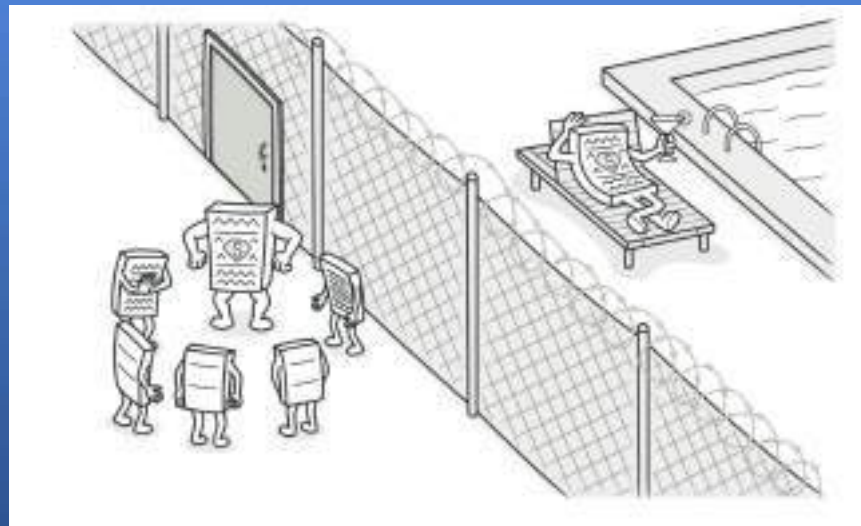


- Verifique se é possível fornecer uma interface mais simples do que a que um subsistema existente já oferece. Você está no caminho certo se essa interface tornar o código do cliente independente de muitas das classes do subsistema.
- Declare e implemente esta interface em uma nova classe de fachada. A fachada deve redirecionar as chamadas do código do cliente para os objetos apropriados do subsistema. A fachada deve ser responsável por inicializar o subsistema e gerenciar seu ciclo de vida subsequente, a menos que o código do cliente já faça isso.
- Para aproveitar ao máximo o padrão, faça com que todo o código do cliente se comunique com o subsistema somente por meio da fachada. Dessa forma, o código do cliente fica protegido contra quaisquer alterações no código do subsistema. Por exemplo, quando um subsistema for atualizado para uma nova versão, você precisará modificar apenas o código na fachada.
- Se a fachada ficar **muito grande**, considere extrair parte do seu comportamento para uma nova classe de fachada mais refinada

Padrões Estruturais - Proxy



Um **proxy** é um padrão de projeto estrutural que permite fornecer um substituto ou marcador para outro objeto. Um proxy controla o acesso ao objeto original, permitindo que você execute alguma ação antes ou depois que a requisição chegar ao objeto original.



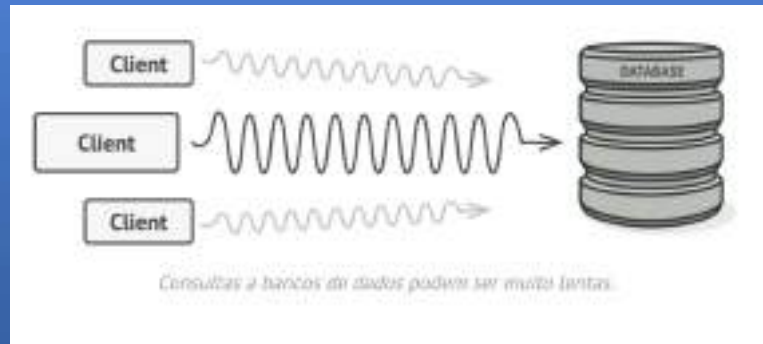
Problemática



Por que você gostaria de controlar o acesso a um objeto? Aqui está um exemplo: você tem um objeto enorme que consome uma grande quantidade de recursos do sistema. Você precisa dele de vez em quando, mas não sempre.

Você poderia implementar inicialização preguiçosa: criar este objeto somente quando ele for realmente necessário. Todos os clientes do objeto precisariam executar algum código de inicialização adiada. Infelizmente, isso provavelmente causaria muita duplicação de código.

Em um mundo ideal, gostaríamos de inserir esse código diretamente na classe do nosso objeto, mas isso nem sempre é possível. Por exemplo, a classe pode fazer parte de uma biblioteca fechada de terceiros.

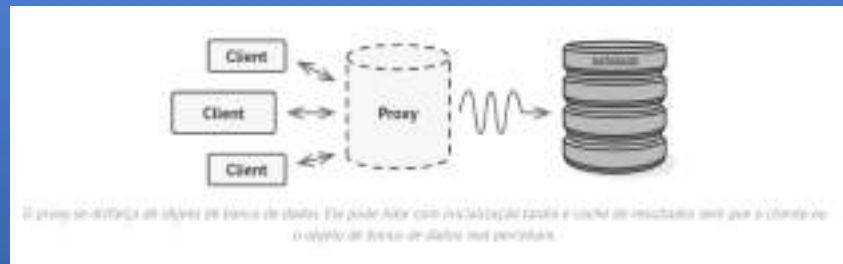


Case 1 - Solução



O padrão Proxy sugere que você crie uma nova classe proxy com a mesma interface de um objeto de serviço original. Em seguida, você atualiza seu aplicativo para que ele passe o objeto proxy para todos os clientes do objeto original. Ao receber uma solicitação de um cliente, o proxy cria um objeto de serviço real e delega todo o trabalho a ele.

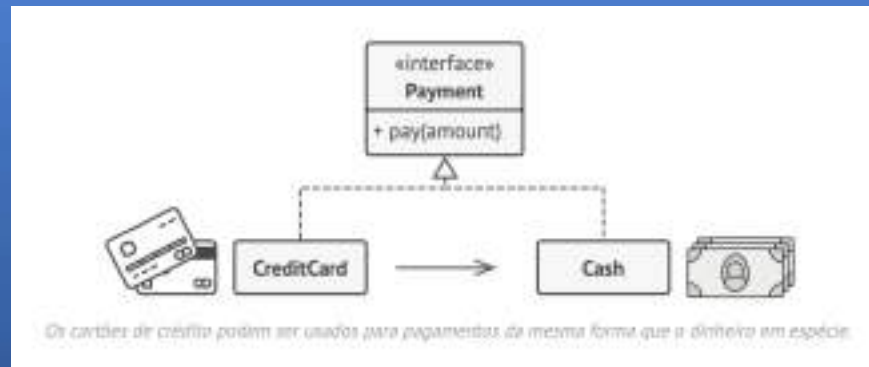
Mas qual é a vantagem? Se você precisar executar algo antes ou depois da lógica principal da classe, o proxy permite que você faça isso sem alterar a classe. Como o proxy implementa a mesma interface que a classe original, ele pode ser passado para qualquer cliente que espere um objeto de serviço real.



Analógia com o mundo real



Um cartão de crédito é um substituto para uma conta bancária, que por sua vez é um substituto para dinheiro em espécie. Ambos implementam a mesma interface: podem ser usados para efetuar pagamentos. O consumidor se sente ótimo porque não precisa carregar grandes quantias em dinheiro vivo. O dono de uma loja também fica satisfeito, já que a receita da transação é adicionada eletronicamente à conta bancária da loja, sem o risco de perder o depósito ou ser assaltado a caminho do banco.



Aplicabilidade - Proxy



- Existem dezenas de maneiras de utilizar o padrão Proxy. Vamos analisar os usos mais populares.
- **Inicialização preguiçosa (proxy virtual).** Isso ocorre quando você tem um objeto de serviço pesado que desperdiça recursos do sistema permanecendo sempre ativo, mesmo que você só precise dele ocasionalmente.
- Em vez de criar o objeto quando o aplicativo é iniciado, você pode adiar a inicialização do objeto para um momento em que ele seja realmente necessário.
- **Controle de acesso (proxy de proteção).** Isso ocorre quando você deseja que apenas clientes específicos possam usar o objeto de serviço; por exemplo, quando seus objetos são partes cruciais de um sistema operacional e os clientes são vários aplicativos em execução (incluindo aplicativos maliciosos).
- O proxy só pode encaminhar a solicitação para o objeto de serviço se as credenciais do cliente corresponderem a determinados critérios.

- **Execução local de um serviço remoto (proxy remoto).** Isso ocorre quando o objeto de serviço está localizado em um servidor remoto.
- Nesse caso, o proxy encaminha a solicitação do cliente pela rede, lidando com todos os detalhes complexos de interagir com a rede.
- **Registro de solicitações (proxy de registro).** Isso é útil quando você deseja manter um histórico das solicitações feitas ao objeto de serviço.
- O proxy pode registrar cada solicitação antes de encaminhá-la ao serviço.
- **Armazenamento em cache dos resultados das solicitações (proxy de cache).** Isso ocorre quando você precisa armazenar em cache os resultados das solicitações do cliente e gerenciar o ciclo de vida desse cache, especialmente se os resultados forem muito grandes.
- O proxy pode implementar o armazenamento em cache para solicitações recorrentes que sempre produzem os mesmos resultados. O proxy pode usar os parâmetros das solicitações como chaves de cache.
-

- O proxy pode implementar o armazenamento em cache para solicitações recorrentes que sempre produzem os mesmos resultados. O proxy pode usar os parâmetros das solicitações como chaves de cache.
- **Referência inteligente. Isso é útil quando você precisa descartar um objeto complexo assim que não houver mais clientes usando-o.**
- O proxy pode rastrear os clientes que obtiveram uma referência ao objeto de serviço ou aos seus resultados. Periodicamente, o proxy pode verificar os clientes para ver se ainda estão ativos. Se a lista de clientes ficar vazia, o proxy pode descartar o objeto de serviço e liberar os recursos do sistema subjacente.
- O proxy também pode rastrear se o cliente modificou o objeto de serviço. Assim, os objetos inalterados podem ser reutilizados por outros clientes.

Como implementar - Proxy



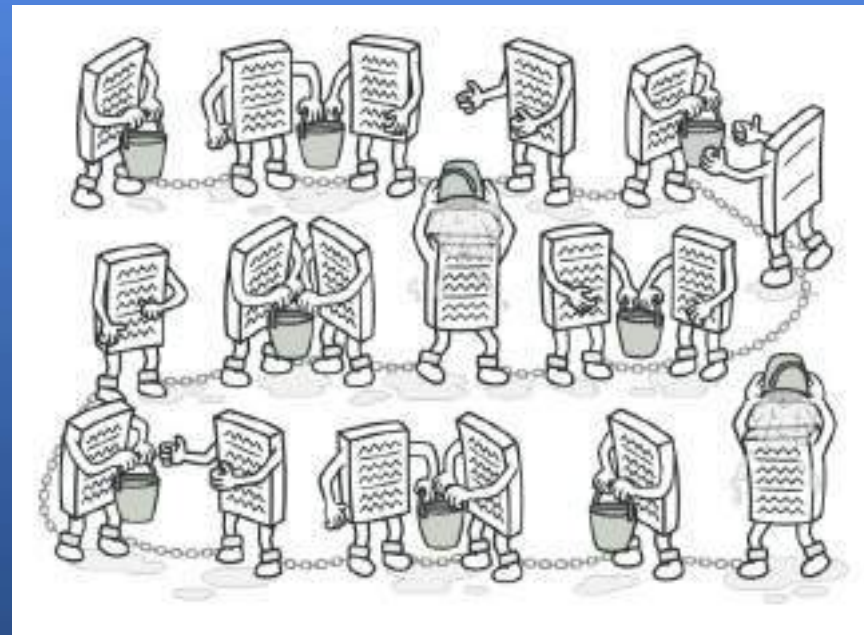
- Se não houver uma interface de serviço preexistente, crie uma para tornar os objetos proxy e de serviço intercambiáveis. Extrair a interface da classe de serviço nem sempre é possível, pois seria necessário alterar todos os clientes do serviço para usar essa interface. A alternativa é fazer do proxy uma subclasse da classe de serviço, e dessa forma ele herdar a interface do serviço.
- Crie a classe proxy. Ela deve ter um campo para armazenar uma referência ao serviço. Normalmente, os proxies criam e gerenciam todo o ciclo de vida de seus serviços. Em raras ocasiões, um serviço é passado para o proxy por meio de um construtor pelo cliente.
- Implemente os métodos proxy de acordo com suas finalidades. Na maioria dos casos, após realizar alguma tarefa, o proxy deve delegar o trabalho ao objeto de serviço.
- Considere introduzir um método de criação que decida se o cliente receberá um proxy ou um serviço real. Isso pode ser um método estático simples na classe proxy ou um método de fábrica completo.
- Considere implementar a inicialização preguiçosa para o objeto de serviço.

- Verifique se é possível fornecer uma interface mais simples do que a que um subsistema existente já oferece. Você está no caminho certo se essa interface tornar o código do cliente independente de muitas das classes do subsistema.
- Declare e implemente esta interface em uma nova classe de fachada. A fachada deve redirecionar as chamadas do código do cliente para os objetos apropriados do subsistema. A fachada deve ser responsável por inicializar o subsistema e gerenciar seu ciclo de vida subsequente, a menos que o código do cliente já faça isso.
- Para aproveitar ao máximo o padrão, faça com que todo o código do cliente se comunique com o subsistema somente por meio da fachada. Dessa forma, o código do cliente fica protegido contra quaisquer alterações no código do subsistema. Por exemplo, quando um subsistema for atualizado para uma nova versão, você precisará modificar apenas o código na fachada.
- Se a fachada ficar **muito grande**, considere extrair parte do seu comportamento para uma nova classe de fachada mais refinada

Padrões Comportamentais - Chain of Responsibility



É um padrão de projeto que permite passar solicitações ao longo de uma cadeia de manipuladores. Ao receber uma solicitação, cada manipulador decide processar a solicitação ou passá-la para o próximo manipulador na cadeia



Problemática



Imagine que você está trabalhando em um sistema de pedidos online. Você deseja restringir o acesso ao sistema para que apenas usuários autenticados possam criar pedidos. Além disso, usuários com permissões administrativas devem ter acesso total a todos os pedidos.

Após um pouco de planejamento, você percebeu que essas verificações devem ser realizadas sequencialmente. O aplicativo pode tentar autenticar um usuário no sistema sempre que receber uma solicitação contendo as credenciais do usuário. No entanto, se essas credenciais estiverem incorretas e a autenticação falhar, não há motivo para prosseguir com outras verificações.



Nos meses seguintes, você implementou várias outras dessas verificações sequenciais.

- Um dos seus colegas sugeriu que não é seguro passar dados brutos diretamente para o sistema de pedidos. Por isso, você adicionou uma etapa extra de validação para higienizar os dados na requisição.
- Mais tarde, alguém percebeu que o sistema era vulnerável a ataques de força bruta para quebrar senhas. Para evitar isso, você prontamente adicionou uma verificação que filtra solicitações repetidas com falha vindas do mesmo endereço IP.
- Outra pessoa sugeriu que você poderia acelerar o sistema retornando resultados em cache em solicitações repetidas contendo os mesmos dados. Portanto, você adicionou outra verificação que permite que a solicitação seja processada pelo sistema somente se não houver uma resposta em cache adequada.



O código das verificações, que já era confuso, ficou cada vez mais extenso à medida que você adicionava cada nova funcionalidade. Alterar uma verificação às vezes afetava as outras. Pior ainda, quando você tentava reutilizar as verificações para proteger outros componentes do sistema, era necessário duplicar parte do código, já que esses componentes exigiam algumas das verificações, mas não todas.

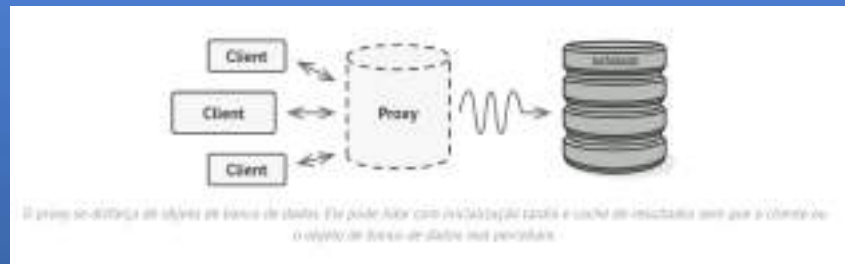
O sistema tornou-se muito difícil de compreender e caro de manter. Você lutou com o código por um tempo, até que um dia decidiu refatorar tudo.

Case 1 - Solução



O padrão Proxy sugere que você crie uma nova classe proxy com a mesma interface de um objeto de serviço original. Em seguida, você atualiza seu aplicativo para que ele passe o objeto proxy para todos os clientes do objeto original. Ao receber uma solicitação de um cliente, o proxy cria um objeto de serviço real e delega todo o trabalho a ele.

Mas qual é a vantagem? Se você precisar executar algo antes ou depois da lógica principal da classe, o proxy permite que você faça isso sem alterar a classe. Como o proxy implementa a mesma interface que a classe original, ele pode ser passado para qualquer cliente que espere um objeto de serviço real.



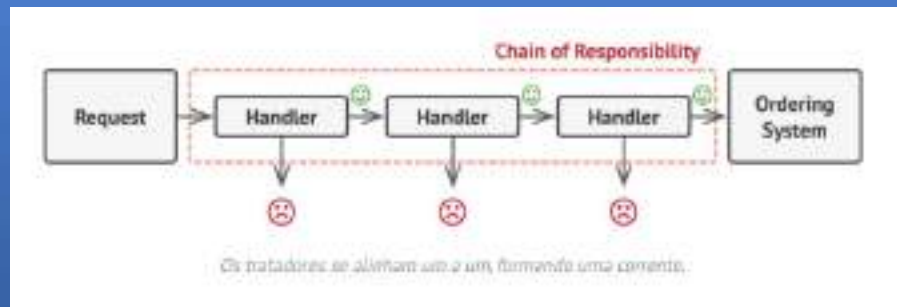
Assim como muitos outros padrões de projeto comportamentais, a **Cadeia de Responsabilidade** se baseia na transformação de comportamentos específicos em objetos independentes chamados *manipuladores* (*handlers*) . No nosso caso, cada verificação deve ser extraída para sua própria classe com um único método que realiza a verificação. A requisição, juntamente com seus dados, é passada para esse método como argumento.

O padrão sugere que você encadeie esses manipuladores. Cada manipulador encadeado possui um campo para armazenar uma referência ao próximo manipulador na cadeia. Além de processar uma solicitação, os manipuladores a repassam adiante na cadeia. A solicitação percorre a cadeia até que todos os manipuladores tenham tido a oportunidade de processá-la.

E aqui está a melhor parte: um manipulador pode decidir não repassar a solicitação adiante na cadeia e, efetivamente, interromper qualquer processamento adicional.

Em nosso exemplo com sistemas de pedidos, um manipulador realiza o processamento e, em seguida, decide se deve passar a solicitação adiante na cadeia. Supondo que a solicitação contenha os dados corretos, todos os manipuladores podem executar seu comportamento principal, seja ele verificar a autenticação ou armazenar em cache.

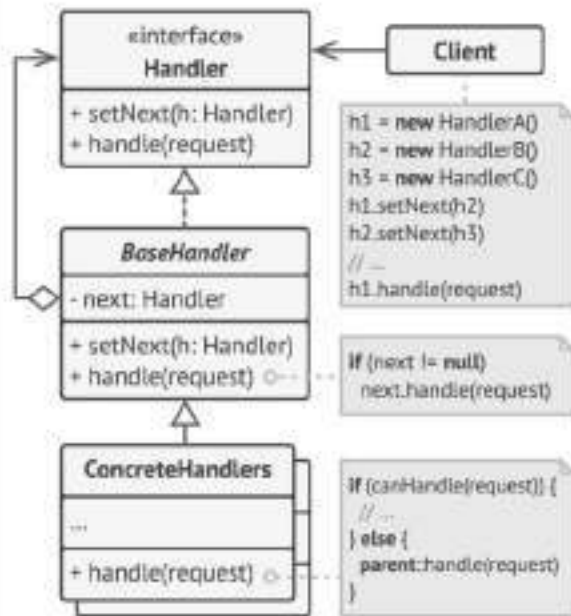
No entanto, existe uma abordagem ligeiramente diferente (e um pouco mais canônica) na qual, ao receber uma solicitação, um manipulador decide se pode processá-la. Se puder, ele não repassa a solicitação adiante. Portanto, ou apenas um manipulador processa a solicitação, ou nenhum. Essa abordagem é muito comum ao lidar com eventos em pilhas de elementos dentro de uma interface gráfica de usuário.



1 O **Handler** declara a interface, comum a todos os handlers concretos. Normalmente, ele contém apenas um único método para lidar com requisições, mas às vezes pode também ter outro método para definir o próximo handler na cadeia.

2 O manipulador base é uma classe opcional onde você pode inserir o código padrão que é comum a todas as classes de manipulador.

Normalmente, essa classe define um campo para armazenar uma referência ao próximo manipulador. Os clientes podem construir uma cadeia passando um manipulador para o construtor ou setter do manipulador anterior. A classe também pode implementar o comportamento de tratamento padrão:



4 O cliente pode compor cadeias de requisições uma única vez ou compô-las dinamicamente, dependendo da lógica da aplicação. Observe que uma requisição pode ser enviada para qualquer manipulador na cadeia – não precisa ser o primeiro.

3 Os manipuladores concretos contêm o código real para processar as solicitações. Ao receber uma solicitação, cada manipulador deve decidir se a processará e, adicionalmente, se a encaminhará adiante na cadeia.

Os manipuladores geralmente são autocontidos e imutáveis, aceitando todos os dados necessários apenas uma vez por meio do construtor.

Aplicabilidade - Chain of responsibility



- **Utilize o padrão Chain of Responsibility quando seu programa precisar processar diferentes tipos de requisições de diversas maneiras, mas os tipos exatos de requisições e suas sequências forem desconhecidos de antemão.**
- Esse padrão permite conectar vários manipuladores em uma única cadeia e, ao receber uma solicitação, "perguntar" a cada manipulador se ele pode processá-la. Dessa forma, todos os manipuladores têm a oportunidade de processar a solicitação.
- **Utilize esse padrão quando for essencial executar vários manipuladores em uma ordem específica.**
- Como você pode encadear os manipuladores em qualquer ordem, todas as solicitações passarão pela cadeia exatamente como planejado.
- **Utilize o padrão CoR quando o conjunto de manipuladores e sua ordem precisarem ser alterados em tempo de execução.**
- Se você fornecer métodos setter para um campo de referência dentro das classes de manipuladores, poderá inserir, remover ou reordenar manipuladores dinamicamente.

Como implementar - Chain of responsibility



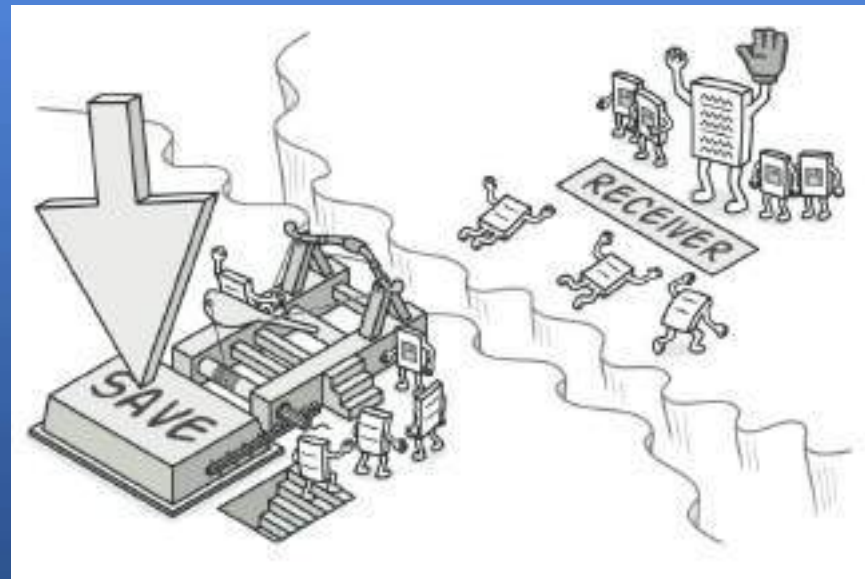
- Declare a interface do manipulador e descreva a assinatura de um método para lidar com as requisições. Decida como o cliente passará os dados da solicitação para o método. A maneira mais flexível é converter a solicitação em um objeto e passá-lo para o método de tratamento como um argumento.
- Para eliminar código repetitivo em manipuladores concretos, pode ser útil criar uma classe base abstrata para o manipulador, derivada da interface do manipulador. Essa classe deve ter um campo para armazenar uma referência ao próximo manipulador na cadeia. Considere tornar a classe imutável. No entanto, se você planeja modificar as cadeias em tempo de execução, precisará definir um método setter para alterar o valor do campo de referência. Você também pode implementar o comportamento padrão conveniente para o método de tratamento, que é encaminhar a solicitação para o próximo objeto, a menos que não haja mais nenhum disponível. Os manipuladores concretos poderão usar esse comportamento chamando o método pai.

- Crie, uma a uma, subclasses concretas de manipuladores e implemente seus métodos de tratamento. Cada manipulador deve tomar duas decisões ao receber uma solicitação:
 - Se o pedido será processado ou não.
 - Se a solicitação será repassada ao longo da cadeia.
- O cliente pode montar cadeias por conta própria ou receber cadeias pré-construídas de outros objetos. Neste último caso, você deve implementar algumas classes de fábrica para construir as cadeias de acordo com a configuração ou as definições de ambiente.
- O cliente pode acionar qualquer manipulador na cadeia, não apenas o primeiro. A solicitação será passada ao longo da cadeia até que algum manipulador se recuse a encaminhá-la adiante ou até que ela chegue ao final da cadeia.
- Devido à natureza dinâmica da cadeia, o cliente deve estar preparado para lidar com os seguintes cenários:
 - A corrente pode ser composta por um único elo.
 - Algumas solicitações podem não chegar ao final da cadeia.
 - Outros podem chegar ao fim da cadeia sem serem tratados.

Padrões Comportamentais - Command



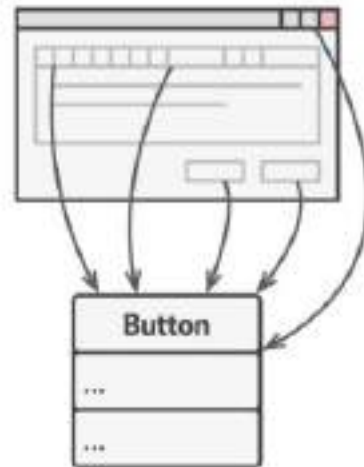
É um padrão de projeto que permite transformar uma requisição em um objeto independente que contém todas as informações sobre a requisição. Essa transformação permite passar requisições como argumentos de um método, atrasar ou enfileirar a execução de uma requisição e suportar operações reversíveis.



Problemática



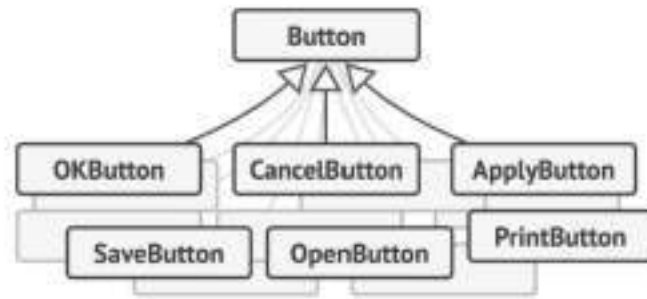
Imagine que você está trabalhando em um novo aplicativo de edição de texto. Sua tarefa atual é criar uma barra de ferramentas com vários botões para diferentes operações do editor. Você criou uma `Button` classe muito interessante que pode ser usada tanto para os botões da barra de ferramentas quanto para botões genéricos em diversas caixas de diálogo.



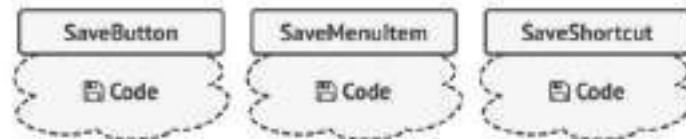
Todos os botões do aplicativo são derivados da mesma classe.

Embora todos esses botões pareçam semelhantes, eles devem executar funções diferentes. Onde você colocaria o código para os vários manipuladores de clique desses botões? A solução mais simples é criar várias subclasses para cada local onde o botão é usado. Essas subclasses conteriam o código que precisaria ser executado ao clicar no botão.

Em pouco tempo, você percebe que essa abordagem é profundamente falha. Primeiro, você tem um número enorme de subclasses, o que não seria um problema se você não corresse o risco de quebrar o código nessas subclasses cada vez que modificasse a `Button` classe base. Simplificando, seu código de interface gráfica tornou-se inexplicavelmente dependente do código volátil da lógica de negócios.



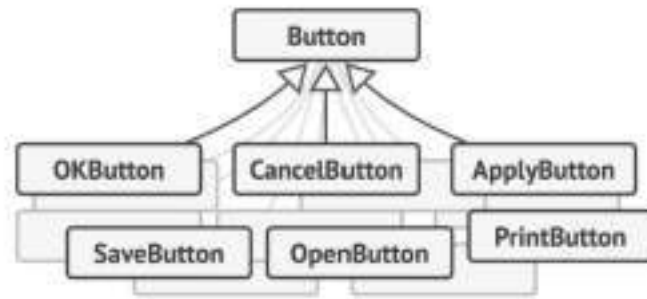
Muitas subclasses de botões. O que pode dar errado?



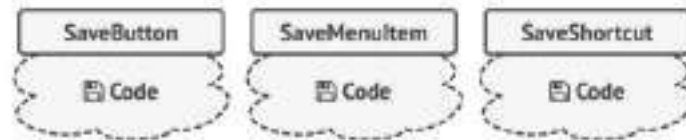
Diversas classes implementam a mesma funcionalidade.

E aqui está a parte mais complicada. Algumas operações, como copiar/colar texto, precisariam ser invocadas de vários lugares. Por exemplo, um usuário poderia clicar em um pequeno botão "Copiar" na barra de ferramentas, copiar algo pelo menu de contexto ou simplesmente pressionar uma `Ctrl+C` tecla no teclado.

Inicialmente, quando nosso aplicativo tinha apenas a barra de ferramentas, era aceitável colocar a implementação de várias operações nas subclasses dos botões. Em outras palavras, ter o código para copiar texto dentro da `CopyButton` subclasse funcionava bem. Mas, ao implementar menus de contexto, atalhos e outros recursos, tivemos que duplicar o código da operação em várias classes ou tornar os menus dependentes dos botões, o que é uma opção ainda pior.



Muitas subclasses de botões. O que pode dar errado?



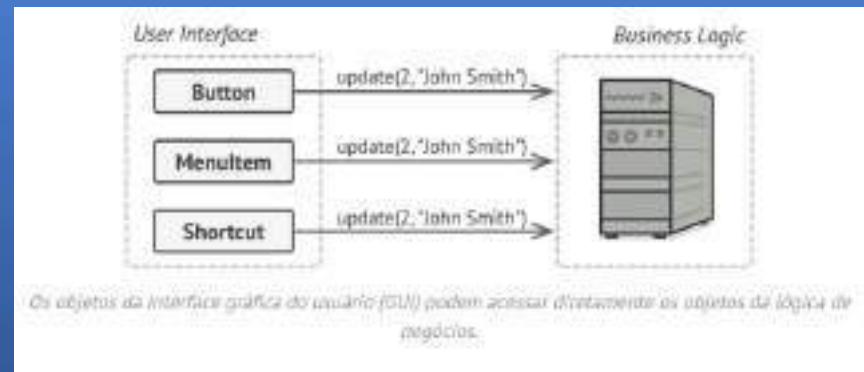
Diversas classes implementam a mesma funcionalidade.

Case 1 - Solução



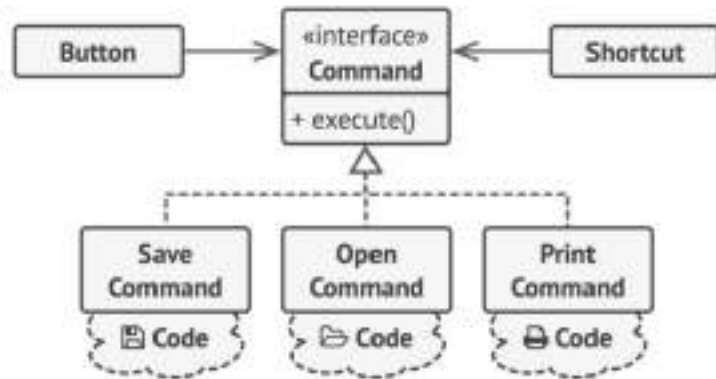
Um bom design de software geralmente se baseia no *princípio da separação de responsabilidades*, o que normalmente resulta na divisão de um aplicativo em camadas. O exemplo mais comum: uma camada para a interface gráfica do usuário (GUI) e outra para a lógica de negócios. A camada da GUI é responsável por renderizar uma imagem atraente na tela, capturar qualquer entrada e exibir os resultados das ações do usuário e do aplicativo. No entanto, quando se trata de realizar algo importante, como calcular a trajetória da Lua ou elaborar um relatório anual, a camada da GUI delega essa tarefa à camada subjacente de lógica de negócios.

No código, isso pode se parecer com isto: um objeto da interface gráfica chama um método de um objeto da lógica de negócios, passando alguns argumentos. Esse processo geralmente é descrito como um objeto enviando uma *solicitação* para outro.



O próximo passo é fazer com que seus comandos implementem a mesma interface. Normalmente, ela possui apenas um único método de execução que não recebe parâmetros. Essa interface permite usar vários comandos com o mesmo remetente de requisição, sem acoplá-lo a classes concretas de comandos. Como bônus, agora você pode alternar entre objetos de comando vinculados ao remetente, alterando efetivamente o comportamento do remetente em tempo de execução.

Você deve ter notado que falta uma peça fundamental nesse quebra-cabeça: os parâmetros da requisição. Um objeto da interface gráfica pode ter fornecido alguns parâmetros ao objeto da camada de negócios. Como o método de execução do comando não possui parâmetros, como passaríamos os detalhes da requisição para o receptor? Acontece que o comando deve ser pré-configurado com esses dados ou ser capaz de obtê-los por conta própria.



Os objetos da interface gráfica delegam o trabalho aos comandos.

Vamos voltar ao nosso editor de texto. Depois de aplicarmos o padrão Command, não precisamos mais de todas aquelas subclasses de botão para implementar os diversos comportamentos de clique. Basta adicionar um único campo à `Button` classe base que armazena uma referência a um objeto de comando e fazer com que o botão execute esse comando ao ser clicado.

Você implementará diversas classes de comando para cada operação possível e as vinculará a botões específicos, dependendo do comportamento pretendido dos botões.

Outros elementos da interface gráfica, como menus, atalhos ou diálogos inteiros, podem ser implementados da mesma forma. Eles serão vinculados a um comando que será executado quando o usuário interagir com o elemento da interface. Como você provavelmente já deve ter imaginado, os elementos relacionados às mesmas operações serão vinculados aos mesmos comandos, evitando qualquer duplicação de código.

Como resultado, os comandos se tornam uma camada intermediária conveniente que reduz o acoplamento entre a interface gráfica do usuário (GUI) e a lógica de negócios. E isso é apenas uma fração dos benefícios que o padrão Command pode oferecer!

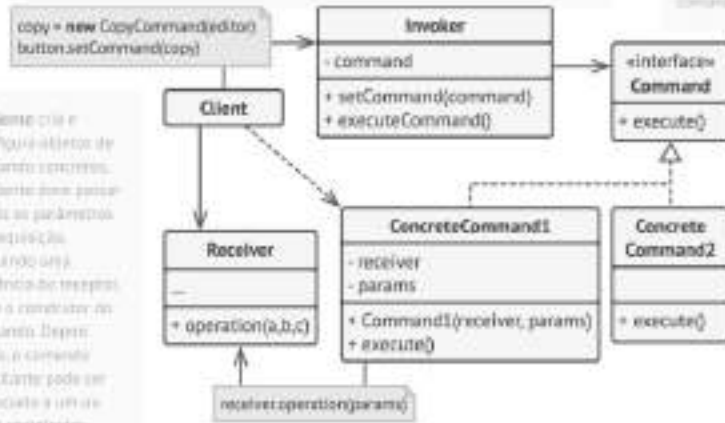
1. A classe **Invoker** (também conhecida como **invocador**) é responsável por iniciar as requisições. Essa classe deve ter um campo para armazenar uma referência a um objeto de comando. O invocado atribui esse comando em vez de enviar a requisição diretamente ao destinatário. Observe que o invocado não é responsável por criar o objeto de comando. Normalmente, ele cria um comando por conta do cliente por meio do construtor.

2. A **Interface Command** geralmente declara apenas um único método (**execute()**), chamado de **comando**.

3. Os **comandos concretos** implementam cada tipo de requisição. Um comando como objeto não deve executar o trabalho por si só, mas sim, passar a responsabilidade para um dos objetos de lógica de negócios. Em outras palavras, para simplificar o código, essas classes podem ser pequenas.

Os parâmetros necessários para executar um método em um objeto receptor podem ser declarados como campos na classe concreta. Isso pode tornar os objetos de comando inválidos por si só. Isso pode ser evitado permitindo a inicialização de uma classe apenas por meio de um construtor.

5. O cliente cria e configura objetos de comando concretos. O cliente deve passar todos os parâmetros da requisição, incluindo uma referência ao receptor, para o construtor do comando. Depois disso, o comando resultante pode ser associado a um ou mais invocadores.



4. A classe **Receiver** contém alguma lógica de negócios. Quase qualquer objeto pode atuar como um receptor. A maioria dos comandos apenas lida com as detalhes de como uma solicitação é passada para o receptor, enquanto o próprio receptor realiza o trabalho em si.

Aplicabilidade - Command



- **Utilize o padrão Command quando desejar parametrizar objetos com operações.**
- O padrão Command permite transformar uma chamada de método específica em um objeto independente. Essa mudança abre um leque de possibilidades interessantes: você pode passar comandos como argumentos de método, armazená-los dentro de outros objetos, alternar entre comandos vinculados em tempo de execução, etc.
- Eis um exemplo: você está desenvolvendo um componente de interface gráfica, como um menu de contexto, e deseja que seus usuários possam configurar itens de menu que acionem operações quando o usuário clicar em um item.
- **Utilize o padrão Command quando desejar enfileirar operações, agendar sua execução ou executá-las remotamente.**
- Assim como qualquer outro objeto, um comando pode ser serializado, ou seja, convertido em uma string que pode ser facilmente gravada em um arquivo ou banco de dados. Posteriormente, a string pode ser restaurada como o objeto de comando original. Dessa forma, é possível atrasar e agendar a execução de comandos. Mas há ainda mais! Da mesma forma, você pode enfileirar, registrar ou enviar comandos pela rede.

- **Utilize o padrão Command quando desejar implementar operações reversíveis.**
- Embora existam muitas maneiras de implementar desfazer/refazer, o padrão Command é talvez o mais popular de todos.
- Para poder reverter operações, é necessário implementar o histórico das operações realizadas. O histórico de comandos é uma pilha que contém todos os objetos de comando executados, juntamente com os respectivos backups do estado da aplicação.
- Esse método tem duas desvantagens. Primeiro, não é tão fácil salvar o estado de um aplicativo, pois parte dele pode ser privada.
- Em segundo lugar, os backups de estado podem consumir bastante RAM. Portanto, às vezes, você pode recorrer a uma implementação alternativa: em vez de restaurar o estado anterior, o comando executa a operação inversa. A operação inversa também tem um preço: pode ser difícil ou até mesmo impossível de implementar.

Como implementar - Command

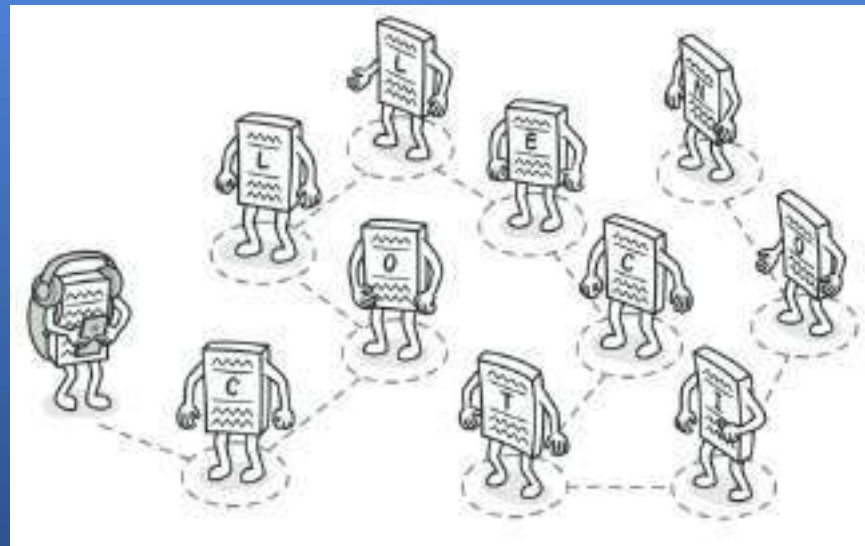


- Declare a interface de comando com um único método de execução.
- Comece extraindo as requisições para classes de comando concretas que implementem a interface de comando. Cada classe deve ter um conjunto de campos para armazenar os argumentos da requisição, juntamente com uma referência ao objeto receptor propriamente dito. Todos esses valores devem ser inicializados através do construtor do comando.
- Identifique as classes que atuarão como *remetentes* . Adicione os campos para armazenar comandos nessas classes. Os remetentes devem se comunicar com seus comandos somente por meio da interface de comando. Normalmente, os remetentes não criam objetos de comando por conta própria, mas os obtêm do código do cliente.
- Alterar os remetentes para que executem o comando em vez de enviar uma solicitação diretamente ao destinatário.
- O cliente deve inicializar os objetos na seguinte ordem:
 - Criar receptores.
 - Crie comandos e associe-os a receptores, se necessário.
 - Crie remetentes e associe-os a comandos específicos.

Padrões Comportamentais - Iterator



É um padrão de projeto que permite que permite percorrer os elementos de uma coleção sem expor sua representação subjacente (lista, pilha, árvore, etc.)



Problemática



As coleções são um dos tipos de dados mais utilizados em programação. No entanto, uma coleção é apenas um contêiner para um grupo de objetos.

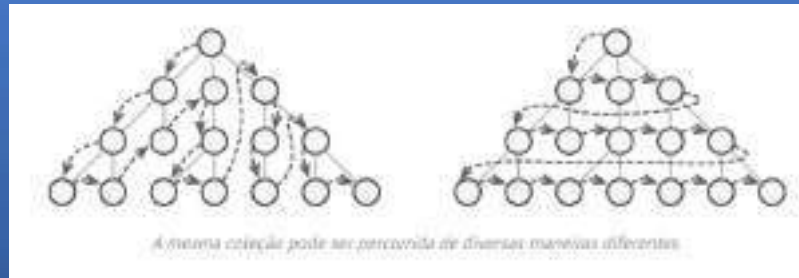
A maioria das coleções armazena seus elementos em listas simples. No entanto, algumas são baseadas em pilhas, árvores, grafos e outras estruturas de dados complexas.

Mas, independentemente da estrutura de uma coleção, ela deve fornecer alguma forma de acessar seus elementos para que outros códigos possam utilizá-los. Deve haver uma maneira de percorrer cada elemento da coleção sem acessar os mesmos elementos repetidamente.



Isso pode parecer uma tarefa fácil se você tiver uma coleção baseada em uma lista. Basta percorrer todos os elementos em um loop. Mas como percorrer sequencialmente os elementos de uma estrutura de dados complexa, como uma árvore? Por exemplo, um dia você pode se dar bem com uma busca em profundidade em uma árvore. No dia seguinte, você pode precisar de uma busca em largura. E na semana seguinte, você pode precisar de algo diferente, como acesso aleatório aos elementos da árvore.

Por outro lado, o código do cliente que deve interagir com várias coleções pode nem se importar com a forma como elas armazenam seus elementos. No entanto, como as coleções oferecem maneiras diferentes de acessar seus elementos, você não tem outra opção a não ser acoplar seu código às classes de coleção específicas.



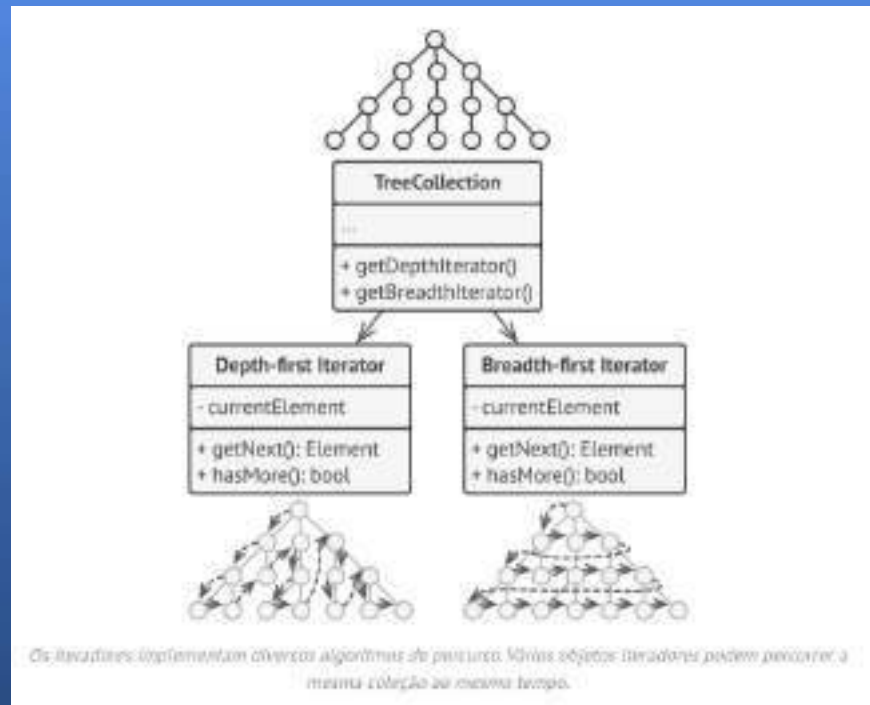
Case 1 - Solução



Além de implementar o próprio algoritmo, um objeto iterador encapsula todos os detalhes da travessia, como a posição atual e quantos elementos faltam para o final. Por causa disso, vários iteradores podem percorrer a mesma coleção ao mesmo tempo, independentemente uns dos outros.

Normalmente, os iteradores fornecem um método principal para buscar elementos da coleção. O cliente pode continuar executando esse método até que ele não retorne nada, o que significa que o iterador percorreu todos os elementos.

Todos os iteradores devem implementar a mesma interface. Isso torna o código do cliente compatível com qualquer tipo de coleção ou qualquer algoritmo de percurso, desde que haja um iterador adequado. Se você precisar de uma maneira específica de percorrer uma coleção, basta criar uma nova classe de iterador, sem precisar alterar a coleção ou o cliente.



Vamos voltar ao nosso editor de texto. Depois de aplicarmos o padrão Command, não precisamos mais de todas aquelas subclasses de botão para implementar os diversos comportamentos de clique. Basta adicionar um único campo à `Button` classe base que armazena uma referência a um objeto de comando e fazer com que o botão execute esse comando ao ser clicado.

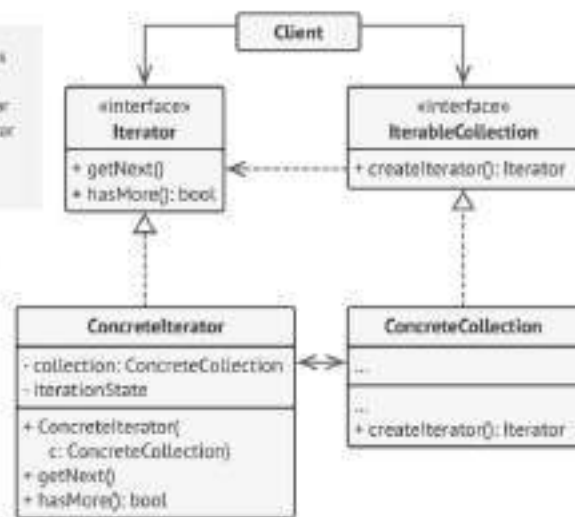
Você implementará diversas classes de comando para cada operação possível e as vinculará a botões específicos, dependendo do comportamento pretendido dos botões.

Outros elementos da interface gráfica, como menus, atalhos ou diálogos inteiros, podem ser implementados da mesma forma. Eles serão vinculados a um comando que será executado quando o usuário interagir com o elemento da interface. Como você provavelmente já deve ter imaginado, os elementos relacionados às mesmas operações serão vinculados aos mesmos comandos, evitando qualquer duplicação de código.

Como resultado, os comandos se tornam uma camada intermediária conveniente que reduz o acoplamento entre a interface gráfica do usuário (GUI) e a lógica de negócios. E isso é apenas uma fração dos benefícios que o padrão Command pode oferecer!

1 A interface `Iterator` declara as operações necessárias para percorrer uma coleção: buscar o próximo elemento, recuperar a posição atual, reiniciar a iteração, etc.

2 Iteradores concretos implementam algoritmos específicos para percorrer uma coleção. O objeto iterador deve acompanhar o progresso da travessia por conta própria. Isso permite que vários iteradores percorram a mesma coleção independentemente uns dos outros.



3 O cliente interage com coleções e iteradores por meio de suas interfaces. Dessa forma, o cliente não fica acoplado a classes concretas, permitindo que use várias coleções e iteradores com o mesmo código do cliente.

Atualmente, os clientes não usam iteradores por conta própria, mas os obtêm da coleção. No entanto, em certos casos, o cliente pode criar um iterador em si, por exemplo, quando define seu próprio iterador específico.

4 A interface `Collection` declara um ou mais métodos para obter iteradores compatíveis com a coleção. Observe que o tipo de retorno dos métodos deve ser declarado como a interface `Iterator` para que as coleções concretas possam retornar vários tipos de iteradores.

5 As coleções concretas retornam novas instâncias de uma classe iteradora concreta específica sempre que o cliente faz uma solicitação. Você pode estar se perguntando onde está o restante do código da coleção? Não se preocupe, ele deve estar na mesma classe. Lembre-se que esses detalhes não são cruciais para o padrão em si, então os exibimos omitidos.

Aplicabilidade - Iterator



- **Utilize o padrão Iterator quando sua coleção tiver uma estrutura de dados complexa internamente, mas você quiser ocultar essa complexidade dos clientes (seja por conveniência ou por motivos de segurança).**
- O iterador encapsula os detalhes do trabalho com uma estrutura de dados complexa, fornecendo ao cliente vários métodos simples para acessar os elementos da coleção. Embora essa abordagem seja muito conveniente para o cliente, ela também protege a coleção de ações descuidadas ou maliciosas que o cliente poderia realizar se trabalhasse diretamente com a coleção.
- **Utilize esse padrão para reduzir a duplicação do código de percurso em todo o seu aplicativo.**
- O código de algoritmos de iteração não triviais tende a ser muito extenso. Quando inserido na lógica de negócios de um aplicativo, pode obscurecer a responsabilidade do código original e dificultar a manutenção. Mover o código de percurso para iteradores designados pode ajudar a tornar o código do aplicativo mais enxuto e limpo.
- **Utilize o Iterator quando desejar que seu código possa percorrer diferentes estruturas de dados ou quando os tipos dessas estruturas forem desconhecidos de antemão.**
- O padrão fornece algumas interfaces genéricas tanto para coleções quanto para iteradores. Dado que seu código agora usa essas interfaces, ele ainda funcionará se você passar para ele vários tipos de coleções e iteradores que implementam essas interfaces.

Como implementar - Iterator

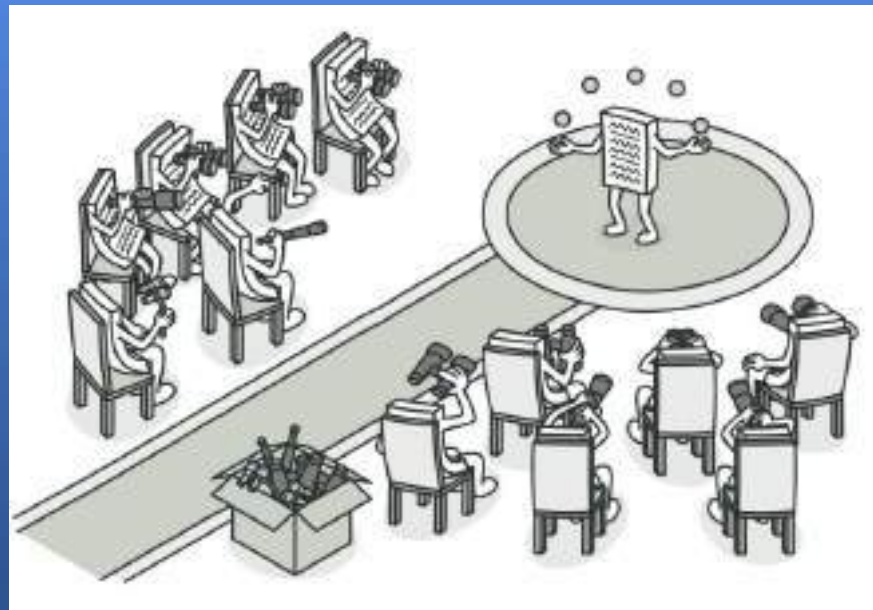


- Declare a interface do iterador. No mínimo, ela deve ter um método para buscar o próximo elemento de uma coleção. Mas, para maior conveniência, você pode adicionar alguns outros métodos, como buscar o elemento anterior, rastrear a posição atual e verificar o fim da iteração.
- Declare a interface da coleção e descreva um método para buscar iteradores. O tipo de retorno deve ser igual ao da interface do iterador. Você pode declarar métodos semelhantes se planeja ter vários grupos distintos de iteradores.
- Implemente classes de iteradores concretas para as coleções que você deseja que sejam percorráveis com iteradores. Um objeto iterador deve ser vinculado a uma única instância de coleção. Normalmente, essa vinculação é estabelecida por meio do construtor do iterador.
- Implemente a interface de coleção em suas classes de coleção. A ideia principal é fornecer ao cliente um atalho para criar iteradores, personalizados para uma classe de coleção específica. O objeto de coleção deve passar a si mesmo para o construtor do iterador para estabelecer uma ligação entre eles.
- Analise o código do cliente para substituir todo o código de percurso da coleção pelo uso de iteradores. O cliente busca um novo objeto iterador cada vez que precisa iterar sobre os elementos da coleção.

Padrões Comportamentais - Observer



É um padrão de projeto que permite definir um mecanismo de assinatura para notificar vários objetos sobre quaisquer eventos que ocorram com o objeto que eles estão observando.



Problemática

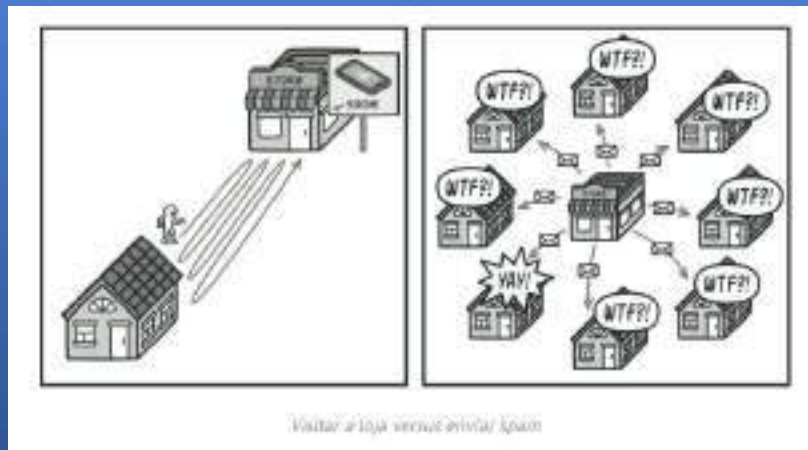


Imagine que você tem dois tipos de objetos: um `Customer` e outro `Store`. O cliente está muito interessado em uma determinada marca de produto (digamos, um novo modelo do iPhone) que deverá estar disponível na loja em breve.

O cliente poderia visitar a loja todos os dias e verificar a disponibilidade do produto. Mas, enquanto o produto estiver a caminho, a maioria dessas visitas seria inútil.

Por outro lado, a loja poderia enviar inúmeros e-mails (que poderiam ser considerados spam) para todos os clientes cada vez que um novo produto fosse lançado. Isso evitaria que alguns clientes tivessem que ir à loja inúmeras vezes. Ao mesmo tempo, isso desagradaria outros clientes que não têm interesse em novos produtos.

Parece que temos um conflito. Ou o cliente perde tempo verificando a disponibilidade do produto, ou a loja desperdiça recursos notificando os clientes errados.



Case 1 - Solução



O objeto que possui um estado relevante é frequentemente chamado de *sujeito* , mas como ele também notifica outros objetos sobre as mudanças em seu estado, vamos chamá-lo de *publicador* . Todos os outros objetos que desejam acompanhar as mudanças no estado do publicador são chamados de *assinantes* .

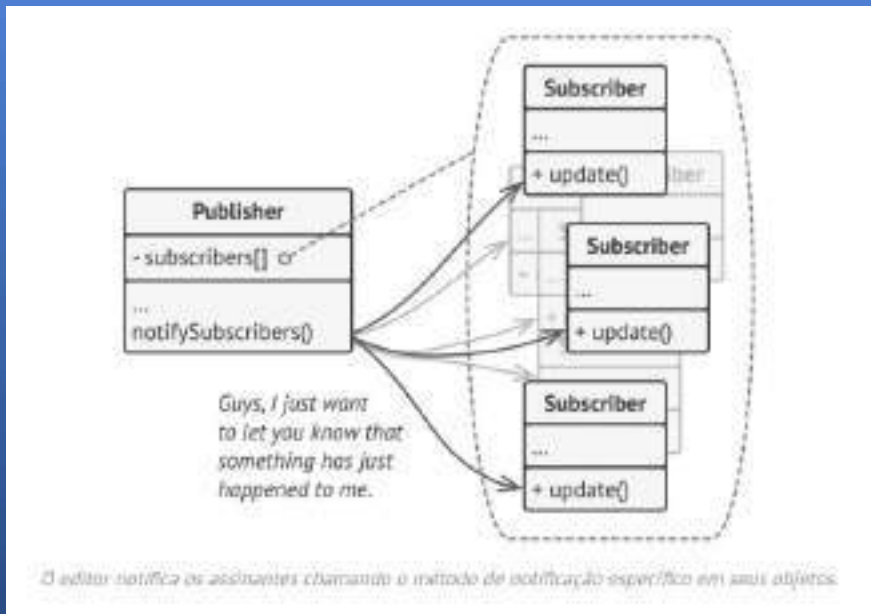
O padrão Observer sugere que você adicione um mecanismo de assinatura à classe publicadora para que objetos individuais possam se inscrever ou cancelar a assinatura de um fluxo de eventos provenientes dessa publicadora. Não se preocupe! Tudo não é tão complicado quanto parece. Na realidade, esse mecanismo consiste em 1) um campo de array para armazenar uma lista de referências a objetos assinantes e 2) vários métodos públicos que permitem adicionar e remover assinantes dessa lista.



Agora, sempre que um evento importante acontece com o editor, ele percorre seus assinantes e chama o método de notificação específico em seus objetos.

Aplicativos reais podem ter dezenas de classes de assinantes diferentes interessadas em rastrear eventos da mesma classe publicadora. Você não gostaria de acoplar a classe publicadora a todas essas classes. Além disso, você pode nem saber da existência de algumas delas antecipadamente, caso sua classe publicadora seja destinada a ser usada por outras pessoas.

Por isso, é crucial que todos os assinantes implementem a mesma interface e que o publicador se comunique com eles somente por meio dessa interface. Essa interface deve declarar o método de notificação juntamente com um conjunto de parâmetros que o publicador pode usar para passar dados contextuais junto com a notificação.



Se seu aplicativo possui vários tipos diferentes de editores e você deseja que seus assinantes sejam compatíveis com todos eles, você pode ir ainda mais longe e fazer com que todos os editores sigam a mesma interface. Essa interface precisaria descrever apenas alguns métodos de assinatura. A interface permitiria que os assinantes observassem os estados dos editores sem se acoplarem às suas classes concretas.

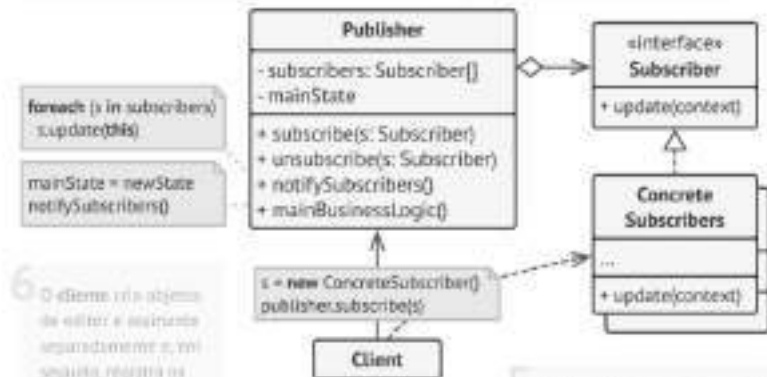
1. O **Publisher** cria eventos de interesse para outros objetos. Esses eventos ocorrem quando o **publisher** altera seu estado ou em uma determinada comportamento. Os **Publishers** contém uma infraestrutura de assinatura que permite que vários assinantes sintam e que os assinantes atualizem sua lista.

2. Quando ocorre um novo evento, o editor percorre a lista de assinantes e chama o método de notificação declarado na interface do assinante em cada objeto assinante.

3. A interface **Subscriber** declara a interface de notificação. Na maioria dos casos, ela consiste em um único método abstrato. O método pode ter vários parâmetros que permitem ao **publisher** passar alguns detalhes do evento juntamente com a atualização.

4. Os assinantes concretos executam algumas ações em resposta às notificações emitidas pelo **publisher**. Todas essas classes devem implementar a mesma interface para que o **publisher** não fique acoplado às classes concretas.

5. Normalmente, os assinantes precisam de algumas informações contextuais para processar a atualização corretamente. Por esse motivo, os editores costumam passar alguns dados contextuais como argumentos do método de notificação. O editor pode passar à si mesmo como argumento, permitindo que o assinante busque diretamente os dados necessários.



6. O cliente cria objetos de editor e assinante separadamente e, em seguida, registra os assinantes para receber atualizações do editor.

Aplicabilidade - Observer



- **Utilize o padrão Observer quando as mudanças no estado de um objeto poderem exigir a alteração de outros objetos, e o conjunto real de objetos for desconhecido antecipadamente ou mudar dinamicamente.**
- Você pode se deparar com esse problema frequentemente ao trabalhar com classes da interface gráfica do usuário. Por exemplo, você criou classes de botões personalizadas e deseja permitir que os clientes associam algum código personalizado a esses botões para que ele seja executado sempre que um usuário pressionar um botão.
- O padrão Observer permite que qualquer objeto que implemente a interface Subscriber se inscreva para receber notificações de eventos em objetos Publisher. Você pode adicionar o mecanismo de inscrição aos seus botões, permitindo que os clientes conectem seu código personalizado por meio de classes Subscriber personalizadas.
- **Utilize esse padrão quando alguns objetos em seu aplicativo precisar observar outros, mas apenas por um período limitado ou em casos específicos.**
- A lista de assinantes é dinâmica, permitindo que os assinantes entrem ou saiam da lista quando precisarem.

Como implementar - Observer



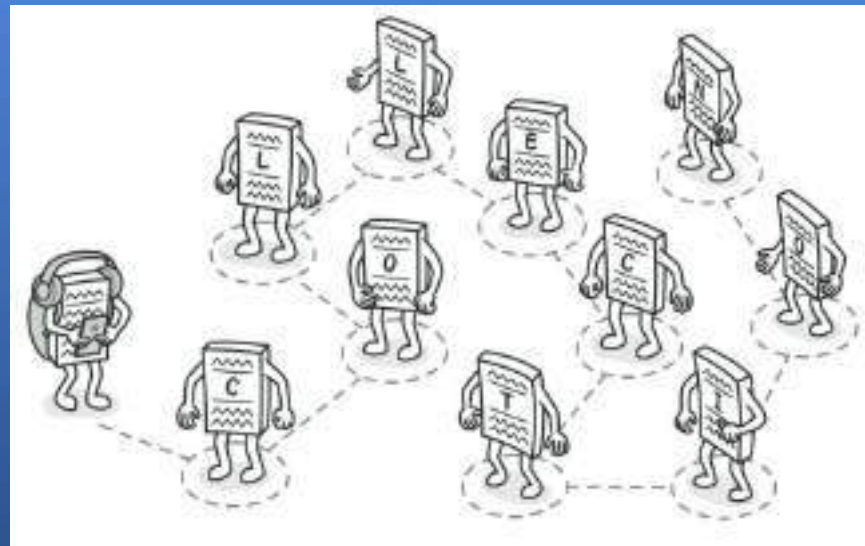
- Analise a sua lógica de negócios e tente dividi-la em duas partes: a funcionalidade principal, independente de outros códigos, atuará como publicadora; o restante se transformará em um conjunto de classes de assinantes.
- Declare a interface do assinante. No mínimo, ela deve declarar um único `update` método.
- Declare a interface do editor e descreva um par de métodos para adicionar e remover um objeto assinante da lista. Lembre-se de que os editores devem interagir com os assinantes somente por meio da interface do assinante.
- Decida onde colocar a lista de assinantes e a implementação dos métodos de assinatura. Normalmente, esse código é semelhante para todos os tipos de publicadores, então o local mais óbvio para colocá-lo é em uma classe abstrata derivada diretamente da interface do publicador. Publicadores concretos estendem essa classe, herdando o comportamento de assinatura. No entanto, se você estiver aplicando o padrão a uma hierarquia de classes existente, considere uma abordagem baseada em composição: coloque a lógica de assinatura em um objeto separado e faça com que todos os publicadores reais o utilizem.

- Crie classes de publicação concretas. Sempre que algo importante acontecer dentro de uma classe de publicação, ela deve notificar todos os seus assinantes.
- Implemente os métodos de notificação de atualização em classes de assinante concretas. A maioria dos assinantes precisará de alguns dados contextuais sobre o evento. Esses dados podem ser passados como argumento do método de notificação.
Mas existe outra opção. Ao receber uma notificação, o assinante pode obter os dados diretamente da notificação. Nesse caso, o publicador deve passar a si mesmo através do método de atualização. A opção menos flexível é vincular o publicador ao assinante permanentemente através do construtor.
- O cliente deve criar todos os assinantes necessários e registrá-los junto às editoras apropriadas.

Padrões Comportamentais - Iterator



É um padrão de projeto que permite que permite percorrer os elementos de uma coleção sem expor sua representação subjacente (lista, pilha, árvore, etc.)



Problemática



As coleções são um dos tipos de dados mais utilizados em programação. No entanto, uma coleção é apenas um contêiner para um grupo de objetos.

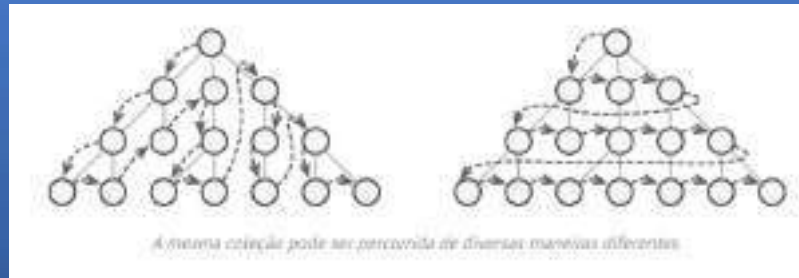
A maioria das coleções armazena seus elementos em listas simples. No entanto, algumas são baseadas em pilhas, árvores, grafos e outras estruturas de dados complexas.

Mas, independentemente da estrutura de uma coleção, ela deve fornecer alguma forma de acessar seus elementos para que outros códigos possam utilizá-los. Deve haver uma maneira de percorrer cada elemento da coleção sem acessar os mesmos elementos repetidamente.



Isso pode parecer uma tarefa fácil se você tiver uma coleção baseada em uma lista. Basta percorrer todos os elementos em um loop. Mas como percorrer sequencialmente os elementos de uma estrutura de dados complexa, como uma árvore? Por exemplo, um dia você pode se dar bem com uma busca em profundidade em uma árvore. No dia seguinte, você pode precisar de uma busca em largura. E na semana seguinte, você pode precisar de algo diferente, como acesso aleatório aos elementos da árvore.

Por outro lado, o código do cliente que deve interagir com várias coleções pode nem se importar com a forma como elas armazenam seus elementos. No entanto, como as coleções oferecem maneiras diferentes de acessar seus elementos, você não tem outra opção a não ser acoplar seu código às classes de coleção específicas.



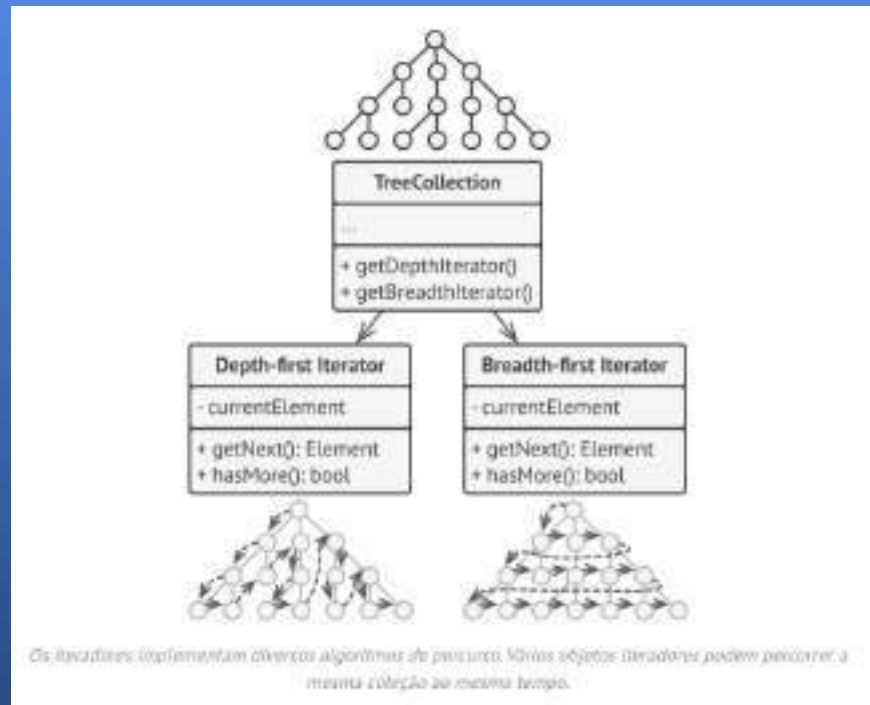
Case 1 - Solução



Além de implementar o próprio algoritmo, um objeto iterador encapsula todos os detalhes da travessia, como a posição atual e quantos elementos faltam para o final. Por causa disso, vários iteradores podem percorrer a mesma coleção ao mesmo tempo, independentemente uns dos outros.

Normalmente, os iteradores fornecem um método principal para buscar elementos da coleção. O cliente pode continuar executando esse método até que ele não retorne nada, o que significa que o iterador percorreu todos os elementos.

Todos os iteradores devem implementar a mesma interface. Isso torna o código do cliente compatível com qualquer tipo de coleção ou qualquer algoritmo de percurso, desde que haja um iterador adequado. Se você precisar de uma maneira específica de percorrer uma coleção, basta criar uma nova classe de iterador, sem precisar alterar a coleção ou o cliente.



Vamos voltar ao nosso editor de texto. Depois de aplicarmos o padrão Command, não precisamos mais de todas aquelas subclasses de botão para implementar os diversos comportamentos de clique. Basta adicionar um único campo à `Button` classe base que armazena uma referência a um objeto de comando e fazer com que o botão execute esse comando ao ser clicado.

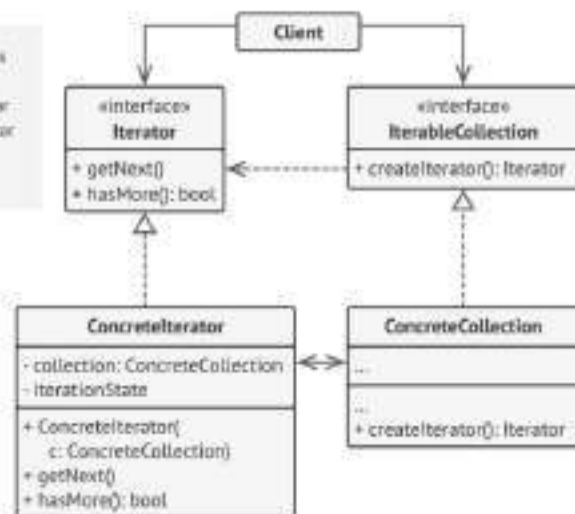
Você implementará diversas classes de comando para cada operação possível e as vinculará a botões específicos, dependendo do comportamento pretendido dos botões.

Outros elementos da interface gráfica, como menus, atalhos ou diálogos inteiros, podem ser implementados da mesma forma. Eles serão vinculados a um comando que será executado quando o usuário interagir com o elemento da interface. Como você provavelmente já deve ter imaginado, os elementos relacionados às mesmas operações serão vinculados aos mesmos comandos, evitando qualquer duplicação de código.

Como resultado, os comandos se tornam uma camada intermediária conveniente que reduz o acoplamento entre a interface gráfica do usuário (GUI) e a lógica de negócios. E isso é apenas uma fração dos benefícios que o padrão Command pode oferecer!

1 A interface `Iterator` declara as operações necessárias para percorrer uma coleção: buscar o próximo elemento, recuperar a posição atual, reiniciar a iteração, etc.

2 Iteradores concretos implementam algoritmos específicos para percorrer uma coleção. O objeto iterador deve acompanhar o progresso da travessia por conta própria. Isso permite que vários iteradores percorram a mesma coleção independentemente uns dos outros.



3 O cliente interage com coleções e iteradores por meio de suas interfaces. Dessa forma, o cliente não fica acoplado a classes concretas, permitindo que use várias coleções e iteradores com o mesmo código do cliente.

Atualmente, os clientes não usam iteradores por conta própria, mas os obtêm da coleção. No entanto, em certos casos, o cliente pode criar um iterador em si, por exemplo, quando define seu próprio iterador específico.

4 A interface `Collection` declara um ou mais métodos para obter iteradores compatíveis com a coleção. Observe que o tipo de retorno dos métodos deve ser declarado como a interface `Iterator` para que as coleções concretas possam retornar vários tipos de iteradores.

5 As coleções concretas retornam novas instâncias de uma classe iteradora concreta específica sempre que o cliente faz uma solicitação. Você pode estar se perguntando onde está o restante do código da coleção? Não se preocupe, ele deve estar na mesma classe. Lembre-se que esses detalhes não são cruciais para o padrão em si, então os exibimos omitidos.

Aplicabilidade - Iterator



- **Utilize o padrão Iterator quando sua coleção tiver uma estrutura de dados complexa internamente, mas você quiser ocultar essa complexidade dos clientes (seja por conveniência ou por motivos de segurança).**
- O iterador encapsula os detalhes do trabalho com uma estrutura de dados complexa, fornecendo ao cliente vários métodos simples para acessar os elementos da coleção. Embora essa abordagem seja muito conveniente para o cliente, ela também protege a coleção de ações descuidadas ou maliciosas que o cliente poderia realizar se trabalhasse diretamente com a coleção.
- **Utilize esse padrão para reduzir a duplicação do código de percurso em todo o seu aplicativo.**
- O código de algoritmos de iteração não triviais tende a ser muito extenso. Quando inserido na lógica de negócios de um aplicativo, pode obscurecer a responsabilidade do código original e dificultar a manutenção. Mover o código de percurso para iteradores designados pode ajudar a tornar o código do aplicativo mais enxuto e limpo.
- **Utilize o Iterator quando desejar que seu código possa percorrer diferentes estruturas de dados ou quando os tipos dessas estruturas forem desconhecidos de antemão.**
- O padrão fornece algumas interfaces genéricas tanto para coleções quanto para iteradores. Dado que seu código agora usa essas interfaces, ele ainda funcionará se você passar para ele vários tipos de coleções e iteradores que implementam essas interfaces.

Como implementar - Iterator

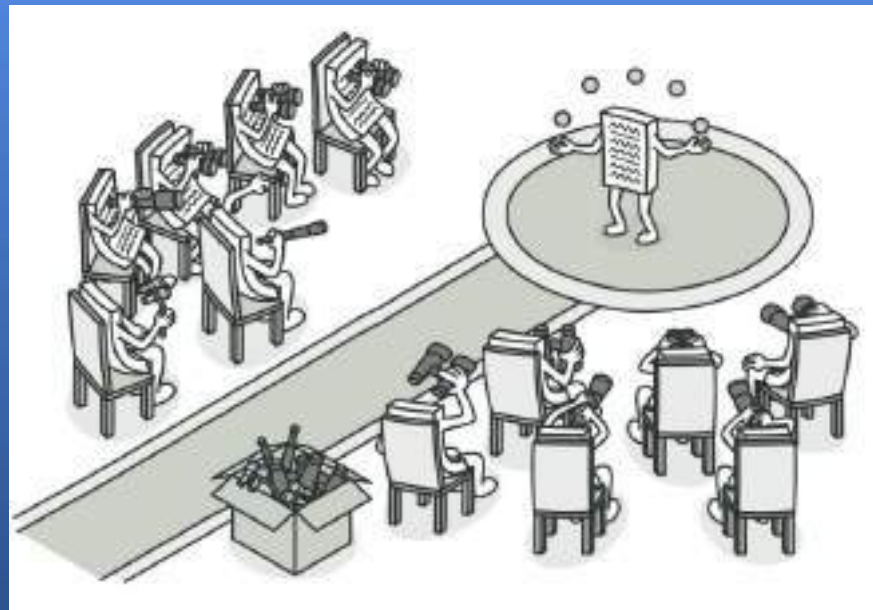


- Declare a interface do iterador. No mínimo, ela deve ter um método para buscar o próximo elemento de uma coleção. Mas, para maior conveniência, você pode adicionar alguns outros métodos, como buscar o elemento anterior, rastrear a posição atual e verificar o fim da iteração.
- Declare a interface da coleção e descreva um método para buscar iteradores. O tipo de retorno deve ser igual ao da interface do iterador. Você pode declarar métodos semelhantes se planeja ter vários grupos distintos de iteradores.
- Implemente classes de iteradores concretas para as coleções que você deseja que sejam percorríveis com iteradores. Um objeto iterador deve ser vinculado a uma única instância de coleção. Normalmente, essa vinculação é estabelecida por meio do construtor do iterador.
- Implemente a interface de coleção em suas classes de coleção. A ideia principal é fornecer ao cliente um atalho para criar iteradores, personalizados para uma classe de coleção específica. O objeto de coleção deve passar a si mesmo para o construtor do iterador para estabelecer uma ligação entre eles.
- Analise o código do cliente para substituir todo o código de percurso da coleção pelo uso de iteradores. O cliente busca um novo objeto iterador cada vez que precisa iterar sobre os elementos da coleção.

Padrões Comportamentais - Observer



É um padrão de projeto que permite definir um mecanismo de assinatura para notificar vários objetos sobre quaisquer eventos que ocorram com o objeto que eles estão observando.



Problemática

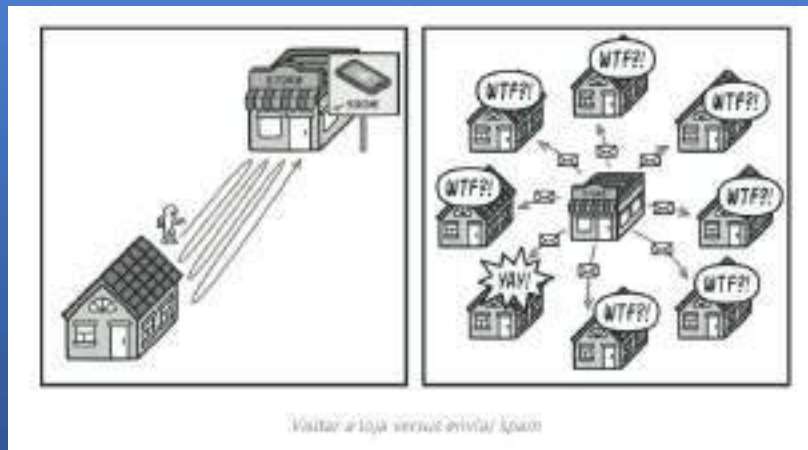


Imagine que você tem dois tipos de objetos: um `Customer` e outro `Store`. O cliente está muito interessado em uma determinada marca de produto (digamos, um novo modelo do iPhone) que deverá estar disponível na loja em breve.

O cliente poderia visitar a loja todos os dias e verificar a disponibilidade do produto. Mas, enquanto o produto estiver a caminho, a maioria dessas visitas seria inútil.

Por outro lado, a loja poderia enviar inúmeros e-mails (que poderiam ser considerados spam) para todos os clientes cada vez que um novo produto fosse lançado. Isso evitaria que alguns clientes tivessem que ir à loja inúmeras vezes. Ao mesmo tempo, isso desagradaria outros clientes que não têm interesse em novos produtos.

Parece que temos um conflito. Ou o cliente perde tempo verificando a disponibilidade do produto, ou a loja desperdiça recursos notificando os clientes errados.



Case 1 - Solução



O objeto que possui um estado relevante é frequentemente chamado de *sujeito* , mas como ele também notifica outros objetos sobre as mudanças em seu estado, vamos chamá-lo de *publicador* . Todos os outros objetos que desejam acompanhar as mudanças no estado do publicador são chamados de *assinantes* .

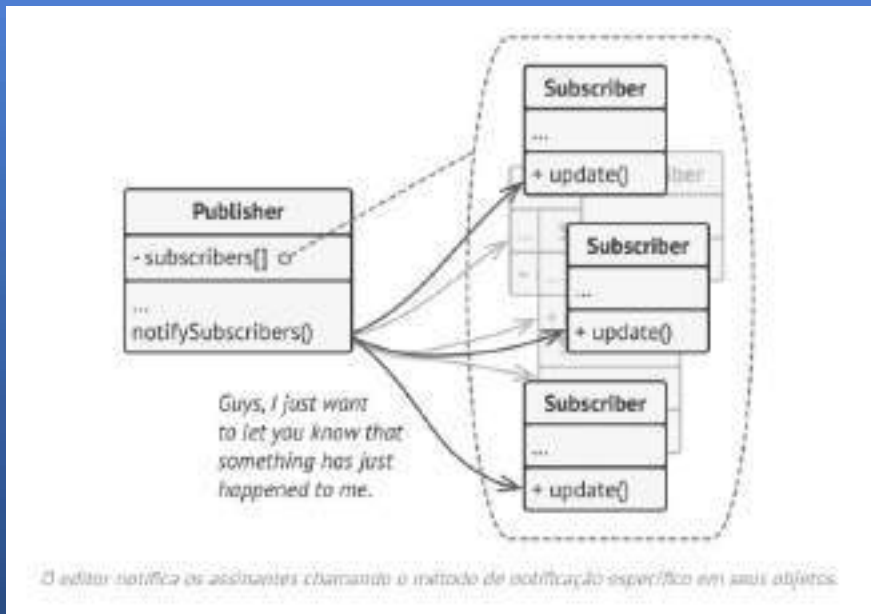
O padrão Observer sugere que você adicione um mecanismo de assinatura à classe publicadora para que objetos individuais possam se inscrever ou cancelar a assinatura de um fluxo de eventos provenientes dessa publicadora. Não se preocupe! Tudo não é tão complicado quanto parece. Na realidade, esse mecanismo consiste em 1) um campo de array para armazenar uma lista de referências a objetos assinantes e 2) vários métodos públicos que permitem adicionar e remover assinantes dessa lista.



Agora, sempre que um evento importante acontece com o editor, ele percorre seus assinantes e chama o método de notificação específico em seus objetos.

Aplicativos reais podem ter dezenas de classes de assinantes diferentes interessadas em rastrear eventos da mesma classe publicadora. Você não gostaria de acoplar a classe publicadora a todas essas classes. Além disso, você pode nem saber da existência de algumas delas antecipadamente, caso sua classe publicadora seja destinada a ser usada por outras pessoas.

Por isso, é crucial que todos os assinantes implementem a mesma interface e que o publicador se comunique com eles somente por meio dessa interface. Essa interface deve declarar o método de notificação juntamente com um conjunto de parâmetros que o publicador pode usar para passar dados contextuais junto com a notificação.



Se seu aplicativo possui vários tipos diferentes de editores e você deseja que seus assinantes sejam compatíveis com todos eles, você pode ir ainda mais longe e fazer com que todos os editores sigam a mesma interface. Essa interface precisaria descrever apenas alguns métodos de assinatura. A interface permitiria que os assinantes observassem os estados dos editores sem se acoplarem às suas classes concretas.

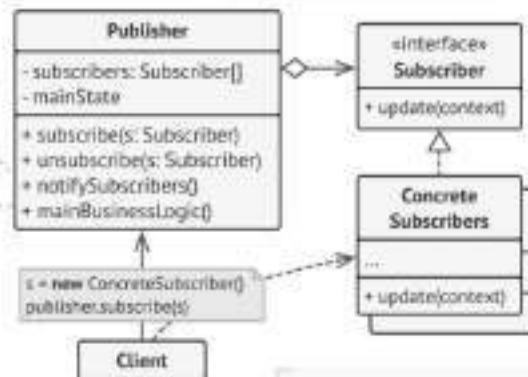
1 O **Publisher** emite eventos de interesse para outros objetos. Esses eventos ocorrem quando o **publisher** altera seu estado ou quando determinados comportamentos. Os **Publishers** contém uma infraestrutura de assinatura que permite que novos assinantes entrem e que os assinantes atuais saiam da lista.

2 Quando ocorre um novo evento, o editor percorre a lista de assinantes e chama o método de notificação declarado na interface do assinante em cada objeto assinante.

3 A interface **Subscriber** declara a interface de notificação. Na maioria dos casos, ela consiste em um único método. O método pode ter vários parâmetros que permitem ao **publisher** passar alguns detalhes do evento juntamente com a atualização.

4 Os assinantes concretos executam algumas ações em resposta às notificações emitidas pelo **publisher**. Todas essas classes devem implementar a mesma interface para que o **publisher** não fique acoplado às classes concretas.

6 O cliente cria objetos de editor e assinante separadamente e, em seguida, registra os assinantes para receber atualizações do editor.



5 Normalmente, os assinantes precisam de algumas informações contextuais para processar a atualização corretamente. Por esse motivo, os editores costumam passar alguns dados contextuais como argumentos do método de notificação. O editor pode passar a si mesmo como argumento, permitindo que o assinante busque diretamente os dados necessários.

Aplicabilidade - Observer



- **Utilize o padrão Observer quando as mudanças no estado de um objeto poderem exigir a alteração de outros objetos, e o conjunto real de objetos for desconhecido antecipadamente ou mudar dinamicamente.**
- Você pode se deparar com esse problema frequentemente ao trabalhar com classes da interface gráfica do usuário. Por exemplo, você criou classes de botões personalizadas e deseja permitir que os clientes associam algum código personalizado a esses botões para que ele seja executado sempre que um usuário pressionar um botão.
- O padrão Observer permite que qualquer objeto que implemente a interface Subscriber se inscreva para receber notificações de eventos em objetos Publisher. Você pode adicionar o mecanismo de inscrição aos seus botões, permitindo que os clientes conectem seu código personalizado por meio de classes Subscriber personalizadas.
- **Utilize esse padrão quando alguns objetos em seu aplicativo precisar observar outros, mas apenas por um período limitado ou em casos específicos.**
- A lista de assinantes é dinâmica, permitindo que os assinantes entrem ou saiam da lista quando precisarem.

Como implementar - Observer



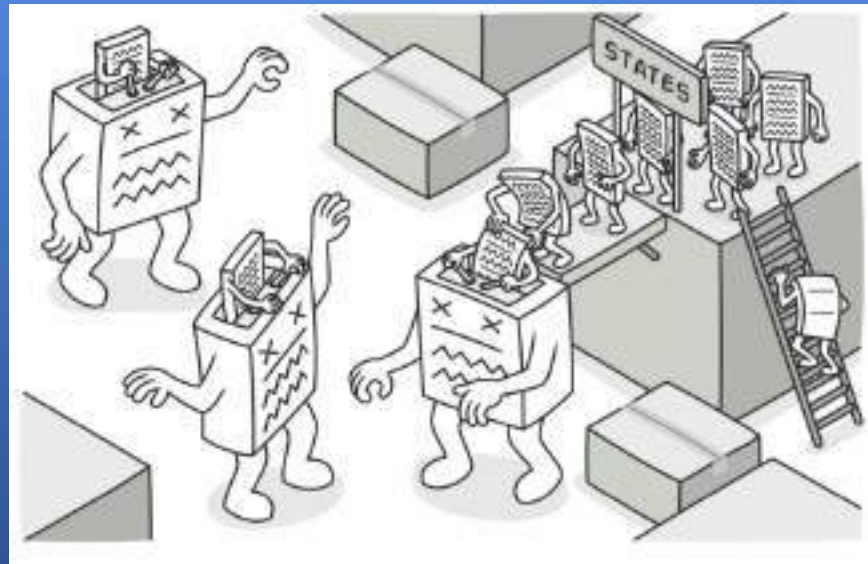
- Analise a sua lógica de negócios e tente dividi-la em duas partes: a funcionalidade principal, independente de outros códigos, atuará como publicadora; o restante se transformará em um conjunto de classes de assinantes.
- Declare a interface do assinante. No mínimo, ela deve declarar um único `update` método.
- Declare a interface do editor e descreva um par de métodos para adicionar e remover um objeto assinante da lista. Lembre-se de que os editores devem interagir com os assinantes somente por meio da interface do assinante.
- Decida onde colocar a lista de assinantes e a implementação dos métodos de assinatura. Normalmente, esse código é semelhante para todos os tipos de publicadores, então o local mais óbvio para colocá-lo é em uma classe abstrata derivada diretamente da interface do publicador. Publicadores concretos estendem essa classe, herdando o comportamento de assinatura. No entanto, se você estiver aplicando o padrão a uma hierarquia de classes existente, considere uma abordagem baseada em composição: coloque a lógica de assinatura em um objeto separado e faça com que todos os publicadores reais o utilizem.

- Crie classes de publicação concretas. Sempre que algo importante acontecer dentro de uma classe de publicação, ela deve notificar todos os seus assinantes.
- Implemente os métodos de notificação de atualização em classes de assinante concretas. A maioria dos assinantes precisará de alguns dados contextuais sobre o evento. Esses dados podem ser passados como argumento do método de notificação.
Mas existe outra opção. Ao receber uma notificação, o assinante pode obter os dados diretamente da notificação. Nesse caso, o publicador deve passar a si mesmo através do método de atualização. A opção menos flexível é vincular o publicador ao assinante permanentemente através do construtor.
- O cliente deve criar todos os assinantes necessários e registrá-los junto às editoras apropriadas.

Padrões Comportamentais - State



É um padrão de projeto que permite que um objeto altere seu comportamento quando seu estado interno muda. Aparece ser como se o objeto tivesse mudado de class

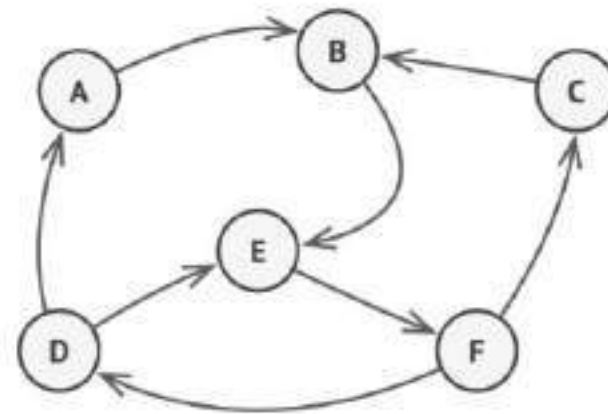


Problemática



O padrão State está intimamente relacionado ao conceito de uma **Máquina de Estados Finitos**.

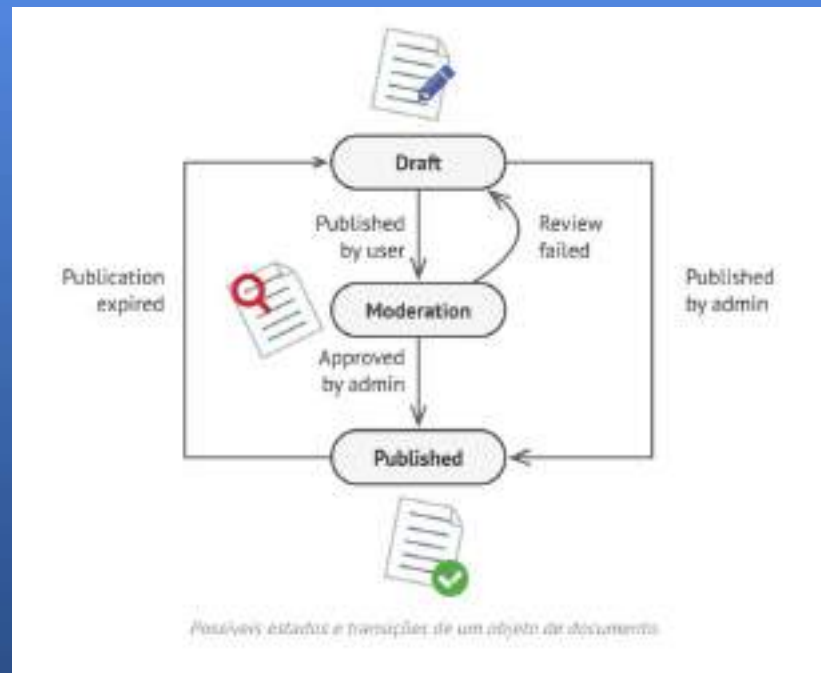
A ideia principal é que, a qualquer momento, existe um número *finito de estados* em que um programa pode estar. Dentro de cada estado único, o programa se comporta de maneira diferente e pode ser alternado instantaneamente entre um estado e outro. No entanto, dependendo do estado atual, o programa pode ou não alternar para outros estados específicos. Essas regras de alternância, chamadas de *transições*, também são finitas e predeterminadas.



Máquina de estados finitos

Você também pode aplicar essa abordagem a objetos. Imagine que temos uma `Document` classe. Um documento pode estar em um de três estados: `Draft`, `Moderation` e `Published`. O `publish` método do documento funciona de maneira um pouco diferente em cada estado:

- Em seguida `Draft`, o documento é encaminhado para moderação.
- Nesse caso `Moderation`, o documento se torna público, mas somente se o usuário atual for um administrador.
- Em `Published`, não faz absolutamente nada.



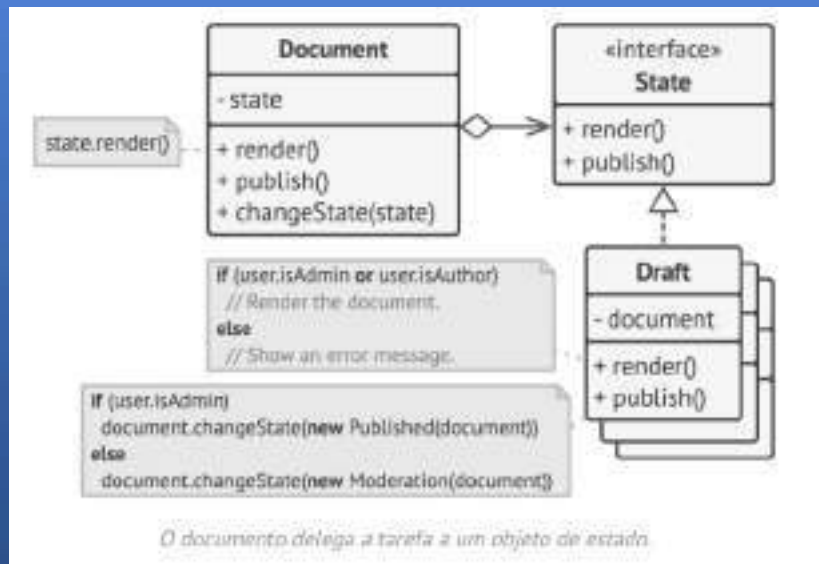
Case 1 - Solução



O padrão State sugere que você crie novas classes para todos os estados possíveis de um objeto e extraia todos os comportamentos específicos de cada estado para essas classes.

Em vez de implementar todos os comportamentos por conta própria, o objeto original, chamado *de contexto*, armazena uma referência a um dos objetos de estado que representa seu estado atual e delega todo o trabalho relacionado ao estado a esse objeto.

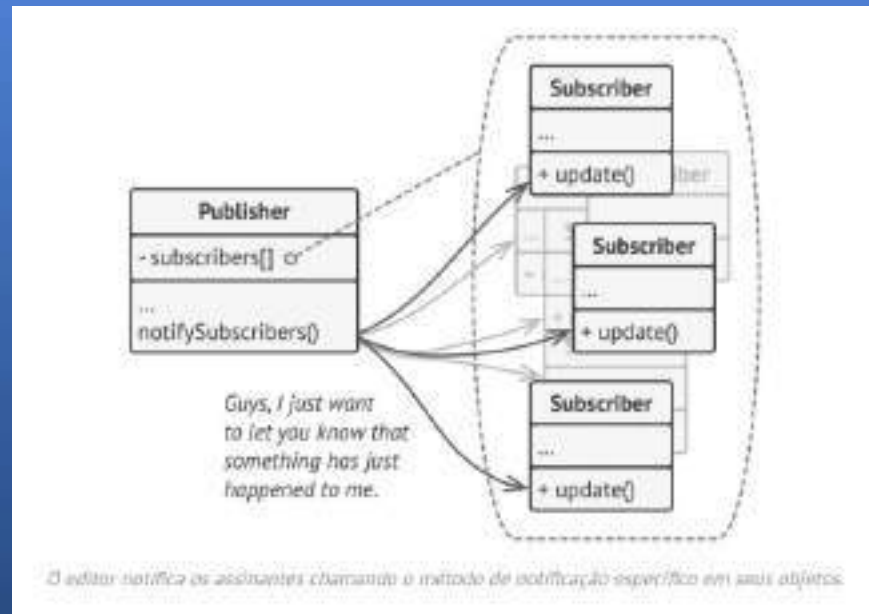
Para fazer a transição do contexto para outro estado, substitua o objeto de estado ativo por outro objeto que represente esse novo estado. Isso só é possível se todas as classes de estado seguirem a mesma interface e o próprio contexto interagir com esses objetos por meio dessa interface.



Agora, sempre que um evento importante acontece com o editor, ele percorre seus assinantes e chama o método de notificação específico em seus objetos.

Aplicativos reais podem ter dezenas de classes de assinantes diferentes interessadas em rastrear eventos da mesma classe publicadora. Você não gostaria de acoplar a classe publicadora a todas essas classes. Além disso, você pode nem saber da existência de algumas delas antecipadamente, caso sua classe publicadora seja destinada a ser usada por outras pessoas.

Por isso, é crucial que todos os assinantes implementem a mesma interface e que o publicador se comunique com eles somente por meio dessa interface. Essa interface deve declarar o método de notificação juntamente com um conjunto de parâmetros que o publicador pode usar para passar dados contextuais junto com a notificação.



Se seu aplicativo possui vários tipos diferentes de editores e você deseja que seus assinantes sejam compatíveis com todos eles, você pode ir ainda mais longe e fazer com que todos os editores sigam a mesma interface. Essa interface precisaria descrever apenas alguns métodos de assinatura. A interface permitiria que os assinantes observassem os estados dos editores sem se acoplarem às suas classes concretas.

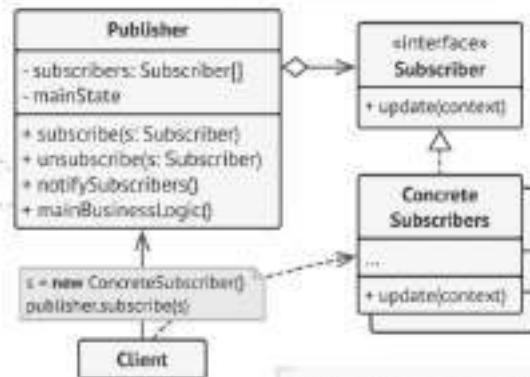
1. O **Publisher** cria eventos de interesse para outros objetos. Esses eventos ocorrem quando o **publisher** altera seu estado ou em uma determinada comportamento. Os **Publishers** contém uma infraestrutura de assinatura que permite que vários assinantes sintam e que os assinantes atualizem sua lista.

2. Quando ocorre um novo evento, o editor percorre a lista de assinantes e chama o método de notificação declarado na interface do assinante em cada objeto assinante.

3. A interface **Subscriber** declara a interface de notificação. Na maioria dos casos, ela consiste em um único método. O método pode ter vários parâmetros que permitem ao **publisher** passar alguns detalhes do evento juntamente com a atualização.

4. Os assinantes concretos executam algumas ações em resposta às notificações emitidas pelo **publisher**. Todas essas classes devem implementar a mesma interface para que o **publisher** não fique acoplado às classes concretas.

6. O cliente cria objetos de editor e assinante separadamente e, em seguida, registra os assinantes para receber atualizações do editor.



5. Normalmente, os assinantes precisam de algumas informações contextuais para processar a atualização corretamente. Por esse motivo, os editores costumam passar alguns dados contextuais como argumentos do método de notificação. O editor pode passar a si mesmo como argumento, permitindo que o assinante busque diretamente os dados necessários.

Aplicabilidade - State



- **Utilize o padrão State quando você tiver um objeto que se comporta de maneira diferente dependendo do seu estado atual, o número de estados for enorme e o código específico de cada estado mudar com frequência.**
- O padrão sugere que você extraia todo o código específico de cada estado para um conjunto de classes distintas. Como resultado, você pode adicionar novos estados ou alterar os existentes independentemente uns dos outros, reduzindo o custo de manutenção.
- **Utilize esse padrão quando você tiver uma classe repleta de condicionais em grande quantidade que alteram o comportamento da classe de acordo com os valores atuais de seus campos.**
- O padrão State permite extrair ramificações dessas condicionais para métodos das classes de estado correspondentes. Ao fazer isso, você também pode remover campos temporários e métodos auxiliares envolvidos em código específico do estado da sua classe principal.
- **Use o State quando você tiver muito código duplicado em estados e transições semelhantes de uma máquina de estados baseada em condições.**
- O padrão State permite compor hierarquias de classes de estado e reduzir a duplicação, extraindo o código comum para classes base abstratas.

Como implementar - State



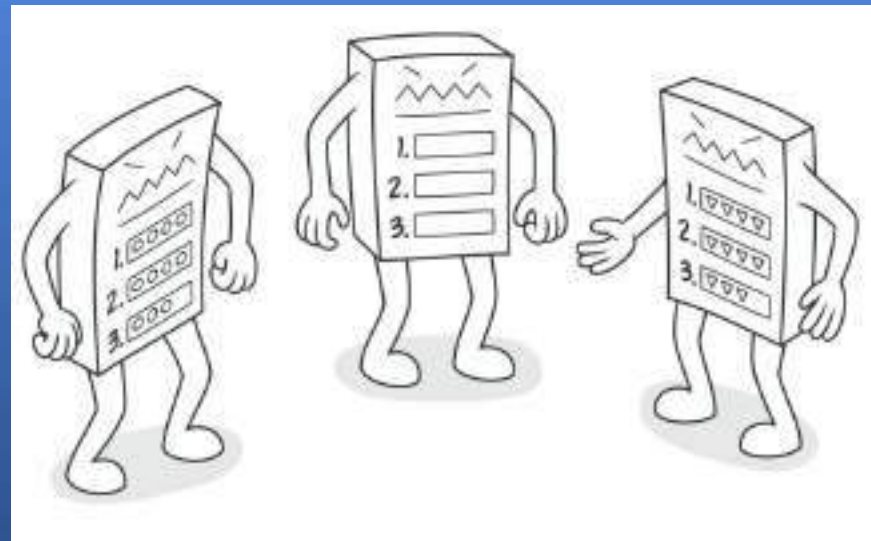
- Decida qual classe atuará como contexto. Pode ser uma classe existente que já contenha o código dependente do estado; ou uma nova classe, caso o código específico do estado esteja distribuído por várias classes.
- Declare a interface de estado. Embora ela possa espelhar todos os métodos declarados no contexto, priorize apenas aqueles que possam conter comportamentos específicos do estado.
- Para cada estado real, crie uma classe que herde da interface de estado. Em seguida, examine os métodos do contexto e extraia todo o código relacionado a esse estado para a classe recém-criada. Ao mover o código para a classe de estado, você pode descobrir que ele depende de membros privados do contexto. Existem várias soluções alternativas:
 - Torne esses campos ou métodos públicos.
 - Transforme o comportamento que você está extraindo em um método público no contexto e chame-o a partir da classe de estado. Essa abordagem é feia, mas rápida, e você sempre pode corrigi-la depois.
 - Aninhe as classes de estado na classe de contexto, mas somente se sua linguagem de programação suportar aninhamento de classes.

- Na classe de contexto, adicione um campo de referência do tipo de interface de estado e um método setter público que permita sobrescrever o valor desse campo.
- Revise o método do contexto e substitua as condicionais de estado vazias por chamadas aos métodos correspondentes do objeto de estado.
- Para alterar o estado do contexto, crie uma instância de uma das classes de estado e passe-a para o contexto. Você pode fazer isso dentro do próprio contexto, em vários estados ou no cliente. Independentemente de onde isso seja feito, a classe se torna dependente da classe de estado concreta que ela instancia.

Padrões Comportamentais - Template Method



É um padrão de projeto que define o esqueleto de um algoritmo na superclasse, mas permite que as subclasses sobrescrevem etapas específicas do algoritmo sem alterar sua estrutura.

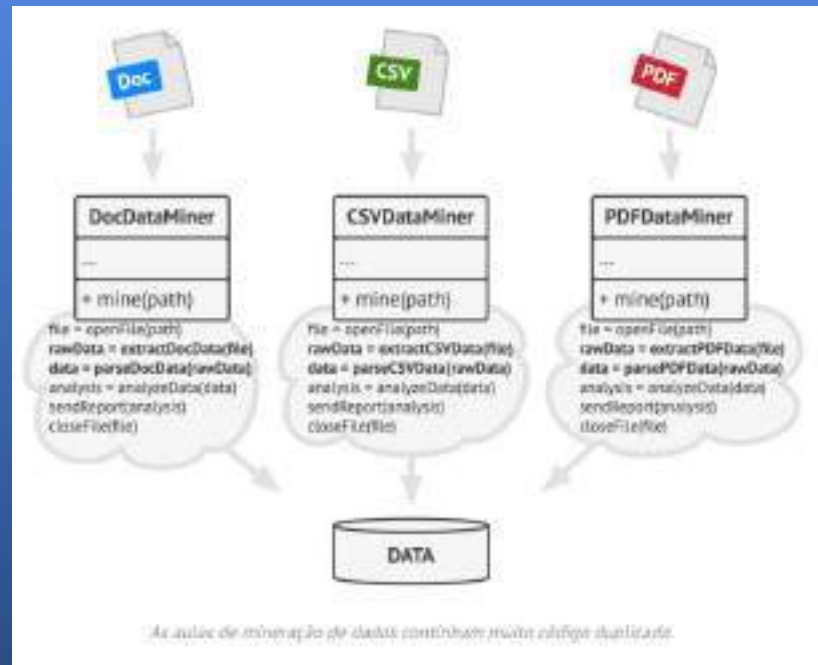


Problemática



Imagine que você está criando um aplicativo de mineração de dados que analisa documentos corporativos. Os usuários fornecem ao aplicativo documentos em vários formatos (PDF, DOC, CSV), e ele tenta extrair dados relevantes desses documentos em um formato uniforme.

A primeira versão do aplicativo funcionava apenas com arquivos DOC. Na versão seguinte, passou a suportar arquivos CSV. Um mês depois, você o "ensinou" a extrair dados de arquivos PDF.



Em algum momento, você percebeu que as três classes têm muito código semelhante. Embora o código para lidar com vários formatos de dados seja completamente diferente em cada classe, o código para processamento e análise de dados é quase idêntico. Não seria ótimo eliminar a duplicação de código, mantendo a estrutura do algoritmo intacta?

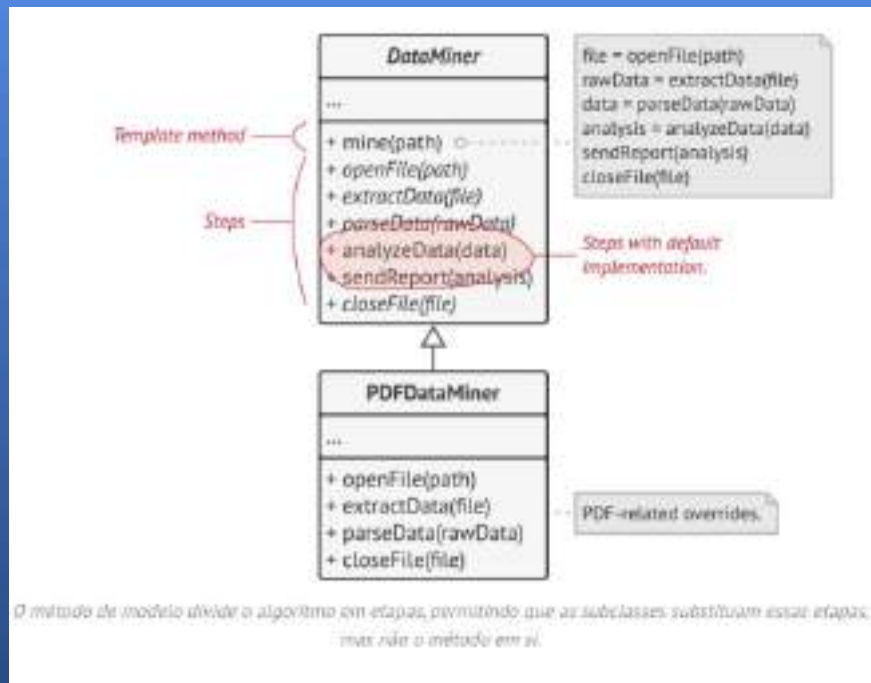
Havia outro problema relacionado ao código do cliente que utilizava essas classes. Ele continha muitas condicionais que escolhiam a ação apropriada dependendo da classe do objeto de processamento. Se todas as três classes de processamento tivessem uma interface comum ou uma classe base, seria possível eliminar as condicionais no código do cliente e usar polimorfismo ao chamar métodos em um objeto de processamento.

Case 1 - Solução



O padrão Template Method sugere que você decomponha um algoritmo em uma série de etapas, transforme essas etapas em métodos e coloque uma série de chamadas a esses métodos dentro de um único *método modelo*. As etapas podem ser abstratas `abstract` ou ter alguma implementação padrão. Para usar o algoritmo, o cliente deve fornecer sua própria subclasse, implementar todas as etapas abstratas e sobrescrever algumas das opcionais, se necessário (mas não o próprio método modelo).

Vamos ver como isso se comportará em nosso aplicativo de mineração de dados. Podemos criar uma classe base para todos os três algoritmos de análise sintática. Essa classe define um método de modelo que consiste em uma série de chamadas para várias etapas de processamento de documentos.



Inicialmente, podemos declarar todas as etapas *abstract*, forçando as subclasses a fornecerem suas próprias implementações para esses métodos. No nosso caso, as subclasses já possuem todas as implementações necessárias, então a única coisa que talvez precisemos fazer é ajustar as assinaturas dos métodos para que correspondam aos métodos da superclasse.

Agora, vamos ver o que podemos fazer para eliminar o código duplicado. Parece que o código para abrir/fechar arquivos e extrair/analisar dados é diferente para vários formatos de dados, então não há motivo para mexer nesses métodos. No entanto, a implementação de outras etapas, como analisar os dados brutos e gerar relatórios, é muito semelhante, então pode ser incorporada à classe base, onde as subclasses podem compartilhar esse código.

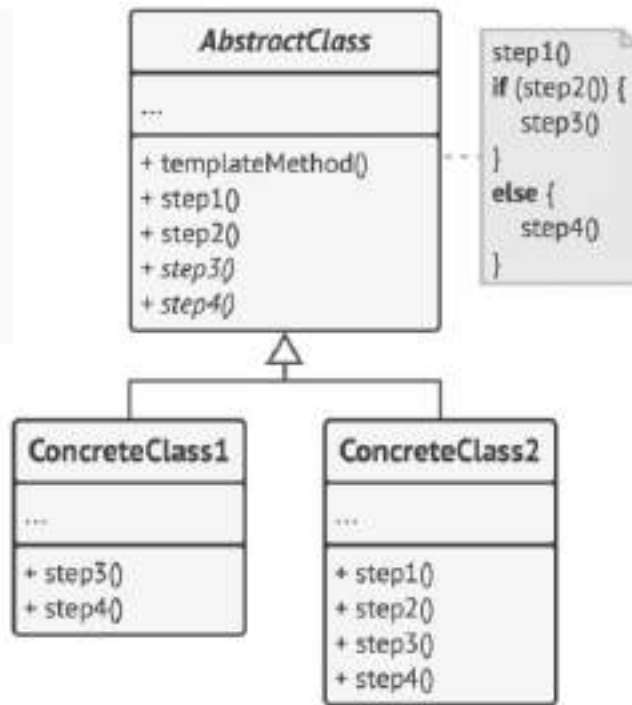
Como você pode ver, temos dois tipos de degraus:

- *Etapas abstratas* devem ser implementadas por cada subclasse.
- *As etapas opcionais* já possuem alguma implementação padrão, mas ainda podem ser substituídas, se necessário.

Existe outro tipo de etapa, chamada de *gancho* (ou *hook*) . Um gancho é uma etapa opcional com um corpo vazio. Um método de modelo funcionaria mesmo que um gancho não fosse sobrescrito. Normalmente, os ganchos são colocados antes e depois de etapas cruciais de algoritmos, fornecendo às subclasses pontos de extensão adicionais para um algoritmo.

1. A classe abstrata declara métodos que atuam como etapas de um algoritmo, bem como o método modelo que chama esses métodos em uma ordem específica. As etapas podem ser declaradas `abstract` ou ter alguma implementação padrão.

2. Classes concretas podem sobrescrever todas as etapas, mas não o próprio método do modelo.



Aplicabilidade - Template Method



- **Utilize o padrão Template Method quando desejar permitir que os clientes estendam apenas etapas específicas de um algoritmo, e não o algoritmo inteiro ou sua estrutura.**
- O método Template permite transformar um algoritmo monolítico em uma série de etapas individuais que podem ser facilmente estendidas por subclasses, mantendo intacta a estrutura definida na superclasse.
- **Utilize esse padrão quando você tiver várias classes que contêm algoritmos quase idênticos com algumas pequenas diferenças. Consequentemente, você poderá precisar modificar todas as classes quando o algoritmo for alterado.**
- Ao transformar um algoritmo desse tipo em um método de modelo, você também pode incorporar as etapas com implementações semelhantes em uma superclasse, eliminando a duplicação de código. O código que varia entre subclasses pode permanecer nas subclasses.

Como implementar - Template Method



- Analise o algoritmo alvo para verificar se é possível dividi-lo em etapas. Considere quais etapas são comuns a todas as subclasses e quais serão sempre exclusivas.
- Crie a classe base abstrata e declare o método modelo e um conjunto de métodos abstratos que representam os passos do algoritmo. Descreva a estrutura do algoritmo no método modelo executando os passos correspondentes. Considere criar um método modelo *final* que impeça as subclasses de sobrescrevê-lo.
- Não há problema se todas as etapas acabarem sendo abstratas. No entanto, algumas etapas podem se beneficiar de uma implementação padrão. As subclasses não precisam implementar esses métodos.
- Considere adicionar pontos de conexão entre as etapas cruciais do algoritmo.
- Para cada variação do algoritmo, crie uma nova subclasse concreta. Ela *deve* implementar todas as etapas abstratas, mas também *pode* sobrescrever algumas das etapas opcionais.

Vamos prática?





OXETECH Academy



SECTI
Secretaria de Estado de
Tecnologia e Inovação



 Softex

REGISTERED BY
CIENCIA, TECNOLOGIA
E INOVACAO

GOVERNHO FEDERAL
BRASIL
Ministério da Educação